

React + PHP: The Thin Stack

Il protocollo miniCMS per Web App moderne

Simone Pizzi

Terza Edizione

RUNTIME EDIZIONI

React + PHP: The Thin Stack
Il protocollo miniCMS per Web App moderne
Terza Edizione — Giugno 2026

© 2026 Simone Pizzi
© 2026 Runtime Edizioni

Tutti i diritti riservati. Nessuna parte di questo libro può essere riprodotta senza il consenso scritto dell'editore, salvo brevi citazioni in recensioni.

I marchi e i nomi di prodotto citati sono dei rispettivi proprietari.
www.runtimeradio.com

A Valerio Galano, perché lui è come Neo, vede mondi nel codice e mi ha insegnato a ragionare in un modo diverso. Chissà se un po' anche io gli ho insegnato qualcosa.

A Giuseppe Pugliese che, anche se vede il mondo in un modo tutto suo, ritiene sia un orgoglio essere uno sviluppatore web e persevera nella sua arte con passione.

Indice

La Visione	9
CAPITOLO 1: Manifesto	10
L'Architettura	15
CAPITOLO 2: Architettura e Struttura Progetto	16
CAPITOLO 3: Database Strategy	20
CAPITOLO 4: Frontend Dependencies	25
I Componenti	31
CAPITOLO 5: Backend Logic (PHP)	32
CAPITOLO 6: Frontend Bridge (API.ts)	37
CAPITOLO 7: Media & Optimization	45
CAPITOLO 8: Advanced Content Editing	53
Il Flusso Operativo	63
CAPITOLO 9: Content Lifecycle	64
CAPITOLO 10: Security & Auth	69
CAPITOLO 11: SEO Pre-rendering con PHP Entry-Point	83
CAPITOLO 12: RSS Feed & Syndication	92
CAPITOLO 13: Newsletter & Email System	100
CAPITOLO 14: Admin Dashboard & Panels	108
I Casi Reali	115
CAPITOLO 15: Database Evolution - Da SQLite a MySQL	116
CAPITOLO 16: Portfolio & Projects Module	126
CAPITOLO 17: Festival Logic - Iscrizioni e Workflow Approvazione	130
CAPITOLO 18: Festival Logic - Votazioni e Protezione Anti-Frode	133
CAPITOLO 19: Festival Logic - Dashboard Admin, Settings e Reporting	137
CAPITOLO 20: Social Interactions & Reactions	140
Appendici	147
APPENDICE A - Boilerplate Checklist: Avvio Nuovo Progetto miniCMS	148
APPENDICE B: Ciclo di vita di un fork	152
APPENDICE C: Testing e Deploy (cenni)	156

P A R T E I

La Visione

Il perché. La filosofia che guida ogni decisione tecnica.

CAPITOLO 1: Manifesto

Perché Esiste Questo Protocollo

Esiste una tensione irrisolta al centro dello sviluppo web moderno.

Da un lato, il frontend ha raggiunto una maturità estetica e funzionale straordinaria: React, TypeScript, Tailwind. Animazioni fluide, componenti riutilizzabili, type safety, hot reload. L'esperienza di sviluppo è diventata un piacere, e il prodotto finale, quando fatto bene, è visivamente e funzionalmente superiore a qualsiasi soluzione del passato.

Dall'altro lato, questa rivoluzione ha portato con sé una complessità infrastrutturale sproporzionata rispetto ai bisogni reali della maggioranza dei siti. Node.js, database cloud, container, pipeline CI/CD, micro-servizi, CMS headless con piani in abbonamento. L'overhead tecnico e il costo operativo sono diventati la norma anche per siti che potrebbero girare perfettamente su un hosting da cinque euro al mese.

Questo protocollo nasce da una domanda precisa: **è possibile avere la potenza estetica e tecnica di React senza abbandonare la semplicità, il controllo e l'economicità di un backend PHP con SQLite?**

La risposta, costruita su mesi di lavoro reale su progetti reali, è sì.

Il Principio Fondativo: La Separazione dei Piani

Il Modello Universale non è una tecnologia. È un'architettura mentale.

Separa con nettezza due piani che spesso vengono confusi:

Il Piano della Presentazione appartiene a React. È il luogo della forma, dell'interazione, dell'animazione, della tipografia, della palette colori. È dove vive il talento estetico, dove Tailwind traduce l'intenzione visiva in CSS preciso, dove framer-motion aggiunge peso e respiro ai movimenti. Questo piano è compilato, ottimizzato, servito come asset statico.

Il Piano dei Dati appartiene a PHP e SQLite (o MySQL quando necessario). È il luogo della persistenza, della logica di business, della sicurezza, del ciclo di vita dei contenuti. Non è «il backend» nel senso pesante del termine: nessun framework, nessun ORM, nessuna dipendenza esterna. Solo PHP nativo, PDO, e un database file-based che non richiede configurazione server.

Questi due piani comunicano attraverso un contratto preciso: le API REST. Il frontend non sa niente del database. Il backend non sa niente di React. La loro separazione è la fonte di tutta la scalabilità e la manutenibilità del sistema.

★

★★

La Scala, non l'Assenza

«Thin stack» non significa «backend assente». Significa un backend ridotto all'osso ma vero, e significa soprattutto una scala. Lo stesso scheletro (PHP nativo, un singleton PDO per richiesta, un file per endpoint, niente framework) si declina su gradini diversi a seconda di cosa serve. Al grado-zero c'è SQLite, un database che è un singolo file, senza un server da configurare né un segreto da custodire. Un gradino sopra c'è MySQL essenziale, quando i dati o il traffico lo richiedono. Più su ancora c'è MySQL ingegnerizzato, con connessioni rinforzate, prelude condivisi e scaffolding dedicato. Non sono tre architetture diverse: sono lo stesso modello a tre altezze.

Questo manuale è costruito su quattro siti reali che occupano punti diversi di quella scala, e li legge proprio così. Quando un capitolo mostra «tre modi di fare la stessa cosa», non sta elencando opzioni a caso: sta misurando quanto si può togliere, o aggiungere, allo stesso scheletro prima che cambi natura. La scala a tre gradini, dal grado-zero all'ingegnerizzato, è la chiave di lettura del libro intero.

★

★★

Cosa Non È Questo Protocollo

Non è un framework. Non impone strutture di codice rigide, non richiede dipendenze specifiche, non vincola le scelte stilistiche.

Non è un CMS tradizionale. Non c'è un'interfaccia visual builder, non ci sono temi preconfezionati, non c'è un marketplace di plugin. Ogni sito costruito con questo protocollo è unico, fatto a mano, su misura per il suo scopo.

Non è una soluzione enterprise. Non è progettato per gestire milioni di utenti simultanei, flussi di dati complessi o architetture distribuite. È progettato per siti che devono essere eccellenti, veloci, sicuri e mantenibili da un team piccolo, o anche da una persona sola.

Non è per chi vuole un sito in dieci minuti. È per chi vuole capire cosa sta costruendo.

★

★★

Quando NON Usare Questo Protocollo

Un manifesto onesto deve dire anche dove finisce. Il Thin Stack scambia la complessità dell'infrastruttura con la disciplina di chi lo scrive: toglie il framework e mette al suo posto la tua attenzione. Quando quel baratto non conviene, conviene un altro stack. Ecco i casi in cui sceglierei altro senza esitare.

Quando il team è grande. Le convenzioni qui non sono imposte da un framework, sono tenute insieme dalle persone. Con uno o due sviluppatori funziona; oltre i quattro o cinque, l'assenza di una struttura rigida diventa un costo, non una libertà, e un framework opinato (Laravel, Next.js con le sue regole) ripaga la curva di apprendimento.

Quando serve il tempo reale. Chat dal vivo, notifiche push istantanee, presenza, collaborazione simultanea su uno stesso documento: sono carichi che vogliono WebSocket e processi persistenti, non il modello richiesta-risposta di PHP su hosting condiviso. Si possono forzare, ma è la strada in salita.

Quando la scala è davvero alta. Decine di migliaia di richieste al secondo, picchi imprevedibili, necessità di scalare orizzontalmente su più nodi: qui servono code, cache distribuite e database gestiti. Il Capitolo 3 mette dei numeri concreti sotto questa frase, così la soglia non resta un'opinione.

Quando i dati sono complessi e molto relazionali. Transazioni distribuite, reportistica analitica pesante, modelli con decine di entità interconnesse: a un certo punto un ORM e un RDBMS ingegnerizzato non sono overhead, sono lo strumento giusto.

Quando la conformità è un requisito esplicito. Audit formali, certificazioni, ambienti regolati che pretendono framework e librerie con supporto commerciale e una catena di responsabilità documentata: il fai-da-te disciplinato, qui, è un rischio che non vale la pena correre.

La regola che tiene insieme questi casi è semplice: il Thin Stack è eccellente finché la complessità del problema sta sotto la complessità che un framework imporrebbe. Quando la supera, il framework smette di essere un peso e diventa una rete. Riconoscere quel punto di sorpasso è parte della stessa onestà tecnica che attraversa il resto del libro.

**

I Valori che Guidano Ogni Decisione

Controllo totale. Chi costruisce un sito con questo protocollo possiede ogni riga del suo stack. Nessun vendor lock-in, nessun aggiornamento forzato che rompe la produzione, nessuna dipendenza da un servizio esterno per la sopravvivenza del sito.

Leggerezza come principio, non come compromesso. SQLite non è la scelta «economica» rispetto a MySQL: è la scelta giusta per il 90% dei casi d'uso. La semplicità non è una limitazione da superare, è un obiettivo da raggiungere e difendere.

La sicurezza come architettura, non come patch. Le decisioni di sicurezza sono integrate nel design del sistema: database fuori dalla root pubblica, uno script di build che non lascia mai il database nel deploy, sessioni PHP con cookie HttpOnly, password mai in chiaro. Non sono misure aggiunte dopo, sono la struttura stessa.

Più ingegnerizzato non vuol dire più sicuro. È la lezione più scomoda che questi siti insegnano, e ritorna in quasi ogni capitolo. Il sito tecnicamente più ricco, con le connessioni più blindate e l'infrastruttura più curata, è spesso quello con i fondamen-

tali più fragili: una password di default scritta nel codice, nessun backup automatico, un argine anti-abuso che manca proprio dove servirebbe. Aggiungere complessità non paga interessi di sicurezza da sola, e a volte distrae dalle due righe che contano davvero. Diffidare dell'equazione «più strati uguale più sicuro» è uno dei fili che tengono insieme questo manuale.

La documentazione come parte del codice. Un sistema che non si capisce è un sistema che non si può mantenere. Questo protocollo è documentato con la stessa cura con cui è costruito, perché la conoscenza deve rimanere accessibile anche quando il contesto cambia.

L'esperienza reale come unico validatore. Ogni pattern documentato in questo manuale è stato estratto da codice che gira in produzione. Le lezioni più importanti vengono da incidenti veri, non da scenari ipotetici: il crash notturno del WAL che ha costretto Runtime Radio a migrare d'urgenza su MySQL, l'ondata di bot che ne ha saturato l'entry-point. La teoria senza la cicatrice non insegna abbastanza.

*
**

A Chi È Rivolto

A chiunque voglia costruire un sito web che sia una cosa viva: non un template, non un WordPress personalizzato, non un sito sputato fuori da un builder.

Al developer che conosce React e vuole un backend senza dover imparare un framework intero.

Al freelance che deve consegnare un sito veloce, mantenibile e sicuro a un cliente che non ha budget per infrastrutture cloud.

All'autore, al musicista, al festival, alla radio che vuole una presenza digitale propria, controllata, indipendente dai capricci delle piattaforme.

A chiunque creda che il web possa essere ancora un posto fatto da persone, per persone, senza intermediari.

*
**

Due Voci: «Dal vivo» e «Il Canone»

Questo manuale fa una cosa che la maggior parte dei testi tecnici evita: mostra il codice reale com'è, non come dovrebbe essere. È la sua forza, ma può confondere, perché in una stessa pagina convivono due cose diverse, e vanno tenute distinte.

La prima voce è «**dal vivo**»: la fotografia di cosa fanno davvero i quattro siti. Qui ci sono le scelte buone e anche le cicatrici, gli anti-pattern, le falle lasciate aperte. Quando il testo racconta che un sito salva l'HTML grezzo senza sanitizzarlo, o espone uno script di migrazione, non lo sta raccomandando: lo sta documentando. È il corpo di ogni capitolo, ed è scritto senza sconti proprio perché la cicatrice insegna più della teoria.

La seconda voce è «**il Canone**»: la regola da seguire, distillata. Ogni capitolo si chiude con un riquadro intitolato **Il Canone**, che separa nettamente la prescrizione dalla foto-

grafia. Lì trovi cosa fare in un progetto nuovo, ripulito dalle imperfezioni dei casi reali. Se hai dieci secondi e vuoi solo la norma, leggi quel riquadro; se vuoi capire *perché* la norma è quella, leggi il capitolo che lo precede.

NOTA

La regola di lettura, in una riga Il corpo del capitolo dice cosa il codice *fa* («dal vivo»); il riquadro finale dice cosa tu *dovresti* fare («il Canone»). Quando i due divergono, ha sempre ragione il Canone: la divergenza è il punto, non un errore di stampa.

*
**

Come Usare Questo Manuale

Il manuale è organizzato in capitoli tematici indipendenti. Non è necessario leggerlo dall'inizio alla fine: ogni capitolo è una reference autonoma.

Per iniziare un nuovo progetto da zero, la **BOILERPLATE-CHECKLIST** è il punto di partenza pratico.

Per migliorare un progetto esistente, i capitoli specifici (Database Strategy, Security & Auth, SEO Pre-rendering) offrono pattern applicabili in modo chirurgico.

Per imparare dalla storia, i capitoli con la voce esperienziale (il crash del WAL, l'attacco dei bot su Runtime Radio, la migrazione a MySQL) sono la lettura più onesta che questo manuale può offrire.

Il codice non mente. Le cicatrici nemmeno.

*
**

IMPORTANTE

Il Canone

- Separa i due piani: React compilato per la presentazione, PHP nativo con PDO per i dati, un contratto REST tra loro.
- Scegli il gradino giusto della scala (SQLite grado-zero → MySQL essenziale → MySQL ingegnerizzato), mai più di quanto serve.
- Tratta la sicurezza come architettura, non come patch, e diffida dell'equazione «più strati uguale più sicuro».
- Documenta com'è il codice, non com'è bello: la cicatrice insegna più della teoria.
- Usa il Thin Stack finché la complessità del problema resta sotto quella che un framework imporrebbe; superato quel punto, scegli il framework.

*
**

«La perfezione si raggiunge non quando non c'è più niente da aggiungere, ma quando non c'è più niente da togliere.» Antoine de Saint-Exupéry

*
**

Prossimo Capitolo: Architettura e Struttura Progetto. Dove le idee diventano cartelle.

P A R T E I I

L'Architettura

Le fondamenta. Struttura del progetto, database, stack tecnologico.

CAPITOLO 2: Architettura e Struttura Progetto

Questo capitolo definisce l'architettura fisica e logica del sistema: come sono disposte le cartelle, dove vivono i segreti, come una richiesta trova il suo file PHP, e quali difese sono cablate nella struttura stessa invece che aggiunte dopo.

1. Topologia delle Cartelle e Separazione degli Asset

Il Modello Universale impone una separazione netta tra i file sorgente (build-time) e i file di runtime (i dati che persistono).

1.1 Struttura Fisica (Root)

```
/
├── public/           # Contenuto pubblico ed entry-point delle API
│   ├── .htaccess    # Routing SPA (cruciale per React Router)
│   ├── index.php    # SEO Engine (Capitolo 11) – entry-point PHP
│   └── api/         # Core backend PHP (vedi 1.2)
├── src/             # Frontend React 19 / TS
├── scripts/         # Utility (build, clean, migration)
├── .env.local       # Configurazione locale (VITE_API_URL)
├── package.json     # Dipendenze e script di automazione
└── clean-dist.js    # Sanitizzazione post-build (SECURITY)
```

1.2 Anatomia dell'Area API (public/api/)

La prima cosa da capire di questa cartella è cosa *non* contiene: un router. Non c'è un `index.php` che smista, non c'è un kernel, non c'è una tabella di rotte. Ogni endpoint è **un file PHP autonomo**: `articles.php`, `upload.php`, `reactions.php`. Ciascuno include all'avvio i suoi mattoni (la connessione al database, l'eventuale prelude di sessione, gli header) e gestisce da sé la propria richiesta. È la scelta fondante del miniCMS, e quella che rende il «thin stack» letteralmente sottile: l'URL `/api/articles.php` è il file `articles.php`, senza intermediari. Il prezzo è qualche ripetizione tra file (lo stesso `require` in cima a ognuno); il guadagno è che ogni endpoint si legge, si testa e si sposta da solo, senza dover capire un sistema di routing.

Accanto agli endpoint vivono le cartelle di runtime, che cambiano a seconda del motore di database scelto:

- **.data/**: contiene il database **SQLite**, e quindi esiste solo nei siti che usano SQLite (come DISINTELLIGENZA). Deve contenere un `.htaccess` con `Deny from all`, perché un file `.sqlite` raggiungibile via web è il database intero scaricabile da chiunque. I siti

migrati a MySQL non hanno questa cartella: il loro database vive sul server MySQL, fuori dalla docroot per natura.

- **.cache/**: file JSON generati per le performance (le liste di contenuti, vedi Capitolo 9), invalidati a ogni scrittura.
- **uploads/**: gli asset caricati dall'utente (immagini, audio). Va esclusa dai backup del codice sorgente, e nei siti più difesi ospita un proprio **.htaccess** che **spegne PHP** (la prima barriera anti-RCE, Capitolo 7).

1.3 Configurazione e Segreti: Tre Approcci senza Librerie

I segreti non stanno mai nel sorgente versionato. Ma il modo di tenerli fuori è una piccola scala a sé, e i tre siti la occupano in tre punti diversi, tutti senza una libreria di configurazione.

```
// SPW config.php (gitignorato) - affiancato da config.example.php versionat
< o
define('DB_HOST', 'localhost');
define('DB_NAME', '...');
define('SITE_URL', 'https://...'); // costante canonica, anti host-poiso
< ning
```

```
; SR .env (gitignorato) - letto da db_credentials.php via parse_ini_file();
< accanto, .env.example
DB_HOST=localhost
TELEGRAM_BOT_TOKEN=...
SMTP_HOST=... ; l'.env è l'hub di TUTTI i segreti del sito
```

SimonePizziWebSite usa un `config.php` con delle `define()`, ignorato da git e accompagnato da un `config.example.php` committato come traccia. SitoRuntime usa un file `.env` letto con `parse_ini_file()`, ed è l'hub di tutti i suoi segreti (database, token Telegram, credenziali SMTP). DISINTELLIGENZA non ha **nessuna** configurazione: il percorso del file SQLite è scritto direttamente in `db.php`, perché un database-a-file non ha credenziali da custodire. È il principio dei «dodici fattori» (configurazione separata dal codice) applicato senza alcuna libreria, con un'eccezione radicale: chi sta al grado-zero della scala non ha proprio un segreto da gestire.

*
**

2. Sicurezza in Build: la Logica del Clean-Dist

Uno dei rischi maggiori è sovrascrivere il database di produzione durante il deploy. Il sistema lo previene con uno script (`clean-dist.js`) eseguito dopo la build, che:

1. analizza la cartella `dist/api/`;
2. rimuove ricorsivamente ogni file con estensione `.sqlite`, `.sqlite3`, `.db` o `.bak`;
3. avvisa l'operatore con un log di sicurezza (□ SECURITY: Removed...);
4. garantisce, come **regola progettuale**, che la `dist/` sia «database-free». Il database va inizializzato sul server o migrato a mano, mai sovrascritto dalla build automatica.

Il pattern esatto varia per sito:

- **SimonePizziWebSite**: "postbuild": "node clean-dist.js" (hook automatico npm);

- **DISINTELLIGENZA:** "build": "tsc -b && vite build && node clean-dist.js && move dist\\index.html dist\\index_react.html" (rinomina index.html per abilitare il SEO Engine PHP, Capitolo 11);
- **SitoRuntime:** "build": "tsc -b && vite build && node scripts/remove-db-from-dist.js" (script dedicato).

ATTENZIONE

Il build può togliere anche le difese, non solo i database Questo script ha un effetto collaterale che ritorna al Capitolo 14: rimuovendo la cartella `.data/` dalla distribuzione, porta via anche l'`.htaccess` di `Deny from all` committato lì dentro. La difesa statica non arriva mai sul server, e va ricreata a runtime. La lezione vale per ogni pipeline di build: chiediti sempre cosa il build *toglie*, non solo cosa aggiunge.

**

3. Routing degli URL: SPA contro API

Perché React Router conviva con le API PHP su Apache, lo standard prevede un `.htaccess` nella root pubblica:

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  # Se la richiesta punta a un file o cartella reale, servilo direttamente (
↪ così le API funzionano)
  RewriteCond %{REQUEST_FILENAME} -f [OR]
  RewriteCond %{REQUEST_FILENAME} -d
  RewriteRule ^ - [L]
  # Altrimenti reindirizza tutto a index.html (o index.php se presente)
  RewriteRule ^ index.html [L]
</IfModule>
```

Apache serve `index.php` prima di `index.html` per priorità predefinita. È il meccanismo che permette al SEO Engine (Capitolo 11) di intercettare le richieste dei bot senza toccare le regole di rewrite.

**

4. Inizializzazione Dinamica del File System

Il backend PHP non dà per scontata l'esistenza delle cartelle di runtime: le crea quando servono.

```
// Auto-scaffolding: crea le cartelle di runtime, e protegge subito quella d
↪ el DB
$paths = [__DIR__ . '/.data', __DIR__ . '/.cache', __DIR__ . '/uploads'];
foreach ($paths as $path) {
  if (!is_dir($path)) {
    mkdir($path, 0755, true);
    if (basename($path) === '.data') { // solo
↪ il DB-a-file
      file_put_contents($path . '/.htaccess', "Order allow,deny\nDeny
↪ from all");
    }
  }
}
```

Chi sta su SQLite (DISINTELLIGENZA) genera la cartella `.data/` e il suo `.htaccess` di `Deny from all` **a runtime**, dentro il codice di connessione: la protezione è prodotta dall'applicazione, non pre-deployata, con un `<Files>` nel `.htaccess` globale come seconda rete. I siti su MySQL non hanno un database-a-file da nascondere, ma applicano lo stesso principio (creare a runtime ciò che il deploy non porta) a `cache`, `uploads` e alla cartella dei backup (Capitolo 14). La regola condivisa: non fidarti che una cartella esista, e non fidarti che una difesa statica arrivi fin sul server.

5. Gestione degli Ambienti

- **Sviluppo:** il frontend gira sul dev-server Vite, che fa da proxy verso il PHP locale per evitare gli errori CORS. SimonePizziWebSite, che in sviluppo attraversa una porta diversa (`localhost:8888`), è l'unico a dover gestire la cosa anche lato client (Capitolo 6).
- **Produzione:** il frontend punta a `/api`, percorso relativo same-origin, così l'autenticazione via cookie funziona senza CORS e senza configurazione di dominio.

6. Il Pattern Fork (FDCA da DISINTELLIGENZA)

FDCA e DISINTELLIGENZA condividono una struttura PHP **identica**: stessi file, stessa logica, perché FDCA nasce da un fork del progetto più maturo. È una scelta deliberata: quando due progetti hanno la stessa base funzionale (un festival con votazioni), si parte da una copia di quello che già funziona.

Il vantaggio è l'indipendenza: nessuna dipendenza condivisa, ogni progetto evolve per conto suo. Il rischio è il rovescio esatto di quel vantaggio: ogni bugfix e ogni miglioramento vanno riapplicati a mano su entrambi i rami, e quando il fix riguarda la sicurezza, dimenticarlo significa lasciare una falla aperta in un sito mentre la chiudi nell'altro. È esattamente quello che è successo qui (la catena RCE dell'upload del Capitolo 7 è stata mappata su DISINTELLIGENZA ma vive immutata in FDCA), e il motivo per cui il ciclo di vita di un fork merita un'appendice tutta sua.

IMPORTANTE Il Canone

- Un file per endpoint, niente router centrale: ogni endpoint PHP include i suoi mattoni ed è autonomo.
- Tieni i segreti fuori dal codice versionato (`config.php/.env` nel `.gitignore`, con un `.example` committato).
- Database-a-file e cartelle sensibili fuori dalla docroot, o protetti da un `.htaccess` di `deny` generato a runtime: il build può rimuovere le difese statiche.
- Separa la `dist/` dai sorgenti e fa' che `clean-dist` tolga i `.sqlite` dalla distribuzione.

Prossimo Capitolo: Database Strategy. Lock, indici, migrazioni e la vera storia del WAL.

CAPITOLO 3: Database Strategy

Il database è il cuore del miniCMS. Questo capitolo definisce le strategie di connessione, integrità e migrazione, distinguendo con cura due cose che è facile confondere: ciò che il Modello **raccomanda** e ciò che i siti reali **fanno davvero**. Non sempre coincidono, e dirlo apertamente è più utile che spacciare una prescrizione per una fotografia.

1. Architettura di Connessione (PDO, un motore alla volta)

Il sistema usa PDO con un singleton memoizzato (la connessione si apre una sola volta per richiesta, Capitolo 5). Il motore sotto, però, non è astratto: SQLite e MySQL aprono la connessione con opzioni diverse, e i tre siti le scelgono su tre livelli.

1.1 Le opzioni di connessione: la fotografia

Prima della prescrizione, cosa accade nel codice reale. I tre siti impostano set di opzioni PDO di crescente paranoia, e questo è un buon esempio della scala a tre gradini del Capitolo 1.

Sito	Motore	Opzioni PDO reali
DISINTELLIGENZA	SQLite (vivo)	solo ERRMODE=EXCEPTION + DEFAULT_FETCH_MODE=ASSOC, via <code>setAttribute()</code> . Nessun PRAGMA nel connect() .
SimonePizziWebSite	MySQL	ERRMODE + FETCH + EMULATE_PREPARES=false (prepared statement veri)
SitoRuntime	MySQL	ERRMODE + FETCH + ATTR_TIMEOUT=5 + MYSQL_ATTR_INIT_COMMAND (SET NAMES)

Il dato che sorprende è la prima riga: l'unico sito che oggi gira **davvero** su SQLite (DISINTELLIGENZA) non imposta nessuno dei PRAGMA «ottimali» che la maggior parte dei manuali dà per scontati. Apre il file e basta, con due sole opzioni. È una scelta minimale che funziona per il suo carico, ma non è la configurazione più robusta possibile, ed è il motivo per cui la prescrizione che segue va presa per quello che è: un consiglio, non una descrizione del codice esistente.

1.2 Le opzioni di connessione: la prescrizione

Per un deploy SQLite su hosting condiviso, il Modello raccomanda tre PRAGMA in aggiunta alle opzioni base:

```
// RACCOMANDATO per SQLite su hosting condiviso (non è ciò che DIS imposta o
↳ ggi)
self::$pdo = new PDO("sqlite:" . $dbPath);
self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
self::$pdo->exec("PRAGMA journal_mode = DELETE;"); // più stabile di WAL s
↳ u hosting condiviso
```

```
self::$pdo->exec("PRAGMA busy_timeout = 5000;");    // attende invece di fa
↳ llire sotto scrittura concorrente
self::$pdo->exec("PRAGMA foreign_keys = ON;");      // integrità referenzia
↳ le (off di default in SQLite)
```

Il `busy_timeout` a 5000ms fa aspettare una scrittura concorrente invece di farla fallire subito (utile quando newsletter e admin scrivono insieme). Il `foreign_keys = ON` serve perché SQLite, a differenza di MySQL, lascia le chiavi esterne disattivate per default. Ma la riga che porta una cicatrice è la prima.

ATTENZIONE

Perché DELETE e non WAL: la lezione arriva da un incidente SitoRuntime, quando ancora girava su SQLite, ha provato a usare `journal_mode = WAL` in produzione per migliorare le prestazioni sotto carico. Su hosting condiviso Apache il WAL ha fatto danni: il file di lock `.sqlite-wal` restava appeso e corrompeva le letture. È servito uno script d'emergenza (`emergency_revert_wal.php`) per tornare a DELETE, e poco dopo quel disastro ha spinto la migrazione a MySQL (la storia completa è al Capitolo 15). La lezione, valida per chi resta su SQLite: su hosting condiviso scegli DELETE, non WAL. È una raccomandazione nata da un guasto vero, non una preferenza di stile.

1.3 Nascondere il database-a-file (solo SQLite)

Chi usa SQLite ha un problema che chi usa MySQL non ha: il database è un file dentro le cartelle del sito, e un file raggiungibile via web è il database intero scaricabile da chiunque. La difesa, in **DISINTELLIGENZA**, è generata a runtime dal codice di connessione: alla prima `connect()`, se la cartella `.data/` non esiste, viene creata e ci si scrive dentro un `.htaccess` di `Deny from all`.

```
// DISINTELLIGENZA db.php - la protezione del DB-a-file è creata dall'app, n
↳ on pre-deployata
$dir = dirname($dbPath);                                // ../.data
if (!is_dir($dir)) mkdir($dir, 0755, true);
$htaccessPath = $dir . '/.htaccess';
if (!file_exists($htaccessPath)) {
    file_put_contents($htaccessPath, "Require all denied\n"); // seconda r
↳ ete: <Files> nel .htaccess globale
}
```

Questo pattern appartiene ai siti SQLite, non a quelli su MySQL. SimonePizziWebSite e SitoRuntime sono migrati a MySQL: non hanno alcuna cartella `.data/` con il database dentro, perché i loro dati vivono sul server MySQL, fuori dalla docroot per natura. Generare la protezione a runtime, anziché affidarla a un file committato, ha una ragione concreta che ritorna al Capitolo 14: lo script di build può rimuovere la cartella `.data/` dalla distribuzione, e con essa l'`.htaccess` di deny, che quindi non arriverebbe mai sul server.

*
**

2. Indicizzazione Strategica

Le query di lettura devono essere istantanee. Il Modello prevede questi indici:

Tabella	Colonna	Tipo	Scopo
news / articles	slug	UNIQUE	Ricerca articolo via URL (SEO)
news / articles	published_at	DESC	Ordinamento cronologico veloce
news / articles	status	INDEX	Filtraggio bozza/pubblicato
speakers	sort_order	ASC	Ordinamento manuale trascinabile
podcasts	slug	UNIQUE	Accesso rapido alle serie
projects	category	INDEX	Filtraggio per categoria portfolio
projects	sort_order	ASC	Ordinamento manuale portfolio

Dopo caricamenti massivi è utile eseguire `ANALYZE`; per ricalcolare le statistiche di accesso e ottimizzare il piano delle query.

**
**

3. Ciclo di Vita delle Migrazioni

Le modifiche allo schema devono essere atomiche, idempotenti e protette. Sono i requisiti; i siti reali li rispettano in modo diseguale, e la differenza conta.

- **Atomicità:** ogni migrazione usa una transazione (`beginTransaction`).
- **Idempotenza:** lo script verifica l'esistenza di colonne o tabelle prima di crearle (`IF NOT EXISTS, PRAGMA table_info`), così rieseguirlo non rompe niente.
- **Protezione:** gli script di migrazione **dovrebbero** essere irraggiungibili da web non autenticato. Qui la realtà diverge: SitoRuntime li nega per prefisso nel `.htaccess` o li tiene dentro `admin.php` (gated), mentre DISINTELLIGENZA lascia i suoi `update_db_*.php` e `migrate_media.php` raggiungibili in HTTP senza gate. È un debito di sicurezza reale (Capitolo 15), non un dettaglio.
- **Nomenclatura:** nessun sito ha una tabella `schema_version`. La versione vive nei **nomi dei file** (`update_db_v0.4.2.php`, `update_db_v0.5.4.php`), allineati alla versione in `package.json`. La storia delle migrazioni si legge dalla cartella, non da un registro nel database.

**
**

4. Normalizzazione dei Dati

Per evitare incoerenze tra PHP, JS e il database, il Modello normalizza prima del salvataggio:

- **Date:** formato `Y-m-d H:i:s` per compatibilità SQL e JS. Attenzione al fuso: il confronto di visibilità `published_at <= NOW` è fatto su stringhe, e una data scritta con il separatore sbagliato sposta la soglia (Capitolo 5 e Capitolo 9).
- **Numerici:** interi o arrotondati a zero decimali, per evitare i bug di virgola mobile.
- **Booleani:** `INTEGER` (0 o 1) in SQLite, `TINYINT(1)` in MySQL.

**
**

5. Manutenzione e Integrità

- **VACUUM**: da eseguire dopo cancellazioni massive per ricompattare il file e ridurne il peso (rilevante su SQLite).
- **Backup**: ogni migrazione **dovrebbe** essere preceduta da una copia del database in una cartella protetta. Anche qui la realtà non è uniforme: SimonePizziWebSite ha un backup automatico fuori docroot, DISINTELLIGENZA fa una copia `.bak` prima delle azioni distruttive, SitoRuntime (il sito che ha sofferto il crash) non ha alcun backup automatico. Il paradosso «cura senza prevenzione» è al Capitolo 14.
- **optimize_db.php**: SitoRuntime include uno script di manutenzione (`VACUUM`, `ANALYZE`, verifica integrità); nonostante l'intestazione «usa e getta» è in realtà non distruttivo (aggiunge solo indici idempotenti).

*
**

6. Quando Passare a MySQL

La storia completa, con gli script reali, è al Capitolo 15. Un punto va chiarito subito, perché è una mezza verità diffusa: la migrazione di SitoRuntime **non** è stata decisa da una soglia di traffico raggiunta con calma. È stata la reazione a un incidente (il crash del WAL della notte) che ha reso SQLite improvvisamente inaffidabile su quell'hosting. La soglia non era un numero su un grafico, era un database corrotto alle tre di notte.

Il contrappunto è altrettanto istruttivo: DISINTELLIGENZA, un festival con votazioni pubbliche, gira **ancora oggi su SQLite** in produzione, senza problemi. SQLite non è un gradino da abbandonare appena possibile: è la scelta giusta finché regge, e «finché regge» dipende dal carico e dall'hosting, non da una regola universale. Si passa a MySQL quando un vincolo concreto lo impone, non per scaramanzia.

6.1 I numeri, con onestà

«Finché regge» non è una risposta che si può lasciare al sentimento. Servono ordini di grandezza, con l'avvertenza che restano tali: dipendono dall'hardware, dal disco dell'hosting condiviso e dalla forma delle query, e vanno misurati sul proprio caso, non presi come garanzie.

Il punto chiave è che in SQLite **lettura e scrittura non scalano allo stesso modo**. Le letture sono concorrenti e velocissime: un sito a prevalenza di lettura (un blog, un portfolio, una radio con news e podcast) regge senza fatica migliaia di letture al secondo dalla cache di pagina e centinaia di query di lettura al secondo dirette al file, perché più richieste possono leggere lo stesso database insieme. Le scritture, invece, sono **serializzate**: SQLite ammette un solo scrittore alla volta e blocca l'intero file durante la scrittura. È lì che si trova il soffitto.

In pratica, su un tipico hosting condiviso:

Segnale	SQLite è a suo agio	Conviene valutare MySQL
Scritture concorrenti	fino a qualche decina al minuto, sporadiche	decine al secondo sostenute, o picchi concorrenti regolari
Profilo di carico	lettura dominante (90%+), scritture isolate	scrittura frequente e simultanea (form, voti, code)
Concorrenza in scrittura	un processo per volta basta	più processi scrivono insieme (newsletter + admin + pubblico)
Topologia	un solo server applicativo	serve scalare su più nodi che condividono i dati
Sintomo nei log	nessun database is locked	busy timeout o database is locked ricorrenti

La soglia vera non è un numero ma un **sintomo**: quando nei log compaiono errori di lock o di busy timeout nonostante il busy_timeout configurato, il database ti sta dicendo che la contesa in scrittura ha superato quello che un file-database regge su quell'hosting. È il momento di MySQL, e non un istante prima. Runtime Radio ci è arrivato per un incidente (Capitolo 15); la maggior parte dei siti non ci arriva mai, ed è giusto così.

IMPORTANTE Il Canone

- SQLite con journal_mode=DELETE (mai WAL su hosting condiviso), busy_timeout, foreign_keys=ON; PDO in ERRMODE_EXCEPTION.
- Indicizza slug (UNIQUE), published_at (DESC), status; ANALYZE dopo i caricamenti massivi.
- Migrazioni atomiche, idempotenti e irraggiungibili da web non autenticato; tieni una tabella schema_version (i siti reali non ce l'hanno: è un debito da non ereditare).
- Backup prima di ogni migrazione, in una cartella protetta fuori docroot.
- Passa a MySQL quando i log mostrano database is locked/busy timeout ricorrenti, non per scaramanzia.

**

Prossimo Capitolo: Frontend Dependencies. La matrice delle dipendenze, le regole di scelta e il costo di ogni libreria.

CAPITOLO 4: Frontend Dependencies

Le dipendenze frontend sono scelte architetturali, non solo righe in `package.json`. Ogni libreria aggiunta ha un costo (dimensione del bundle, superficie di aggiornamento, complessità di integrazione) e deve guadagnarsi il posto. Questo capitolo documenta le scelte fatte nei quattro siti di riferimento, con le motivazioni che le guidano, e dedica spazio anche a un'assenza: le librerie che il Modello sceglie di **non** usare.

1. Il Core Stack (Ogni Progetto)

Queste dipendenze sono presenti in tutti e quattro i siti senza eccezioni:

Libreria	Versione	Ruolo
react + react-dom	^19.2.x	Framework UI: componenti, stato, rendering
react-router-dom	^7.x	Routing client-side, React Router v7 (API stabile)
typescript	~5.x	Type safety a compile time
vite	^7.x	Build tool: dev server HMR, bundle ottimizzato
@vitejs/plugin-react	^5.x	Plugin Vite per la trasformazione JSX/TSX
lucide-react	^0.5xx	Icone SVG tree-shakeable, native in React

React 19 introduce miglioramenti al rendering concorrente e alle Server Components. Per il Modello Universale (sola SPA client-side) la differenza pratica con la v18 è minima, ma tenere l'ultima versione stabile è la scelta corretta per i nuovi progetti.

1.1 L'Assenza che Conta: Niente Librerie di Data-Fetching

Prima di elencare cosa c'è, vale un elenco di cosa manca, perché è una scelta tanto quanto le altre. Nessuno dei quattro siti usa una libreria di data-fetching o di gestione dello stato: niente Axios, niente React Query, niente SWR, niente Redux, niente Zustand. Il ponte verso il backend PHP è l'oggetto `api`, un wrapper sottile su `fetch` nativo (Capitolo 6); lo stato condiviso vive nel router (SimonePizziWebSite, con i loader di React Router) oppure è `useState` locale ai componenti (SitoRuntime e DISINTELLIGENZA).

Non è pigrizia, è coerenza con la filosofia del Capitolo 1: ogni dipendenza che non aggiungi è una superficie di aggiornamento che non devi mantenere, un bundle che non cresce, un comportamento che non devi imparare. `fetch` fa quasi tutto ciò che serve a un CMS; il poco che manca (un punto unico per gli errori, un interceptor per la sessione scaduta) si scrive in poche righe, e il Capitolo 6 racconta dove i siti hanno scelto di non scriverle.

2. Tailwind CSS: v3 contro v4

I quattro siti usano versioni diverse di Tailwind, con configurazione sensibilmente diversa.

SitoRuntime: Tailwind v3 (setup classico)

```
// devDependencies
"tailwindcss": "^3.4.17",
"autoprefixer": "^10.4.22",
"postcss": "^8.5.6"
```

Richiede `tailwind.config.js` e `postcss.config.js` espliciti. Il CSS di ingresso:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

SimonePizziWebSite, DISINTELLIGENZA, FDCA: Tailwind v4

```
// devDependencies
"tailwindcss": "^4.x",
"@tailwindcss/vite": "^4.x" // oppure "@tailwindcss/postcss"
```

Tailwind v4 usa il plugin Vite nativo: nessun `postcss.config.js` separato, nessun `tailwind.config.js` obbligatorio. Il CSS:

```
@import "tailwindcss";
```

Per i nuovi progetti la scelta è la v4. La migrazione da v3 richiede attenzione alle classi rinominate (per esempio `shadow-sm` è diventato `shadow-xs`).

@tailwindcss/typography

Presente in tutti i siti come devDependency. Aggiunge la classe `prose`, che applica stili tipografici curati a blocchi di HTML generato dinamicamente (l'output di un editor di testo ricco, o di un convertitore Markdown).

```
<div class="prose prose-invert max-w-none">
  <!-- HTML generato da Tiptap o da showdown -->
</div>
```

3. Editing e Rendering del Contenuto

Tiptap v3: il Rich Text Editor dei flagship

Usato in: SimonePizziWebSite e SitoRuntime (i due siti con redazione editoriale).

L'editor di testo ricco dei due flagship è **Tiptap v3**, non un editor monolitico ma un insieme di pacchetti `@tiptap/*` che si compongono come mattoncini: una decina di moduli per il nucleo e le estensioni effettivamente usate.

```
"@tiptap/react": "^3.x",
"@tiptap/pm": "^3.x",
"@tiptap/starter-kit": "^3.x",
"@tiptap/extension-image": "^3.x",
"@tiptap/extension-text-style": "^3.x",
```

```
"@tiptap/extension-color": "^3.x",
"@tiptap/extension-text-align": "^3.x",
"@tiptap/extension-table": "^3.x",
"@tiptap/extension-youtube": "^3.x"
```

Tiptap produce **HTML** (non Markdown), che viene salvato grezzo nel database e sanificato al momento del rendering con DOMPurify (la trattazione completa, comprese le guardie all'inserimento dei link e il choke-point di sanitizzazione, è al Capitolo 8). SitoRuntime ci è arrivato con una migrazione: prima usava Quill, e nel codice restano i segni di quel passaggio (uno shim di conversione, qualche type-def `react-quill` residua). Quei residui non sono l'editor attivo, sono cicatrici della migrazione a Tiptap.

DISINTELLIGENZA non usa nessuna libreria di editing: il suo editor è costruito a mano con `contentEditable` ed `execCommand`. È il gradino grado-zero della scala, con la conseguenza di sicurezza che il Capitolo 8 mette in chiaro (è l'unico sito senza DOMPurify).

showdown: Markdown → HTML

Usato in: SimonePizziWebSite, SitoRuntime, DISINTELLIGENZA, per i contenuti scritti in Markdown.

```
"showdown": "^2.1.0",
"@types/showdown": "^2.0.6"
```

```
import showdown from 'showdown';
const converter = new showdown.Converter({ tables: true, strikethrough: true
  ↵ });
const html = converter.makeHtml(markdownContent);
```

L'HTML generato da contenuto non fidato va sempre sanificato (vedi sotto): showdown e DOMPurify vanno in coppia.

dompurify: sanitizzazione XSS

Usato in: SimonePizziWebSite, SitoRuntime (sempre accoppiato all'editor o al convertitore Markdown). **Assente in DISINTELLIGENZA**, ed è il buco di sicurezza che il Capitolo 8 documenta.

```
"dompurify": "^3.3.x",
"@types/dompurify": "^3.0.5"
```

```
import DOMPurify from 'dompurify';
const safeHtml = DOMPurify.sanitize(rawHtml);
// Solo ora è sicuro usare dangerouslySetInnerHTML
<div dangerouslySetInnerHTML={{ __html: safeHtml }} />
```

ATTENZIONE

HTML grezzo più `dangerouslySetInnerHTML` senza DOMPurify è una vulnerabilità XSS

Vale per l'output di showdown, di Tiptap, di qualunque sorgente. Salvare HTML grezzo è una scelta legittima (preserva la formattazione), ma il momento del rendering è dove la sanitizzazione diventa obbligatoria. DISINTELLIGENZA salta questo passaggio, e il Capitolo 8 mostra cosa significa.

4. Animazioni ed Effetti Visivi

framer-motion

Usato in: tutti e quattro i siti.

```
"framer-motion": "^12.x"
```

```
import { motion } from 'framer-motion';
<motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} t
  ↪ransition={{ duration: 0.4 }}>
  {/* contenuto */}
</motion.div>
```

framer-motion è pesante (circa 100KB gzip). Si giustifica quando le animazioni sono parte dell'identità visiva del sito; dove sono marginali, le transizioni CSS native sono la scelta giusta.

typewriter-effect

Usato in: SimonePizziWebSite. Effetto macchina da scrivere per le headline animate, specifico per un portfolio personale.

tailwindcss-animate

Usato in: DISINTELLIGENZA. Plugin Tailwind con classi come `animate-in`, `fade-in`, `slide-in-from-bottom`; alternativa leggera a framer-motion per animazioni semplici basate su CSS.

5. SEO e Meta Tag

react-helmet-async

Usato in: SimonePizziWebSite, DISINTELLIGENZA. Gestisce `<title>`, `<meta>`, `<link>` dal lato React, per gli utenti con JavaScript attivo.

```
"react-helmet-async": "^2.0.5"
```

La versione originale (`react-helmet`) non è più mantenuta e ha problemi di memory leak con React 18+; `react-helmet-async` è il fork attivo, compatibile con React 19.

I crawler dei social e dei motori di ricerca non eseguono JavaScript, quindi i meta tag che contano per loro li produce il **SEO Engine PHP** (Capitolo 11). I due si completano, non sono alternativi: SitoRuntime, infatti, rinuncia del tutto a `react-helmet-async` e delega l'intero SEO all'engine lato server.

6. Utilities

clsx + tailwind-merge

Usati in: SimonePizziWebSite, DISINTELLIGENZA.

```
"clsx": "^2.1.1",
"tailwind-merge": "^3.x"
```

`clsx` compone `className` in modo condizionale; `tailwind-merge` risolve i conflitti tra classi Tailwind (`text-sm text-lg` diventa `text-lg`). Insieme danno il classico helper `cn()`.

date-fns

Usato in: DISINTELLIGENZA, per formattare le date del festival nel frontend. Gli altri siti gestiscono le date lato PHP (`date()`, `strtotime()`), riducendo la complessità nel client.

7. Ottimizzazione in Build

sharp

Usato in: SimonePizziWebSite, come `dependency` ma solo negli script `post-build`.

```
"sharp": "^0.34.5"
```

`sharp` è una libreria Node.js per la manipolazione di immagini ad alte prestazioni; viene usata negli script di build per ottimizzare gli asset statici. È una `dependency` (non `devDependency`) perché il `postbuild` la richiede al momento della build, ma non entra mai nel bundle React: nessun file `.tsx` la importa. La conversione WebP degli upload a runtime, invece, è tutta lato PHP con GD (Capitolo 7): `sharp` e GD coprono due momenti diversi.

8. Matrice delle Dipendenze per Sito

Libreria	SitoRuntime	DISINTELLIGENZA	FDCA	SimonePizziWebSite
React 19	sì	sì	sì	sì
react-router-dom v7	sì	sì	sì	sì
framer-motion	sì	sì	sì	sì
showdown	sì	sì	no	sì
dompurify	sì	no	no	sì
lucide-react	sì	sì	sì	sì
Tailwind v3	sì	no	no	no
Tailwind v4	no	sì	sì	sì
@tailwindcss/typography	sì	sì	sì	sì
react-helmet-async	no	sì	sì	sì
@tiptap/* (editor)	sì	no	no	sì
clsx + tailwind-merge	no	sì	sì	sì
sharp	no	no	no	sì
date-fns	no	sì	sì	no
typewriter-effect	no	no	no	sì
tailwindcss-animate	no	sì	sì	no
librerie di data-fetching	no	no	no	no

L'ultima riga è la più importante del Modello: nessun sito ne ha.

9. Regole per i Nuovi Progetti

1. **Parti dal minimo:** core stack, framer-motion, lucide-react. Aggiungi solo quando la funzionalità serve davvero.
2. **Niente data-fetching library:** `fetch` e un oggetto `api` bastano (Capitolo 6). Aggiungere React Query o Axios è quasi sempre peso che non ripaga in un CMS thin-stack.
3. **showdown richiede sempre dompurify** quando il contenuto è user-generated o viene dal database.
4. **Tiptap per l'editing ricco, <textarea> per il resto:** monta i pacchetti `@tiptap/*` solo dove serve un editor visivo per i redattori; per un input semplice, una `textarea` mantiene il bundle piccolo.
5. **react-helmet-async per il SEO client, PHP engine per i crawler:** si completano, non sono alternativi.
6. **Tailwind v4 per i nuovi progetti:** configurazione più semplice; se migri da v3, verifica le classi rinominate.

**

Capitoli correlati: Capitolo 2 (struttura del progetto) per la configurazione Vite; Capitolo 6 (Frontend Bridge) per l'oggetto `api` su `fetch`; Capitolo 8 (Advanced Content Editing) per Tiptap e la sanitizzazione; Capitolo 11 (SEO) per il rapporto tra `react-helmet-async` e l'engine PHP.

IMPORTANTE Il Canone

- Parti dal minimo: niente libreria di data-fetching (bastano `fetch` e un oggetto `api`), niente ORM, niente state-manager finché non servono davvero.
- Per l'editing ricco usa Tiptap (HTML salvato grezzo + DOMPurify al render); una `textarea` per tutto il resto.
- Ogni dipendenza ha un costo: aggiungila solo se ripaga, e distingui il build-time (`sharp`) dal runtime (`GD`).

**

Prossimo Capitolo: Backend Logic (PHP). CRUD unificato, gestione dei buffer e sanitizzazione degli input.

P A R T E I I I

I Componenti

I mattoni. Backend, frontend, media, editor: i building block del sistema.

CAPITOLO 5: Backend Logic (PHP)

Il backend del miniCMS è il motore che elabora i dati e fa da guardiano. Questo capitolo definisce come è fatto un endpoint, come si avvia, come gestisce gli identificatori, il fuso orario e gli errori. Il filo conduttore è sempre lo stesso: niente framework, un file PHP per responsabilità, e tre modi di declinare lo stesso scheletro a seconda di quanto il sito ha scelto di ingegnerizzare.

1. Gestione degli Identificatori (Slug)

Gli URL devono essere parlanti e univoci. Il Modello genera lo slug lato server e ne garantisce l'unicità contro le collisioni.

1.1 Algoritmo base

```
function createSlug($string) {  
    // minuscolo, trim e rimozione dei caratteri non alfanumerici (eccetto i  
    ↪ 1 trattino)  
    return strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-', $string)));  
}
```

1.2 Algoritmo avanzato (normalizzazione degli accenti italiani)

Il pattern base produce slug malformati con le parole accentate (caffè diventa caff-). La variante di **SimonePizziWebSite** risolve il problema con una mappa esplicita prima della pulizia:

```
function generateSlug($title, $pdo) {  
    $accents      = ['à','è','é','ì','ò','ù','À','È','É','Ì','Ò','Ù','â','ê'  
    ↪ ', 'î','ô','û','ä','ë','ï','ö','ü'];  
    $replacements = ['a','e','e','i','o','u','a','e','e','i','o','u','a','e'  
    ↪ ', 'i','o','u','a','e','i','o','u'];  
    $title = str_replace($accents, $replacements, $title);  
    $slug  = strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-', $title)))  
    ↪ ;  
  
    // anti-collisione: se lo slug esiste, aggiunge un suffisso temporale  
    $stmt = $pdo->prepare("SELECT COUNT(*) FROM articles WHERE slug = ?");  
    $stmt->execute([$slug]);  
    if ($stmt->fetchColumn() > 0) $slug .= '-' . time();  
    return $slug;  
}
```

Per i siti con contenuto italiano va usato sempre il pattern avanzato. (Le tre filosofie di slug dei vari siti, accenti compresi o elisi, sono dettagliate al Capitolo 9.)

2. Gestione del Fuso Orario

Su un hosting internazionale (un server a Los Angeles, per dire) `date()` e `time()` usano il fuso del server, e questo rompe la logica di visibilità `published_at <= NOW`, che per un sito italiano deve ragionare in ora italiana.

```
// all'inizio di ogni endpoint con logica temporale
date_default_timezone_set('Europe/Rome');
$ita_now_str = date('Y-m-d H:i:s'); // ora italiana per i confronti SQL
```

La regola è semplice, la sua applicazione no. SimonePizziWebSite forza il fuso in **ogni** endpoint; SitoRuntime e DISINTELLIGENZA lo fanno solo in alcuni (`index.php`, `news.php`), e altrove no. Il risultato è una soglia di pubblicazione che si sposta a seconda di quale file la valuta.

ATTENZIONE

Il fuso va forzato ovunque, o non serve a niente Forzare il timezone in un solo endpoint dà una falsa sicurezza: un articolo programmato può risultare già pubblicato per un file e ancora futuro per un altro, perché la stessa stringa `published_at` viene confrontata con un `NOW` calcolato in fusi diversi. SitoRuntime porta la cicatrice di questo problema in un `debug_time.php` che documenta un incidente sul separatore della data (lo spazio diventato `τ`). La logica del confronto sulle date, e i tre modi di sbagliarla, è al Capitolo 9; qui basta la regola di bootstrap: se forzi il fuso, fallo nel prelude condiviso, non endpoint per endpoint.

3. Anatomia di un Endpoint: il Router su `REQUEST_METHOD`

Non c'è un router centrale. Ogni file in `public/api/` è un endpoint autonomo, e smista da sé in base al verbo HTTP della richiesta. È il pattern che rende il thin stack leggibile: l'URL `/api/articles.php` è il file `articles.php`, e dentro quel file c'è tutto ciò che lo riguarda.

```
$method = $_SERVER['REQUEST_METHOD'];

if      ($method === 'GET')    { /* lettura: page, limit, slug, id, category
↳ , admin */ }
elseif ($method === 'POST')  { Auth::check(); /* creazione */ }
elseif ($method === 'PUT')   { Auth::check(); /* sostituzione completa */
↳ }
elseif ($method === 'PATCH') { Auth::check(); /* aggiornamento parziale: t
↳ ogg le is_visible, sort_order */ }
elseif ($method === 'DELETE') { Auth::check(); /* eliminazione */ }
```

Il gate non è uniforme sul file ma **selettivo sul ramo**: il `GET` di lettura resta pubblico, i rami che mutano lo stato passano da `Auth::check()`. Nei progetti più vecchi (DISINTELLIGENZA, SitoRuntime prima del refactor) le mutazioni viaggiavano tutte come `POST` con un campo `action` nel body; il pattern con i verbi separati è più leggibile e si sposa meglio con un client TypeScript espressivo.

4. I Tre Stili di Bootstrap

Prima di smistare la richiesta, un endpoint deve avviarsi: aprire la connessione, far partire la sessione, mandare gli header, gestire l'eventuale preflight CORS. Qui i tre siti occupano tre gradini, ed è un buon ritratto della scala del Capitolo 1.

SimonePizziWebSite: prelude inline. Ogni file include in testa i suoi mattoni (require 'db.php', require 'auth_helper.php', gli header) e apre la connessione subito. L'auth_helper.php incapsula session_start(), il Content-Type e la classe Auth:

```
// auth_helper.php - session, header e Auth in un solo include
require_once 'db.php';
session_start();
header('Content-Type: application/json');

class Auth {
    public static function check() {
        if (!isset($_SESSION['user_id'])) {
            http_response_code(401);
            echo json_encode(['status' => 'error', 'message' => 'Non autoriz
↳ zato']);
            exit;
        }
    }
}
```

Concentrare session_start() e header() in un file solo riduce il rischio di «headers already sent» da spazi o BOM sparsi nei file.

SitoRuntime: prelude condiviso cors.php. Il sito serve un frontend che in sviluppo arriva da un'altra origine, quindi antepone a tutto un cors.php che gestisce header, Content-Type e il preflight OPTIONS, poi apre la connessione in modo lazy (getDB() con uno static, copiato in ogni file). La CORS reale **non** è aperta a tutti: è una allowlist di origini note, con riflessione dell'origin ammesso.

```
// cors.php - prelude condiviso: allowlist, non "*"; risponde al preflight e
↳ termina
$allowed = ['https://runtimeradio.com', 'https://www.runtimeradio.com', 'htt
↳ ps://runtimeradio.it'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';
if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: $origin");
    header('Vary: Origin');
}
header('Content-Type: application/json');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') { http_response_code(204); exi
↳ t; }
```

DISINTELLIGENZA: inline minimale, niente CORS. Sviluppa e pubblica same-origin, quindi non ha bisogno di header CORS: il .htaccess garantisce che frontend e API stiano sulla stessa origine, e il cookie di sessione parte da solo. Il bootstrap è ridotto all'osso.

NOTA

Access-Control-Allow-Origin: * **quasi mai** L'asterisco va bene solo per un'API pubblica di sola lettura, senza cookie. Appena c'è una sessione, riflettere una allowlist di origini è la scelta corretta, e va accompagnata da una decisione esplicita SU Access-Control-Allow-Credentials (SitoRuntime non lo emette, e di conseguenza l'autenticazione resta de-facto same-origin: il dettaglio, col suo riflesso sul client, è ai Capitoli 6 e 10).

L'errore di connessione: tre risposte

Cosa succede quando il database non risponde è un piccolo test di maturità, e i tre siti lo passano in modo diverso.

```
// SPW: codice HTTP corretto + JSON + log per la diagnosi
catch (PDOException $e) {
    http_response_code(500);
    error_log('DB: ' . $e->getMessage()); // resta nei log,
    ↪ non va al client
    echo json_encode(['status' => 'error', 'message' => 'Database non dispon
    ↪ibile']);
    exit;
}
```

SimonePizziWebSite risponde 500, in JSON, e scrive l'eccezione nei log del server. SitoRuntime risponde 503 in JSON ma **non logga**, e così perde la diagnostica proprio quando servirebbe. DISINTELLIGENZA fa la cosa da non fare: `die("Connection failed: " . $e->getMessage())`, che stampa al client il messaggio dell'eccezione (path inclusi), senza un codice HTTP e fuori dal formato JSON.

ATTENZIONE

Un errore di connessione non deve mai parlare al client Il messaggio di una `PDOException` può contenere percorsi del filesystem, nomi di database, dettagli che a un attaccante fanno comodo. La regola: codice HTTP corretto (500 o 503), un messaggio generico al client, e il dettaglio vero solo nei log del server con `error_log()`. Stampare `$e->getMessage()` in pagina, come fa DISINTELLIGENZA, è information disclosure gratuita.

5. Elaborazione Media

Caricare un file non è un semplice `move_uploaded_file`, è una trasformazione. Il type dell'upload mappa su cartelle diverse, ognuna con la sua politica: le immagini sono pubbliche e ridimensionate, gli audio dei podcast solo via admin, gli audio dei partecipanti su una cartella isolata e (nel caso del festival) ad accesso aperto. Ogni immagine dell'admin viene normalizzata a una larghezza massima (1920px), preservando il canale alpha di PNG e WebP. La trattazione completa, comprese le insidie di sicurezza dell'upload (la validazione dei byte reali, il naming, la catena RCE dell'upload pubblico), è al Capitolo 7: qui basta sapere che il file passa per GD prima di posarsi su disco.

6. Sicurezza dell'Input e dell'Output

- **Nomi dei file:** ogni upload viene rinominato con `uniqid()` e ripulito dai caratteri speciali, per non lasciare all'utente il controllo del nome (e quindi dell'estensione). Il perché è al Capitolo 7.
- **Integrità del JSON:** ogni risposta è preceduta da `header('Content-Type: application/json')`, e in caso di errore porta il codice HTTP giusto (400, 401, 403, 500) insieme a un messaggio JSON descrittivo.
- **FILTER_SANITIZE_STRING è deprecato** (da PHP 8.1): al suo posto, `strip_tags(trim($var))`.

7. Gestione del Buffer

Un Notice o un Warning di PHP stampato in mezzo a una risposta la rende JSON non valido, e il frontend si rompe. La difesa è duplice: `display_errors = 0` in produzione (gli

errori vanno nei log, non in pagina) ed eventualmente `ob_start()` per controllare cosa esce. È lo stesso principio dell'errore di connessione del §4: il client riceve dati puliti, la diagnostica resta sul server.

IMPORTANTE **Il Canone**

- File-per-endpoint con router su `REQUEST_METHOD`; gate selettivo sul ramo (GET pubblico, mutazioni dietro `Auth:::check`).
- CORS via allowlist di origini con `Vary`, mai `*` insieme alle credenziali.
- L'errore di connessione si scrive nei log e risponde al client in modo generico: mai `getMessage()` in chiaro.
- Forza il timezone in **ogni** endpoint con logica temporale, e in produzione `display_errors = 0`.

Prossimo Capitolo: Frontend Bridge (API.ts). La connessione tra React e PHP, e i tre modi di leggere un payload che non ha un contratto stabile.

CAPITOLO 6: Frontend Bridge (API.ts)

React parla con PHP attraverso un solo punto. Non chiamate `fetch` sparse per i componenti, ma un oggetto `api` che le raccoglie tutte: un metodo per azione (`login`, `getNews`, `uploadImage`, `submitVote`), raggruppati per dominio con qualche commento, importati ovunque con `import { api }`. Niente Axios, niente React Query, niente Redux, nessuno store globale. È la versione thin-stack del data access layer: una facciata piatta sopra `fetch`, coerente con la filosofia del modello (CAP 4), meno dipendenze possibile.

Sotto quella superficie comune c'è un problema condiviso, ed è la vera lente del capitolo: l'API PHP non ha un contratto uniforme. Gli endpoint, cresciuti per accrescimento, rispondono con buste diverse (un array nudo, oppure `{ data, total }`, oppure `{ success, ... }`), e a volte con un HTTP 200 anche quando le cose sono andate male. Nessuno dei tre siti versiona quel contratto. Ognuno si limita a leggere in modo difensivo quello che arriva, e il modo in cui se ne fa carico è ciò che li distingue.

Le tre risposte si leggono bene come una scala di *investimento*. `SimonePizziWebSite` investe nello state layer: i loader di `react-router` come data layer, e il «Double Read» del payload. `SitoRuntime` investe nella sicurezza del client: un token CSRF tenuto in una variabile di modulo. `DISINTELLIGENZA` investe in un codemod, uno script che rattoppa il client a posteriori. Tre modi di tenere insieme React e PHP quando nessuno dei due lati ha un contratto stabile. E come nei capitoli precedenti, l'ordine d'investimento non coincide con la solidità: chi spende di più non è chi sbaglia di meno.

Il quarto sito, `FDCA`, non entra qui. Il fork ha riscritto e ridotto il frontend a vetrina pubblica, e non ha alcun `api.ts` né `fetch` verso `/api`: dove sul backend era byte-identico a `DIS`, sul ponte è semplicemente assente. Un guscio scollegato dal CMS, che vive nel capitolo dedicato al forking, non in questo.

✱

✱

1. Un solo client, sottile su `fetch`

Prima delle differenze, i tratti che i tre siti hanno in comune. Il client è un oggetto-namespacing, non una classe: nessuna istanza, nessuna dependency injection, si importa e si chiama. Non c'è libreria di data-fetching: lo stato condiviso o vive nel router (SPW) o è `useState` locale ai componenti (SR, DIS), e il «fetching» è `fetch` nativo. Dove possibile i metodi sono tipizzati (`NewsArticle[]`, `UserRole`), così il chiamante sa cosa aspettarsi, ma il contratto vero lo detta il PHP, e non sempre combacia con i tipi dichiarati.

L'autenticazione, infine, viaggia col cookie di sessione `HttpOnly` (CAP 10): il client non porta token di bearer né stato d'autenticazione persistente. Da qui discende un dettaglio che divide subito i tre siti, la base URL e il cookie.

```
// SPW api.ts:1-12 - base URL che commuta prod/dev, config condivisa col coo
↳ kie
export const API_URL = import.meta.env.PROD ? '/api' : 'http://localhost:888
↳ 8/api';

const fetchConfig: RequestInit = {
  credentials: 'include', // propaga il cookie di sessione PHP (in dev at
↳ traversa localhost:8888)
  headers: { 'Accept': 'application/json', 'Content-Type': 'application/js
↳ on' }
};
```

```
// SR api.ts:4 - base fissa, nessuno switch: sempre same-origin
const API_BASE = '/api'; // niente localhost:8888, e - punto cruciale -
↳ niente credentials:'include'
```

NOTA

credentials:'include': quando serve e quando è rumore SPW mette `credentials:'include'` su ogni chiamata, e deve farlo: in sviluppo il frontend e il backend stanno su due origini diverse (`localhost:8888`), e senza quell'opzione il cookie di sessione non partirebbe. SR e DIS non lo mettono mai, perché sviluppano e pubblicano `same-origin`, dove il cookie viaggia da solo con la policy di default di `fetch`. C'è una simmetria esatta col lato server (CAP 10): la CORS di SR non emette `Access-Control-Allow-Credentials`, quindi l'autenticazione è di fatto `same-origin` su entrambi i lati, e lì `credentials:'include'` sarebbe una riga che non fa nulla. La regola: quell'opzione serve quando client e API stanno su origini diverse; `same-origin`, è rumore.

✱
✱

2. Il contratto non è uniforme: tre modi di leggere il payload

È il cuore del capitolo. Gli endpoint rispondono con buste diverse, e il client deve sopravvivere a questa incoerenza. Le tre tecniche sono il GOLD del cluster.

SPW vive il caso più ostico: *lo stesso* endpoint a volte risponde con un array nudo, a volte con un oggetto paginato `{ data, total }`. La risposta è leggere il payload in due forme possibili, ed è questo, e soltanto questo, il pattern che il modello chiama «Double Read».

```
// SPW loaders.ts:30-31 - lo stesso endpoint a volte è array, a volte { data
↳ , total }
const articlesData = Array.isArray(articlesRes) ? articlesRes : articlesRes.
↳ data;
const projectsData = Array.isArray(projectsRes) ? projectsRes : projectsRes.
↳ data;
```

SR non ha lo stesso endpoint ambiguo, ha un mosaico: ogni endpoint ha la *sua* busta, e il client la conosce a memoria. Su due di essi, però, dove la risposta è un array nudo in caso di successo ma un oggetto `{ success:false, error }` in caso di errore, mette una guardia di tipo.

```
// SR - ogni endpoint ha la sua busta, letta per forma nota:
// news.php → { success, data, meta }      admin?action=list → { success,
↳ articles, total }
// speakers.php / podcasts.php → ARRAY NUDO, oppure { success:false, error
↳ } su errore
if (Array.isArray(res)) setPodcasts(res);    // Podcasts.tsx:14 - guardia di
↳ tipo anti-errore
```

DIS sceglie la via opposta: non normalizza affatto. Ritorna il JSON così com'è («busta zero») e lascia che sia il chiamante a sapere com'è fatto.

```
// DIS api.ts - "busta zero": il client non tocca la forma, la passa grezza
↳ al chiamante
return await res.json();
```

CONSIGLIO

Contratti elastici: Double Read, guardia di tipo, busta zero Stessa radice, tre rimedi. Quando un'API cresce per accrescimento e nessuno versiona la forma delle risposte, il client deve decidere quanto lavoro fare al posto del contratto mancante. SPW legge lo stesso payload in due forme possibili (il «Double Read» vero: array nudo o { data, total }). SR conosce la busta di ogni endpoint e, dove la forma è ambigua, mette una guardia `Array.isArray` che distingue il dato buono dall'errore. DIS non normalizza: ritorna il JSON com'è e si fida del chiamante. Nessuna delle tre è sbagliata; sono tre punti sulla scala di quanta incoerenza del server il client accetta di assorbire. Un avvertimento di terminologia: «Double Read» è *questo*, la lettura della forma del payload di successo. Non è il clonaggio della risposta per estrarne un messaggio d'errore, che è un'altra cosa e ha un'altra storia (il prossimo paragrafo).

*
**

3. Quando il backend ha torto: il 200 che mente e il messaggio perso

A volte il backend non collabora in due modi distinti. Il primo: risponde HTTP 200 anche su un fallimento logico, mettendo l'errore nel body. È il caso di alcuni endpoint di DIS (la newsletter), e il client lo rattoppa a mano, metodo per metodo.

```
// DIS api.ts:341-343 - alcuni endpoint tornano HTTP 200 anche in errore: ra
↳ ttoppo per-metodo
const data = await res.json();
if (data.status === 'error') throw new Error(data.message);    // non un inte
↳ rceptor unico
```

Attenzione all'attribuzione, perché è facile scambiare per una regola generale: questo è il pattern di DIS, non una prescrizione universale del modello. È la conseguenza di un backend che non usa sempre i codici HTTP, e il bridge se ne fa carico caso per caso. Il secondo modo in cui il backend non collabora è l'opposto: risponde con un errore preciso, e il client lo butta via. Per estrarre quel messaggio quando c'è, serve clonare la risposta.

```
// il blocco che ESTRAE il messaggio d'errore dal body (response cloning) -
↳ NON è il "Double Read"
if (!res.ok) {
  let err = 'Errore imprevisto dal server';
  try { const j = await res.clone().json(); err = j.message || err; } catch
↳ h (e) {}
  throw new Error(err);
}
return await res.json();
```

In DIS questo blocco compare identico in venticinque metodi. Non perché qualcuno l'abbia scritto venticinque volte, ma perché l'ha iniettato uno script.

```
// DIS fix_api.cjs:4-23 - regex-replace: trova ogni fetch e appende il blocco
↳ o, se manca
const regex = /(const res = await fetch\([\s\S]*?\))/g;
code = code.replace(regex, (match, p1, offset, string) => {
  const nextLine = string.substring(offset + p1.length).trim().split('\n')
↳ [0];
  if (nextLine.startsWith('if (!res.ok)')) return match; // salta chi ce
↳ l'ha già → i metodi "sfuggiti"
  const check = '\n if (!res.ok) { /* ... res.clone().json() ... */ throw new
↳ Error(err); }';
  return match + check;
});
```

ATTENZIONE

Il codemod che rattoppa il client: potenza e tracce DIS non ha scritto a mano la gestione degli errori del suo client: l'ha generata con un codemod. `fix_api.cjs` cerca ogni `const res = await fetch(...)` e, se non c'è già un `if (!res.ok)`, vi appende un blocco che clona la risposta ed estrae `message`. Funziona, e in pochi secondi copre l'intero file. Ma lascia tre impronte. La prima: lo stesso blocco identico, parola per parola, ovunque. La seconda: i metodi «sfuggiti», quelli che avevano già un loro `if (!res.ok)` con un messaggio generico (`login` con «Login failed», `uploadFile` con «Upload failed»), che lo script ha saltato lasciando il bridge disomogeneo. La terza: una riga duplicata, residuo di una seconda passata. È un caso reale di manutenzione del client per trasformazione automatica del sorgente, con i suoi vantaggi e i suoi effetti collaterali. Quel blocco è esattamente ciò che un manuale potrebbe vendere come «lo standard» della gestione errori: qui è il prodotto di una macchina, non di una scelta, ed è bene saperlo riconoscere.

Resta il problema più diffuso, e il più istruttivo perché comune a tutti e tre: il messaggio d'errore del backend che si perde. Il server (CAP 10) si è dato la pena di confezionare stati e testi precisi, un 429 con «Troppi tentativi, riprova tra quindici minuti». Il client lo getta, in tre punti diversi della catena.

```
// SPW api.ts:22 - il body d'errore (anche un 429 parlante) viene scartato q
↳ ui, nel client
if (!res.ok) throw new Error('Login fallito');
```

```
// SR LoginForm.tsx:18-25 - qui il body è preservato da api.login, ma la UI
↳ lo butta comunque
catch (err) { setError('Login fallito. Controlla le credenziali.')} // e
↳ rr.message ignorato
```


In SPW il messaggio si perde nel client, in SR nella UI, in DIS a macchia di leopardo (i metodi toccati dal codemod lo preservano, quelli sfuggiti no). Tre punti diversi, lo stesso esito: l'utente sotto rate-limit vede sempre «Login fallito» e mai il 429 che gli direbbe quanto aspettare.

ATTENZIONE

Leggere il body anche sui rami d'errore `res.ok` dice se la richiesta è andata a buon fine, non *perché* è fallita; il perché sta nel body. Buttarlo via, come fanno tutti e tre i siti sul login, vanifica il lavoro fatto sul server per rendere parlanti gli errori, e lascia l'utente davanti a un messaggio inutile. La lezione è semplice e spesso ignorata: il body va letto anche, e soprattutto, quando la risposta non è ok. Dove farlo conta meno del farlo: l'importante è non sostituire un 429 «riprova tra quindici minuti» con un generico «errore».

*
**

4. Il token CSRF lato client

È l'investimento che distingue SitoRuntime, e il meccanismo più sofisticato del cluster. Il backend di SR (CAP 10) restituisce un `csrf_token` nel body di `login` e `check_auth`, e pretende l'header `X-CSRF-Token` su tutte le mutazioni. Il client gestisce l'handshake nel modo più minimale possibile: una variabile a livello di modulo.

```
// SR api.ts:6-10 - il token CSRF vive in una variabile di modulo, in memori
↪ a
let csrfToken = ''; // non localStorage, non Context
↪ t, non stato di componente
function csrfHeaders() { return csrfToken ? { 'X-CSRF-Token': csrfToken } :
↪ {}; }
```

```
// SR api.ts:31,37 - catturato dal body di login/check_auth, rispedito solo
↪ sulle mutazioni
if (data.csrf_token) csrfToken = data.csrf_token;
// ...e su ogni POST/DELETE: headers: { 'Content-Type': 'application/json',
↪ ...csrfHeaders() }
```

Il token viene catturato all'login, reiniettato sulle scritture, azzerato al logout. SPW non ha niente di tutto questo (gli basta il controllo di Origin/Referer lato server), e DIS non ha CSRF affatto: è una soluzione tutta di SR. Ma tenerlo in una variabile di modulo ha un prezzo nascosto.

ATTENZIONE

Il token CSRF e il reload: una garanzia accoppiata La variabile vive in memoria: a un ricaricamento della pagina sparisce, e la prima mutazione successiva partirebbe senza `X-CSRF-Token`, prendendo un 403. Il sistema regge solo perché il componente admin rifà `checkAuth()` ogni volta che si monta, e quella chiamata ri-restituisce il token. È una dipendenza reale ma non dichiarata da nessuna parte: se domani una pagina admin montasse un editor senza passare da quel `checkAuth`, le scritture fallirebbero in modo opaco, con un 403 che a video è indistinguibile da un errore di salvataggio. Se un token deve sopravvivere al reload, va messo dove sopravvive

davvero (un `sessionStorage`, o un handshake esplicito a ogni avvio), non lasciato a un effetto collaterale del montaggio di un componente.

**

5. Proteggere l'area admin: loader o componente

La protezione delle rotte riservate lato client è due scuole per la stessa cosa. SPW usa una guardia dichiarativa: un loader montato sulla rotta padre `/admin`, che verifica la sessione prima che la pagina si monti e reindirige se manca.

```
// SPW loaders.ts:10-20 - una guardia, N pagine figlie: la sessione è verifi
↳ cata prima del render
export const adminAuthLoader = async () => {
  const session = await api.checkSession();
  if (!session || !session.user) return redirect('/admin/login');
  return session;
};
```

SR e DIS usano invece una guardia imperativa dentro un componente: `checkAuth` al montaggio, una macchina a stati che mostra il login finché non c'è un utente.

```
// SR Admin.tsx:74-189 - guardia dentro il componente, eseguita al montaggio
useEffect(() => { checkAuth(); }, []);
if (loading) return <Loader />;
if (!user) return <LoginForm onLogin={handleLogin} />; // nessun utente
↳ → form di login
```

Il confronto architetturale completo, con le sue conseguenze (per esempio la guardia di DIS che controlla l'utente ma non il ruolo), è al CAP 14. Qui conta una cosa sola.

NOTA

Loader o componente: due scuole, una sola difesa vera La differenza pratica tra le due è un lampo: col loader non si vede mai contenuto riservato prima del redirect; col componente c'è un istante di «loading» prima del verdetto. Ma è una differenza di esperienza, non di sicurezza. In tutti e tre i siti la difesa reale è il gate server-side (CAP 10), e la guardia client serve solo a non mostrare una porta che il server terrebbe comunque chiusa. Nascondere una pagina è esperienza utente; impedire un'azione è sicurezza, e va fatta sul server.

**

6. La sessione che scade mentre lavori

C'è un buco che attraversa tutti e tre i siti, ed è forse il più importante del capitolo perché nessuno lo copre. La guardia dell'area admin scatta una volta sola: quando navighi (SPW) o quando la pagina si monta (SR, DIS). Ma la sessione può morire *dopo*, mentre stai scrivendo un articolo, anche solo perché un cambio password fatto altrove l'ha invalidata via `session_version` (CAP 10).

```
// in tutti e tre: la mutazione riceve 401/403, il client mostra un errore g
↳ enérico
```

```
// e NON redirige. Manca un punto unico che riconosca lo status e forzi il r
↳ e-login.
```

ATTENZIONE

Gestire la scadenza di sessione nel thin stack A sessione morta, il salvataggio prende un 401, il client mostra «errore nel salvataggio» e ti lascia su una pagina che non funziona più, col lavoro ancora a schermo e nessun invito a rifare il login. Nessuno dei tre siti ha un interceptor che riconosca il 401/403 e forzi il re-login; in SR è perfino peggio, perché un 403 da token CSRF scaduto è identico, a video, a un errore qualsiasi. È il rovescio del fatto che la gestione errori vive sparsa in ogni metodo invece che in un solo strato: un piccolo `request(path, opts)` centralizzato, oltre a togliere la ripetizione del wrapper `fetch`, sarebbe il posto naturale dove gestire la sessione scaduta una volta per tutte, insieme al Double Read e agli header.

*
**

7. Upload e paginazione: le stesse tre mani

Restano due superfici dove le tre filosofie si vedono in piccolo. La prima è l'upload. Il bridge invia un `FormData`, e in tutti e tre c'è lo stesso accorgimento: si toglie l'header `Content-Type`, altrimenti il browser non scrive il boundary multipart e il file non arriva. Cambia il contorno.

```
// SPW api.ts:438-509 – per FormData si TOGLIE Content-Type; in alternativa
↳ XHR per la barra di avanzamento
const { headers, ...rest } = fetchConfig;
const res = await fetch(`${API_URL}/upload.php`, { ...rest, method: 'POST',
↳ body: formData,
  headers: { 'Accept': 'application/json' } }); // variante: XMLHttpRequest
↳ est + xhr.upload.onprogress
```

SPW offre la barra di avanzamento via `XMLHttpRequest`; SR manda il `FormData` con l'`X-CSRF-Token` ma senza progresso, un semplice spinner; DIS lo manda senza progresso e senza CSRF. Il lato server di tutto questo (la validazione, la conversione WebP, la catena RCE da upload pubblico di DIS) è il CAP 7.

La seconda superficie è la paginazione, ed è legata a doppio filo al Double Read del §2. È proprio il contratto `{ data, total }` a rendere possibile, e necessaria, la lettura in due forme: senza un `total` non si sa quante pagine restano.

```
// SPW useFetchArticles.ts:32-49 – il { data, total } del Double Read alimen
↳ ta il load-more con dedup
const data = Array.isArray(res) ? res : res.data;
const total = !Array.isArray(res) && res.total !== undefined ? res.total : d
↳ ata.length;
// ...accumula le pagine, deduplica per id, hasMore = (lista unita).length <
↳ total
```

SPW accumula le pagine deduplicando per `id` e calcola `hasMore` sulla lunghezza unita. SR fa il load-more senza dedup, fidandosi del backend per non ripetere. DIS non pagina affatto. E c'è una trappola silenziosa: quando l'endpoint torna un array nudo, senza `total`, SPW ripiega su `data.length` come totale, e `hasMore` diventa `false` anche quando

ci sarebbero altre pagine. È la conseguenza diretta di leggere un contratto che non è garantito. Il lato server (il `{ data, total }`, il `COUNT` e il `LIMIT/OFFSET`) è al CAP 9.

✱
✱ ✱

In sintesi

Il ponte tra React e PHP è in tutti e tre lo stesso oggetto su `fetch`, ma è cresciuto attorno a un'API senza contratto stabile, e ognuno ci ha messo del suo. SPW nello state layer, con i loader e il Double Read del payload. SR nel token CSRF tenuto in una variabile di modulo. DIS in un codemod che ha rattoppato la gestione errori a posteriori, lasciando le sue impronte (la ripetizione, i metodi sfuggiti, la riga duplicata). Tre investimenti diversi, e nessuno che renda il bridge davvero solido: il messaggio d'errore del backend si perde in tutti e tre, e in tutti e tre manca la cosa che servirebbe di più, un punto unico che gestisca la sessione scaduta. La morale non è «scegli il client più ricco». È che un wrapper su `fetch` non è mai solo trasporto: è il posto dove un'API imperfetta diventa, o non diventa, un'esperienza affidabile. Le decisioni che contano (leggere il body anche sugli errori, dove vive il token, cosa fare quando la sessione muore) stanno tutte lì.

IMPORTANTE Il Canone

- Un solo oggetto `api` su `fetch`, auth via cookie di sessione; per gli upload multipart toglie l'header `Content-Type` e lascia che lo metta il browser.
- Definisci una forma di payload stabile e leggila in un punto solo: un contratto uniforme è meglio del «Double Read».
- Gestisci il token CSRF sulle mutazioni e centralizza la reazione a 401/403 (un interceptor) per le sessioni scadute.
- Leggi il corpo della risposta anche sui rami d'errore: non perdere il messaggio del backend (es. il 429).

✱
✱ ✱

Prossimo Capitolo: Media & Optimization. L'upload come superficie d'attacco: validare i file davvero, spegnere PHP nelle cartelle pubbliche, e la catena che da un'immagine porta all'esecuzione di codice.

CAPITOLO 7: Media & Optimization

Questo capitolo segue un file dal disco di chi lo carica al disco del server: come arriva, come viene controllato, come viene alleggerito, dove finisce e come viene riservito. È un percorso breve, e proprio per questo pericoloso. Nei tre siti lo scheletro è lo stesso (un `upload.php` che riceve un `multipart`, lo valida, lo ottimizza con GD e lo mette su disco), ma è il cluster in cui la sicurezza scala **all'inverso del buon senso**.

La difesa contro l'esecuzione di codice da un upload va da **tre barriere indipendenti** (SimonePizziWebSite) a **una sola** (SitoRuntime) a **quasi zero su un upload pubblico** (DISINTELLIGENZA), dove la catena che porta dall'immagine alla shell è verificata. E due dettagli ribaltano l'intuizione: il naming «più minimale» (SR, che butta via il nome del file) è il **più sicuro**, mentre quello «più gentile» (DIS, che conserva nome ed estensione) è ciò che apre la porta. La domanda che attraversa il capitolo è una sola: *quanto puoi togliere a un sistema di upload prima che diventi insicuro*. E DIS mostra cosa c'è un passo oltre quel limite, perché un upload aperto al pubblico cambia tutte le regole.

Una nota di campo: qui si parla di **media**, non di cache né di SEO. La cache delle liste di contenuti vive nel ciclo di vita del contenuto (CAP 9); il pre-rendering dei metadati per i bot è il capitolo dell'entry-point PHP (CAP 11). Questo capitolo resta sul file: caricarlo, validarlo, ottimizzarlo, servirlo.

*
**

1. Lo scheletro comune

Prima delle divergenze, i tratti che i tre siti condividono.

Un `upload.php` riceve **un file per volta**: SOLO POST, `multipart/FormData` (inviato dal client del CAP 6), `$_FILES['file']`. Niente libreria di gestione media, niente servizio esterno; il file arriva nello stesso request che lo elabora. L'ottimizzazione dell'immagine avviene **sincrona, dentro l'endpoint**, via GD, dietro una guardia `extension_loaded('gd')` che degrada con grazia se GD manca: niente coda, niente worker, niente cron. Il nome viene reso anti-collisione con `uniqid()`. La «libreria» e la «cancellazione» vivono in un file separato (`media.php`), distinto dall'upload vero. E il riferimento al file, dentro i contenuti, è sempre una **stringa URL** (`cover_image`, `audio_file`), non una relazione con chiave esterna.

Quest'ultimo tratto ha una conseguenza condivisa, il **dangling media**: cancellare un file non avvisa chi lo citava, e nessuno dei tre tiene un conteggio dei riferimenti. Un'immagine usata come copertina, una volta cancellata, lascia un `<img>` rotto e nessuno se ne accorge finché la pagina non si apre.

```
// SPW upload.php - il flusso canonico: gate, valida, ottimizza, registra, r
↳ ispondi
Auth::check(); // solo l'admin carica (SPW/
↳ SR; DIS NO, vedi §5)
// 1) valida estensione + byte reali 2) sceglie sottocartella 3) nome anti
↳ -collisione
// 4) resize/WebP via GD 5) INSERT in `media` 6) echo { status, url, id, n
↳ ame }
```

2. Validare un file: l'estensione non basta, e nemmeno il Content-Type

Il primo lavoro dell'endpoint è decidere se il file è davvero ciò che dichiara di essere. SPW e SR lo fanno a **due strati**: prima l'estensione contro una whitelist, poi i **byte reali** letti con `finfo/mime_content_type` contro una whitelist di MIME. È il principio «non fidarti del nome del file»: un `shell.php.jpg` supera il primo filtro ma viene fermato dal secondo, perché i suoi byte non sono quelli di un'immagine. Lo stesso MIME reale decide anche la sottocartella, così la classificazione non si fida mai dell'estensione dichiarata.

```
// SPW upload.php:25-54 - due strati: estensione, poi i byte reali
$allowedExts = ['jpg', 'jpeg', 'png', 'webp', 'gif', 'pdf', 'zip', 'rar', 'mp3'];
if (!in_array($fileExt, $allowedExts)) { /* 400 */ }

$realMime = mime_content_type($file['tmp_name']); // legge i byte, non l'e
↳ stensione
$allowedMimes = ['image/jpeg', 'image/png', 'image/webp', 'image/gif', 'applicat
↳ ion/pdf', 'application/zip', /* ... */];
if (!in_array($realMime, $allowedMimes)) { /* 400: contenuto camuffato blocc
↳ ato */ }
```

DIS fa un'altra scelta, e qui comincia la storia di sicurezza del capitolo: si fida di `$_FILES['type']`, cioè del Content-Type che **dichiara il browser**. Nessun `finfo`, nessuna whitelist di estensioni, solo il valore che chi invia la richiesta controlla per intero.

```
// DIS upload.php:100 - la "validazione" guarda il MIME dichiarato dal clien
↳ t (spoofabile)
if (!in_array($file['type'], $allowed)) { // $file['type'] = heade
↳ r del browser
    http_response_code(400); die(json_encode(['status'=>'error', 'message'=>'
↳ Invalid file format']));
}
```

ATTENZIONE

`$_FILES['type']` non è una validazione Il Content-Type dentro `$_FILES` non lo calcola il server: lo scrive il client nella richiesta `multipart`, e si falsifica con una riga di `curl`. Validare contro quel valore vuol dire chiedere all'attaccante se il suo file è ammesso. La difesa reale è leggere i byte iniziali del file (`finfo_file / mime_content_type`) e confrontarli con una whitelist di MIME, usando il tipo *reale* anche per decidere dove salvarlo. SPW e SR lo fanno; DIS no, e quel «no» è il primo anello di una catena che vedremo al §5.

3. Il nome del file è un vettore

Anche il nome con cui il file finisce su disco è una decisione di sicurezza, e i tre siti la prendono in tre modi che corrispondono esattamente a tre livelli di rischio. SPW antepone un `uniqid()` a una base ripulita dai punti, così non può mai nascere un `shell.php.jpg` eseguibile via `mod_mime`. SR fa la mossa più radicale: scarta del tutto il nome originale e usa un `uniqid()` puro, nessuna stringa dell'utente entra nel nome finale. DIS conserva nome **ed** estensione, e i punti restano nel set di caratteri ammessi.

```
// SPW upload.php:73-75 - base SENZA punti interni, poi uniqid: niente doppi
↳ a estensione
$safeBase = preg_replace('/[^A-Za-z0-9\_\-]/', '', pathinfo($fileName, PATHINFO_FILENAME));
$newFileName = uniqid() . '-' . $safeBase . '.' . $fileExt;
```

```
// SR upload.php:62 - elisione totale: il nome utente è buttato, resta solo
↳ uniqid + estensione tecnica
$baseId = uniqid('', true); // nessun input utente nel n
↳ ome del file
```

```
// DIS upload.php:110-111 - conserva nome ed estensione (i punti sono permes
↳ si): il più debole
$filename = uniqid() . '_' . basename($file['name']);
$filename = preg_replace('/[^a-zA-Z0-9\_\-]/', '', $filename); // il "." re
↳ sta -> estensione preservata
```

CONSIGLIO

Meno ti fidi del nome, più sei al sicuro: la scala dell'elisione È il punto più contro-intuitivo del capitolo. La scelta «più gentile», conservare il nome che l'utente ha dato al file, è la più rischiosa, perché lascia all'attaccante il controllo dell'estensione finale. La scelta «più brutale», cancellare il nome e sostituirlo con un identificatore generato dal server, è la più sicura, perché toglie ogni appiglio. SR sta all'estremo protettivo non perché abbia aggiunto una difesa, ma perché ne ha tolta una superficie: il nome del file non è un dato da preservare, è un input da neutralizzare.

4. Difesa in profondità: tre barriere, una, zero

La validazione e il naming sono difese applicative, e vivono dentro `upload.php`. Ma cosa succede se quel punto viene aggirato, o se domani si aggiunge un secondo modo di scrivere nella cartella? Qui entra la barriera che non dipende dal codice PHP: l'`.htaccess` della cartella `uploads/` che **spegne il motore PHP**. Se Apache non interpreta più i `.php` lì dentro, un file malevolo caricato resta un file inerte, qualunque cosa sia successa prima.

```
# SPW public/uploads/.htaccess - la PRIMA barriera anti-RCE: niente esecuzione
↳ ne PHP qui dentro
php_flag engine off
<FilesMatch "\.(php|phtml|phar|cgi|pl)$">
    Require all denied
</FilesMatch>
```

È facile guardare l'`.htaccess` di `uploads/` come una faccenda di **cache-control** (`Expires`, `max-age`), un dettaglio di prestazioni, e fermarsi lì. Il suo uso critico è un altro: è lo

spegnimento di PHP, ed è la prima delle tre barriere indipendenti di SPW. Le altre due le abbiamo già viste: il naming che non genera nomi eseguibili (§3) e la validazione sui byte reali (§2). Tre reti per lo stesso rischio, e ognuna copre il buco dell'altra: se l'.htaccess non venisse letto, salva il naming; se il naming fallisse, salva l'.htaccess; il contenuto camuffato lo ferma il controllo MIME.

SR ha una sola di queste reti, la validazione applicativa: non esiste un uploads/.htaccess (la cartella è creata a runtime e non è nel repository), e l'.htaccess globale non spegne PHP. Finché upload.php resta l'unica via di scrittura, regge; ma non c'è la seconda rete. DIS non ha praticamente nessuna delle tre.

ATTENZIONE

Una sola barriera non è difesa in profondità «Validare bene l'upload» è necessario, non sufficiente. La difesa in profondità significa che ogni singola barriera può fallire senza che il sistema cada, perché ce n'è un'altra dietro. Spegner PHP nella cartella degli upload costa due righe di .htaccess e trasforma un'eventuale falla nella validazione da «esecuzione di codice» a «file inutile sul disco». È la barriera con il miglior rapporto tra costo e protezione di tutto il capitolo, ed è anche quella che SR e DIS non hanno.

✱
✱✱

5. La tempesta perfetta: la catena RCE da upload pubblico

Le condizioni viste finora, prese una alla volta, sarebbero gestibili. In DIS si sommano, e su un fronte che gli altri due siti non hanno: l'**upload pubblico**. Essendo un sito-festival, DIS accetta le tracce audio caricate dai partecipanti durante l'iscrizione, e per abbassare l'attrito quel caricamento non richiede login. Il gate è deciso per tipo, e per due tipi è semplicemente assente.

```
// DIS upload.php:64-98 - il gate è per-tipo e INCOERENTE: audio_participant
↪ è pubblico
if ($type === 'image') {
    if (!isset($_SESSION['user_id'])) { http_response_code(401); die(...); }
    ↪ // GATED
    $uploadDir = __DIR__ . '/../uploads/images/';
} elseif ($type === 'audio_participant') {
    $uploadDir = __DIR__ . '/../uploads/audio/participants/';
    ↪ // NESSUN gate auth
} elseif ($type === 'audio_podcast') {
    if (!isset($_SESSION['user_id'])) { http_response_code(401); die(...); }
    ↪ // GATED
    $uploadDir = __DIR__ . '/../uploads/audio/podcasts/';
}
```

Adesso si mettono in fila i quattro anelli. Primo: type=audio_participant non chiede login. Secondo: la validazione guarda solo \$_FILES['type'], il MIME dichiarato dal browser (§2), falsificabile. Terzo: il naming conserva nome ed estensione (§3). Quarto: non c'è uploads/.htaccess e l'.htaccess globale non spegne PHP (§4), nega soltanto i file .sqlite e .bak. Il risultato è una richiesta sola.


```
# La catena: una POST pubblica deposita uno script PHP eseguibile
curl -F 'type=audio_participant' -F 'file=@shell.php;type=audio/mpeg' https:
↳ //sito/api/upload.php
# -> nessun login richiesto -> il MIME "audio/mpeg" dichiarato passa la w
↳ hitelist
# -> salvato come /uploads/audio/participants/<uniqid>_shell.php -> Apach
↳ e lo esegue
```

ATTENZIONE

L'upload pubblico cambia tutte le regole Un upload dietro login è un problema di igiene; un upload pubblico è una superficie d'attacco aperta a Internet, e va trattato con la massima diffidenza. Le quattro debolezze di DIS, da sole, sarebbero rilievi minori. Insieme, su un endpoint senza autenticazione, producono l'esecuzione di codice remoto: il caso peggiore. La difesa non è una sola contromisura ma la loro somma: autenticare dove si può, validare sui byte reali, neutralizzare il nome, e soprattutto spegnere PHP nella cartella, così che anche se tutto il resto cede il file resti inerte (CAP 10). Quando l'upload è pubblico (le iscrizioni del festival, CAP 17), queste non sono raccomandazioni: sono il minimo.

C'è un corollario sgradevole. FDCA, il fork di DIS, ha un `upload.php` byte-identico: eredita la catena intatta, l'upload pubblico, il naming debole e l'assenza di PHP-off, immutati. Un difetto di sicurezza copiato con un `git clone` si moltiplica senza che nessuno lo riscriva. E si noti il ribaltamento rispetto al solito ritornello «più ingegnerizzato uguale più fragile»: qui il sito più scarno (SR, che ha tolto tutto il toglibile) è più sicuro del sito più «accogliente» (DIS, che ha aggiunto l'apertura al pubblico senza aggiungere le difese che quell'apertura richiede).

*
**

6. Ottimizzare l'immagine: WebP, resize, e cosa il libro prometteva di troppo

Superata la validazione, l'immagine viene alleggerita. Qui le scelte divergono, ed è facile appiattirle in una regola unica che il codice reale non rispetta.

SPW e SR **convertono** le raster in WebP via GD (qualità 82, ridimensionamento se la larghezza supera 1920px), poi cancellano l'originale. DIS **non converte**: ridimensiona soltanto, mantenendo il formato di partenza (un PNG resta un PNG, più piccolo). La «transcodifica WebP obbligatoria, standard ufficiale» è dunque il pattern di due siti su tre, non di tutti.

```
// SPW upload.php:83-119 - conversione WebP + resize, sincrona nell'endpoint
↳ , dietro guardia GD
if (in_array($realMime, ['image/jpeg','image/png','image/gif']) && extension
↳ _loaded('gd') && function_exists('imagewebp')) {
    if ($origW > 1920) { /* imagecopyresampled a 1920px, alpha preservato */
↳ }
    if (@imagewebp($img, $webpDestination, 82)) { unlink($destination); /* 1
↳ 'originale se ne va */ }
}
```

Tre dettagli meritano una precisazione netta. La GIF è una scelta, non un automatismo: SPW la appiattisce in un fotogramma (l'animazione si perde), SR la **preserva animata** escludendola dalla conversione. La ricodifica GD, come effetto collaterale, **strippa l'EXIF**: la geolocalizzazione e il modello del dispositivo spariscono senza che nessuno lo chieda, una piccola difesa di privacy ottenuta gratis (SR la annota, gli altri no). E il vincolo di dimensione tocca solo la **larghezza** sopra i 1920px: un limite in altezza non c'è, quindi un'immagine altissima e stretta resta enorme.

NOTA

WebP+resize nel thin stack: le varianti, e i suoi limiti L'ottimizzazione è sincrona: avviene dentro il request di upload, senza coda né worker. Su un'immagine grande l'utente aspetta GD prima di vedere la risposta, ma per un sito a basso traffico (con la barra di avanzamento del client, CAP 6) è un compromesso ragionevole. Le varianti reali sono tre: converti in WebP appiattendo la GIF (SPW), converti preservando la GIF animata (SR), ridimensiona soltanto lasciando il formato com'è (DIS). Nessuna è «quella giusta» in assoluto; quella sbagliata è dichiarare universale una regola che due righe di codice contraddicono.

**

7. La libreria e la cancellazione: quando il disco è il database

Caricato il file, serve elencarlo e poterlo cancellare. Su questo asse SPW si separa dagli altri due. SPW tiene una tabella `media` con due colonne-nome: `file_path`, l'URL tecnico che carica il browser, e `filename`, il nome umano originale. Quel secondo nome serve a una cortesia: `download.php`, un proxy pubblico che fa lo streaming del file con `readfile`, restituisce all'utente `relazione.pdf` invece di `64f1a2-relazione.pdf`.

SR e DIS non hanno tabella: la libreria è il filesystem, letto con `scandir` (piatto in SR, ricorsivo in DIS). Il disco è il database dei media. Semplice, ma con un costo: niente nome originale, niente MIME salvato, ordinamento per data fisica del file. E il dangling media, comune a tutti, qui diventa pure impossibile da tracciare, perché manca persino un punto da cui partire per contare i riferimenti. In DIS gli orfani sono tracce di concorso: dati che si perdono.

La cancellazione è il punto più delicato, perché è una mutazione che tocca il disco, e va protetta su due fronti: il percorso e la richiesta. Sul percorso, i tre siti scalano di nuovo. SPW risolve il path con `realpath` e verifica che stia davvero dentro `/uploads/`, senza fidarsi nemmeno del proprio database. SR usa solo `basename()`. DIS si limita a rifiutare i `..` con uno `strpos`, poi tenta l'`unlink` su più percorsi candidati.

```
// SPW media.php:31-39 - path-guard con realpath: non si fida nemmeno del DB
$physicalPath = realpath(__DIR__ . '/../' . $filePath);
$uploadsBase  = realpath(__DIR__ . '/../uploads');
if ($physicalPath && $uploadsBase && str_starts_with($physicalPath, $uploads
↳ Base)) {
    unlink($physicalPath);
    // solo se DAVVERO contenut
↳ o in /uploads
}
```

```
// SR media.php — niente auth_utils, sessione nuda, e NESSUN validateCsrf su
↳ lla delete
session_start();
if (!isset($_SESSION['user_id'])) { http_response_code(401); exit; } // no
↳ CSRF, no controllo di ruolo
$filename = basename($_input['filename'] ?? ''); // unica difesa traversal
if (file_exists($uploadDir.$filename)) unlink($uploadDir.$filename);
```

ATTENZIONE

La delete dei media: il path-guard e il token che manca Sul secondo fronte, la richiesta, SR e DIS hanno lo stesso buco: la cancellazione **non ha protezione CSRF**. In SR `media.php` non include nemmeno il prelude di autenticazione del resto del sito, gira con una sessione nuda e non controlla il ruolo; in DIS la protezione CSRF non esiste affatto. Significa che un sito malevolo può far cancellare file all'amministratore loggato con una richiesta forgiata, mitigato solo dal `SameSite` del cookie (CAP 10). È l'incoerenza tipica: l'upload chiede il token, la delete no, eppure cancellare è distruttivo quanto caricare. Un endpoint che cambia lo stato va protetto come tale, sempre, anche quando vive in un file «di servizio» a cui si presta meno attenzione.

*
**

8. Far evolvere lo storage senza fermare il sito

Ogni sito porta le cicatrici di una migrazione dello storage: da raster a WebP, da cartella piatta a sottocartelle. Si leggono negli script one-shot, e raccontano la stessa sequenza: prima gli upload piatti, poi una conversione batch di ciò che c'era già, poi lo smistamento in sottocartelle, infine il riallineamento dei riferimenti rimasti puntati al vecchio percorso.

La differenza che conta è quanta protezione hanno questi script potenti, che spostano file e riscrivono il database. Quelli di SPW seguono il pattern «carica via FTP, esegui dal browser, cancella subito», con il dry-run attivo di default (si guarda prima di toccare). Quelli di SR vivono dentro `admin.php`, dietro il login. Quello di DIS, `migrate_media.php`, **non ha alcun gate**: chiunque, da Internet, può innescare lo spostamento massivo dei file e l'aggiornamento delle righe. La meccanica completa di queste migrazioni, e la disciplina che richiedono, è il capitolo sull'evoluzione del database (CAP 15); qui basta il sintomo, e l'avvertenza che la manutenzione potente esposta in HTTP è una porta sul retro quanto l'upload sul davanti.

*
**

In sintesi

Il lato media dei tre siti parte dallo stesso scheletro e diverge su un solo asse che conta davvero: quante difese indipendenti stanno tra un file caricato e l'esecuzione di codice. SPW ne ha tre (PHP spento nella cartella, naming che non genera eseguibili, validazione sui byte reali), e ognuna copre il fallimento dell'altra. SR ne ha una, la

validazione applicativa, robusta finché resta l'unica via di scrittura. DIS ne ha quasi nessuna, e per giunta su un upload pubblico: la somma di gate assente, MIME fidato dal client, nome conservato e PHP non spento è la catena RCE che il fork FDCA ha pure ereditato intatta. Le scelte di ottimizzazione (WebP o solo resize, GIF appiattita o animata) e di archiviazione (tabella `media` o filesystem nudo) contano per le prestazioni e per la manutenibilità, ma non spostano il rischio. Il rischio sta nel nome del file, nei byte che non controlli e nel motore che non hai spento. La regola del capitolo è che un upload non va reso «più ricco»: va reso ridondante, perché la prima barriera, prima o poi, cede.

IMPORTANTE **Il Canone**

- Valida i file per magic-bytes (`finfo`), mai per il MIME dichiarato dal client.
- Rinomina con `uniqid()` e scarta il nome originale; ricostruisci l'estensione da una allowlist.
- Difesa in profondità: `.htaccess` con PHP-off nelle cartelle di upload (prima barriera anti-RCE), più validazione e naming. Una barriera sola non è difesa in profondità.
- Mai un upload pubblico non gated verso una cartella eseguibile; la delete dei media passa per un path-guard (`realpath/containment`) e un token CSRF.

Prossimo Capitolo: Advanced Content Editing & Media Integration. L'editor del contenuto, e come l'HTML che produce viene tenuto al sicuro al momento di mostrarlo.

CAPITOLO 8: Advanced Content Editing

L'editor di contenuti è il punto in cui un CMS si fa toccare con mano: è lì che l'autore scrive, formatta, incolla, inserisce un'immagine o un video. Sembra un dettaglio di interfaccia, ma porta con sé due domande di fondo che questo capitolo tiene insieme. La prima è quanto editor ti serve davvero, perché si va da una libreria di una decina di pacchetti fino a zero dipendenze. La seconda, più seria, è dove vive la difesa contro l'XSS: il contenuto che l'autore compone è HTML, quell'HTML finisce nel database, e prima o poi qualcuno lo stampa in pagina. Se nessuno lo ripulisce, un `<script>` scritto da chi ha accesso all'editor diventa codice eseguito nel browser di ogni lettore.

I tre siti rispondono su tre gradini. SimonePizziWebSite (SPW) ha un editor Tiptap blindato, con guardie all'inserimento e sanitizzazione al render. SitoRuntime (SR) usa lo stesso motore Tiptap, con la stessa difesa al render, ma con guardie più deboli e una storia in più alle spalle: una migrazione da Quill che ha lasciato cicatrici nel codice. DISINTELLIGENZA (DIS) scende al gradino artigianale, un `contentEditable` pilotato a mano, e qui la difesa XSS sparisce del tutto. La scala dell'editor e la scala della sicurezza, vedremo, non coincidono: il discrimine vero non è quale motore usi, ma se ripulisci l'HTML al momento di stamparlo.

NOTA

Due malintesi da sciogliere subito. È facile pensare a questo editor come «una soluzione native-React che rinuncia alle pesanti dipendenze esterne». È vero solo per DIS. I due flagship (SPW e SR) usano **Tiptap v3** (ProseMirror), cioè una decina di pacchetti `@tiptap/*`: una dipendenza esterna tutt'altro che leggera. E la «Paste Protection», che a prima vista sembra una difesa capace di rimuovere «script e attributi pericolosi», è in realtà solo una pulizia cosmetica dell'incolla. La difesa XSS reale è altrove, al momento del render, come vedremo in questo capitolo.

1. L'editor: una scala a tre gradini

Lo stesso bisogno, comporre HTML ricco, è risolto a tre livelli di dipendenza.

1.1 SPW: Tiptap v3 «blindato»

SPW fattorizza un componente `RichTextEditor` riusabile attorno a Tiptap v3, con un set mirato di estensioni: testo formattato, immagini, colore, allineamento, tabelle, embed YouTube in modalità `nocookie`. Ogni estensione porta le sue classi Tailwind di default, così l'HTML salvato è già stilizzato per il render pubblico.

```
// src/components/admin/RichTextEditor.tsx:95-138 (estratto)
const editor = useEditor({
  shouldRenderOnTransaction: true, // v3: serve per gli stati attivi d
  ella toolbar
  extensions: [
    StarterKit.configure({ heading: { levels: [1, 2, 3, 4] } }),
    Image.configure({ HTMLAttributes: { class: 'max-w-full h-auto rounde
  d-x1 my-4 ...' } }),
    TextStyle, Color,
    TextAlign.configure({ types: ['heading', 'paragraph'] }),
    Table.configure({ resizable: true }), TableRow, TableHeader, TableCe
  ll,
    Youtube.configure({ nocookie: true }),
  ],
  content: value,
  onUpdate: ({ editor }) => { onChange(editor.getHTML()); updateCounts(edi
  tor.getText()); },
});
```

1.2 SR: lo stesso Tiptap, ma con una migrazione alle spalle

SR usa lo stesso motore, però incorporato direttamente dentro `ArticleEditor.tsx` (toolbar e `useEditor` nello stesso file), e ne tiene una seconda copia in `SpeakerEditor.tsx` per la biografia degli speaker. L'editing ricco non è centralizzato in un componente unico come in SPW: è duplicato per dominio, e una modifica alla configurazione va replicata in due posti.

La sua identità, però, sta altrove: SR è migrato da Quill a Tiptap, e la migrazione è scritta nel codice. Gli articoli vecchi contengono `<iframe>` YouTube «nudi» lasciati da Quill, che Tiptap non sa più editare. Una funzione li riavvolge nel formato che Tiptap riconosce, prima di passarli all'editor:

```
// src/components/admin/ArticleEditor.tsx:29-36
// Converti i bare iframe di Quill in div[data-youtube-video] che Tiptap sa
  editare.
const QUILL_YT_RE = /<iframe\b(?:\s|>)*\bsrc=["'](?:https?:\/\/(?:www\.)?youtube(
  \.com\/embed\/[^\s">]+)[^"]*(?:<\/iframe>|\/>)/gi;
function prepareForEditor(html: string): string {
  return html.replace(QUILL_YT_RE, (_m, src) =>
    `<div data-youtube-video=""><iframe src="${src}" allowfullscreen="tr
  ue" frameborder="0"></iframe></div>`);
}
// uso: setLoadedContent(prepareForEditor(res.article.content || ''));
```

Restano altre due tracce della vecchia libreria: la classe CSS `q1-video` di Quill, tenuta dentro la configurazione di Tiptap, e il file `react-quill.d.ts` ancora nel repo. Tre indizi convergenti di un cambio di editor mai del tutto ripulito.

CONSIGLIO

Cambiare editor a contenuti già scritti: il compatibility shim Quando si cambia il motore dell'editor, i contenuti vecchi restano nel formato di prima. Hai due strade: una migrazione una-tantum che riscrive tutti i record nel DB, oppure uno shim che converte il vecchio formato «al volo», ogni volta che apri un contenuto per modificarlo. SR ha scelto la seconda, con `prepareForEditor()`. Il vantaggio è che non tocchi il database e non rischi una migrazione di massa andata storta; il prezzo è che lo

shim resta nel codice per sempre, o almeno finché esiste un solo articolo vecchio non riaperto. È un debito che si paga a rate piccole invece che in un colpo solo.

1.3 DIS: l'editor artigianale

DIS rinuncia del tutto alla libreria. L'editor è un `<div contentEditable>` pilotato da `document.execCommand`, l'API del DOM ormai deprecata. L'unica dipendenza esterna è showdown, e serve solo a convertire il markdown incollato.

```
// src/components/RichTextEditor.tsx:94-102 (estratto)
const exec = (command: string, value?: string) => {
  document.execCommand(command, false, value); // API deprecata
  if (editorRef.current) onChange(editorRef.current.innerHTML);
};
// <div ref={editorRef} contentEditable onInput={handleInput} className="pro
< se prose-invert ..." />
```

Funziona, e nei browser di oggi gira ancora. Ma è l'opzione più fragile: il `contentEditable` produce HTML diverso da un browser all'altro, `execCommand` è deprecato e il suo comportamento futuro non è garantito.

NOTA

Quanto editor ti serve davvero DIS dimostra il minimo assoluto, zero pacchetti per l'editing, e insieme ne mostra il prezzo: HTML inconsistente, API deprecata, e come «source of truth» l'`innerHTML` grezzo del DOM invece dell'output controllato di un serializzatore come `getHTML()`. La scala non è «più pacchetti uguale meglio». È una scelta di compromesso: SPW e SR pagano una decina di dipendenze per avere uno schema controllato e una UX ricca; DIS non paga nulla e accetta la fragilità. Quello che DIS non può permettersi di togliere, lo vedremo al §3, non è l'editor: è la difesa al render.

✱
✱ ✱

2. L'HTML è la source of truth, e il server lo salva grezzo

Sotto le tre implementazioni c'è una scelta architetturale comune. Nessuno dei tre tiene uno stato JSON o un AST proprietario da ri-serializzare: l'editor emette HTML (il `getHTML()` di Tiptap, o l'`innerHTML` del `contentEditable`) e quell'HTML è esattamente ciò che finisce in `articles.content` o `news.content`. Il contenuto è già nella forma in cui sarà mostrato.

E il server non lo tocca. Coerentemente con quanto visto al CAP 9 sul ciclo di vita dei contenuti, il backend salva il corpo HTML così com'è, senza `strip_tags` né `htmlspecialchars`:

```
// SPW articles.php:252 - il content entra grezzo nel DB
$content = $data['content'] ?? ''; // nessun strip_tags
< / htmlspecialchars
$stmt = $pdo->prepare("INSERT INTO articles (title, slug, content, ...) VALU
< ES (?, ?, ?, ...)");
$stmt->execute([$title, $slug, $content, ...]); // HTML così com'è
```

Questa non è una svista, è il modello: il contenuto ricco viene conservato fedele e la difesa si sposta al momento in cui lo si stampa. Tutto il peso della sicurezza, quindi, grava su un solo gesto. Vediamo dove.

**

3. Sicurezza dei contenuti ricchi: dove vive la difesa XSS

Il content è salvato grezzo e scrivere è riservato agli autenticati (login admin o editor, come al CAP 10). Si tratta quindi di uno stored-XSS da autore autenticato: non è il visitatore anonimo a iniettare lo script, ma chi ha accesso all'editor. Il rischio resta reale, perché un account editor compromesso, o un copia-incolla distratto da una fonte ostile, basta a piazzare HTML pericoloso nel database. La domanda è: c'è un punto in cui quell'HTML viene ripulito prima di arrivare al browser del lettore?

3.1 Il choke-point: DOMPurify a render-time

In SPW la risposta è una sola funzione, chiamata appena prima dell'unico dangerouslySetInnerHTML del componente pubblico. Passa l'HTML per DOMPurify, e con un hook ammette solo gli <iframe> di YouTube, rimuovendo qualunque altro:

```
// src/components/SingleArticle.tsx:28-45
const sanitizeArticleHtml = (html: string): string => {
  DOMPurify.addHook('uponSanitizeElement', (node, data) => {
    if (data.tagName === 'iframe') {
      const src = (node as HTMLInputElement).getAttribute('src') || '';
      if (!src.startsWith('https://www.youtube.com/embed/') &&
        !src.startsWith('https://www.youtube-nocookie.com/embed/'))
        node.parentNode?.removeChild(node); // iframe non-YouTube
    }
  });
  try {
    return DOMPurify.sanitize(html, {
      ADD_TAGS: ['iframe'],
      ADD_ATTR: ['style', 'allowfullscreen', 'frameborder', 'allow', '']
    }, {
      src: ''
    });
  } finally {
    DOMPurify.removeHooks('uponSanitizeElement'); // niente hook glob
  }
};
```

Due dettagli meritano attenzione. L'hook viene sempre rimosso nel finally, così non resta registrato a livello globale tra un render e l'altro. E l'aver concesso l'attributo style allarga la superficie: serve per i colori inline dell'editor, ed è DOMPurify a ripulire il CSS pericoloso, ma è un compromesso tra fedeltà visiva e sicurezza da tenere presente.

SR fa la stessa cosa, con DOMPurify, su tutte le pagine di dettaglio (articolo, speaker, podcast). Stessa filosofia, stesso choke-point a render-time.

DIS non lo fa. NewsDetail.tsx inietta l'HTML grezzo, e dompurify non è nemmeno tra le dipendenze del progetto:


```
// src/pages/NewsDetail.tsx - nessuna sanitizzazione
<div dangerouslySetInnerHTML={{ __html: news.content }} /> // HTML del DB
↪ iniettato così com'è
```

ATTENZIONE

Dove sanitizzare l'HTML di contenuto: il choke-point a render-time Salvare grezzo e ripulire al render è una scelta legittima, a una condizione: che il render ripulisca davvero, e che sia l'unico modo in cui quel contenuto raggiunge un browser. SPW e SR rispettano la condizione con DOMPurify. DIS no, ed è l'unico dei tre dove lo stored-XSS non incontra alcun choke-point: grezzo in scrittura, grezzo al render. La sua unica difesa è il confine admin/editor sull'accesso all'editor. Zero difesa in profondità. Il discrimine tra i due siti sicuri e quello scoperto non è l'editor (Tiptap contro `contentEditable`): è la presenza o l'assenza di questa funzione.

3.2 Le guardie all'inserimento: un secondo livello, non il primo

C'è una seconda difesa, più visibile all'autore ma meno decisiva. Quando si inserisce un link dalla toolbar, SPW ne valida l'URL sul posto, bloccando gli schemi pericolosi:

```
// src/components/admin/RichTextEditor.tsx:36-42
const isSafeLinkUrl = (url: string): boolean => {
  const trimmed = url.trim();
  return /^https?:\/\//i.test(trimmed) || trimmed.startsWith('/')
    || trimmed.startsWith('#') || /^mailto:/i.test(trimmed); // blocca
↪ javascript: e data:
};
```

SR e DIS non hanno questo filtro. SR fa `setLink({ href: url })` nudo; DIS prende l'URL da un `prompt()` e lo passa a `createLink` senza guardarlo, così un `javascript:alert(1)` digitato finisce tale e quale nell'HTML:

```
// DIS RichTextEditor.tsx:104-107
const promptLink = () => {
  const url = prompt('Inserisci URL:');
  if (url) exec('createLink', url); // nessun filtro: 'javascript:...'
↪ verrebbe inserito così com'è
};
```

CONSIGLIO

Due livelli, un solo choke-point reale La guardia all'inserimento (`isSafeLinkUrl`) e la sanitizzazione al render (DOMPurify) non sono intercambiabili. La prima copre solo ciò che passa dalla toolbar, e non tocca l'HTML incollato o costruito in altro modo: è difesa in profondità e buona UX, perché avvisa subito l'autore invece di lasciar passare l'URL fino alla pagina pubblica. Ma la barriera vera, quella che intercetta qualunque HTML comunque sia entrato nel DB, è il render. Se devi sceglierne una sola, scegli il choke-point al render. Averle entrambe è meglio; avere solo la prima è un'illusione di sicurezza.

3.3 Ripulire l'incolla non è difendersi dall'XSS

Resta da sciogliere il malinteso della vecchia «Paste Protection». DIS, quando intercetta un incolla, fa due cose: se riconosce del markdown lo converte con `showdown`, altrimenti

ripulisce l'HTML dagli stili inline e dalle `class` di Word o Wikipedia. Utile per non importare font e sfondi indesiderati. Ma non è sicurezza:

```
// DIS RichTextEditor.tsx:63-85 (estratto) - pulizia COSMETICA
const doc = new DOMParser().parseFromString(htmlText, 'text/html');
doc.querySelectorAll('[style]').forEach(el => { el.style.backgroundColor='';
↳ el.style.color=''; });
doc.querySelectorAll('*').forEach(el => el.removeAttribute('class')); // t
↳ ooglie stili/classi, NON script/handler
document.execCommand('insertHTML', false, doc.body.innerHTML);
```

Toglie `style` e `class`, ma lascia passare `<script>`, gli attributi `on*` e gli URL `javascript:..` Il commento nel codice parla di «classi nemiche», non di XSS, ed è onesto: è cosmesi. Va distinta nettamente da una sanitizzazione vera. Rimuovere la formattazione di Word migliora l'aspetto del testo; non protegge nessuno.

**

4. Box-ancora: i quattro emettitori del content

Qui si apre un filo che attraversa quattro capitoli, e che conviene fissare una volta sola. Il punto di partenza è ciò che abbiamo appena visto: il `content` è salvato grezzo nel database, e la sanitizzazione vive solo nel render React. Ma il render React non è l'unico posto da cui quel contenuto esce verso il mondo. Lo stesso `articles.content` viene riletto e riemesso da almeno quattro «emettitori» diversi, e ognuno deve difendersi per conto suo. La difesa che vive in un solo componente non copre gli altri tre.

Emettitore	Dove	Cosa fa col content	Difesa anti-XSS	Esito
Render React	pagina pubblica (questo capitolo)	emette via dangerouslySetInnerHTML	DOMPurify (SPW, SR) / niente (DIS)	choke-point reale; DIS scoperto
Prerender SEO	index.php per i bot (CAP 11)	ne riemette il corpo HTML	strip_tags con allowlist (SPW, SR) / non emette il corpo (DIS)	buccia attributi vivo in SPW e SR; DIS immune per sottrazione
Feed RSS	rss.php/feed_* (CAP 12)	emette l'articolo	non emette il content (SPW usa l'excerpt; DIS fa solo podcast) / lo escapa (SR: strip_tags + htmlspecialchars)	chiuso
Newsletter	invio email (CAP 13)	manda l'articolo	nessuno emette il content	chiuso

La lettura è questa. Il feed e la newsletter chiudono il problema, perché o non emettono il corpo o lo escapano del tutto. Il render lo chiude dove c'è DOMPurify e lo lascia aperto in DIS. L'unica falla che resta viva nei flagship è il prerender SEO: per servire ai bot un corpo indicizzabile, SPW e SR riemettono il `content` passandolo per `strip_tags` con

una allowlist di tag, che non è DOMPurify. `strip_tags` rimuove i tag non ammessi ma non tocca gli attributi: un `onerror` o un `javascript:` dentro un tag permesso sopravvive. E poiché SR ha copiato il motore SEO di SPW alla lettera, ne ha copiato anche la falla.

IMPORTANTE

La lezione del quadro: una sanitizzazione server-side condivisa Quando salvi il contenuto grezzo e affidi la pulizia a chi lo stampa, stai scommettendo che *ogni* punto di stampa si ricordi di ripulire. Bastano quattro emettitori e basta che uno solo dimentichi: in DIS dimentica il render, nei flagship dimentica il prerender. La conclusione che attraversa i prossimi tre capitoli è che la sanitizzazione dovrebbe vivere una volta sola, lato server, là dove il contenuto è prodotto o riletto, invece di essere reinventata da ogni consumatore. Il filo si riapre al CAP 11 (dove il buco del prerender è vivo), si richiude al CAP 12 (il feed che escapa) e si chiude del tutto al CAP 13 (la newsletter che non emette).

*
**

5. Inserire media nel contenuto

Una cosa è gestire la libreria dei media (upload, ottimizzazione, formati: è il CAP 7); un'altra è incorporare un'immagine *dentro* il testo dell'articolo. Qui interessa la seconda, ed è un altro punto in cui i tre siti divergono.

SPW apre una galleria modale, `MediaSelectorModal`, che riusa lo stesso bridge di upload del CAP 7: l'autore sceglie un'immagine già caricata oppure ne carica una nuova con barra di progresso, e l'URL torna all'editor.

```
// src/components/admin/MediaSelectorModal.tsx:65-71 (estratto)
const result = await api.uploadMediaWithProgress(file, (p) => setUploadProgr
< ess(p));
if (result.url) onSelect(result.url);    // l'URL diventa un <img src> nell
< 'editor
// galleria: onClick={() => onSelect(item.file_path)}
```

SR salta la galleria: un `<input type="file">` creato al volo, upload diretto, e via. Più semplice, ma niente riuso di immagini già presenti. DIS non permette affatto immagini nel corpo dell'articolo: l'editor copre solo testo formattato e link, e la cover si gestisce altrove.

Sul versante della portabilità, tutti e tre salvano nel database **solo percorsi relativi** (per esempio `/api/uploads/file.jpg`), così il contenuto non si lega a un dominio. Quando serve un URL assoluto, al momento di copiare o condividere un link, è il client a ricostruirlo da `window.location.origin`. La cosa salvata resta portabile; la cosa mostrata si adatta al contesto.

*
**

6. Le micro-cure editoriali

Sotto le differenze, l'esperienza di scrittura condivide alcune attenzioni che vale la pena raccogliere, perché sono il genere di dettaglio che distingue un editor usabile da uno che frustra.

Tutti e tre mostrano in tempo reale il conteggio delle parole e una stima del tempo di lettura. Tutti hanno una toolbar che resta visibile mentre si scorre un articolo lungo (*sticky*), con i pulsanti che non perdono la selezione del testo (`onMouseDown` con `preventDefault`). E tutti devono risolvere lo stesso problema sottile: quando il contenuto cambia «da fuori» (perché si apre un articolo esistente), aggiornare l'editor senza far saltare il cursore né marcare il form come modificato. Tiptap lo fa con `setContent(value, { emitUpdate: false })`, e il commento nel codice di SPW spiega perché:

```
// SPW RichTextEditor.tsx:157-165 – la gotcha di Tiptap v3
// In v3 setContent emette onUpdate di default, marcando il form "dirty"
// e creando bozze fantasma in localStorage. emitUpdate:false lo evita.
editor.commands.setContent(value, { emitUpdate: false });
```

DIS ottiene lo stesso risultato a mano, aggiornando l'`innerHTML` solo quando l'editor non è in focus.

Dove i siti divergono davvero è nella rete di sicurezza contro la perdita del lavoro. SPW salva una bozza in `localStorage` ogni paio di secondi e, alla riapertura, offre un banner per ripristinare la sessione interrotta; lo stato «modificato» alimenta anche un blocco alla navigazione. SR si ferma al `beforeunload`, l'avviso del browser quando chiudi la scheda con modifiche non salvate. DIS non ha né l'uno né l'altro.

CONSIGLIO

Non perdere il lavoro dell'editor senza un backend Nel thin stack non c'è un endpoint di autosave: la bozza vive nel browser. La soluzione di SPW (`localStorage` più banner di ripristino) è la più robusta a costo zero di backend, e trasforma un crash del browser in una scelta esplicita («ripristina» o «ignora») invece che in lavoro perso. Il limite resta: se cambi dispositivo o pulisci la cache, la bozza non salvata sparisce comunque. Ma tra il niente di DIS e il solo avviso di SR, una bozza locale è la differenza tra perdere mezz'ora di scrittura e ritrovarla.

IMPORTANTE Il Canone

- Editor Tiptap, HTML come fonte di verità salvato grezzo.
- La difesa contro l'XSS è la sanitizzazione **al render** (`DOMPurify`), choke-point unico; la pulizia all'incolla è cosmetica, non sicurezza.
- Guardie all'inserimento dei link (`isSafeLinkUrl`): niente `javascript:` né URL non validati.
- I quattro emettitori del `content` (`render`, `prerender` SEO, `feed`, `newsletter`) condividono la stessa sanitizzazione server-side: se uno la dimentica, il buco XSS si riapre.

Prossimo Capitolo: Content Lifecycle. Il ciclo di vita dei contenuti, dalla bozza alla pubblicazione programmata, e le tre regole di visibilità che decidono cosa il pubblico vede davvero.

PARTE IV

Il Flusso Operativo

*Come i contenuti vivono. Dal ciclo di vita alla distribuzione,
passando per sicurezza e SEO.*

CAPITOLO 9: Content Lifecycle

Questo capitolo trasforma un database in un sistema editoriale: come un contenuto nasce bozza, viene programmato, diventa pubblico, e come l'API che lo serve decide forma, conteggio e visibilità. È anche il capitolo in cui si chiude un filo aperto al Capitolo 6: il contratto di payload che il client legge in modo difensivo nasce qui, lato server.

1. Stati Dinamici contro Stati Persistenti

Il database salva uno stato fisso (*status*), ma l'applicazione ne calcola uno dinamico, che dipende dal tempo. La matrice pulita è questa:

status (DB)	published_at	Stato reale (UI)	Descrizione
draft	qualsiasi	bozza	mai visibile al pubblico
published	nel futuro	programmato	visibile solo agli admin, in attesa
published	nel passato	pubblicato	visibile a tutti

Questa è la matrice di **SimonePizziWebSite**, ed è la più ordinata. Gli altri due siti portano le cicatrici delle loro migrazioni: **SitoRuntime** filtra la lista pubblica con `status = 'published' OR status IS NULL`, dove l'`IS NULL` è il residuo del fatto che la colonna `status` è stata aggiunta fuori dallo schema base; **DISINTELLIGENZA** interroga ancora `status = 'scheduled'`, uno stato che una migrazione ha dichiarato superato ma che il codice continua a cercare. La matrice ordinata è un punto d'arrivo, non lo stato naturale delle cose (Capitolo 15).

2. Programmazione senza Cron: le Tre Strategie del Presente

L'idea è elegante e condivisa: nessun job schedulato: un articolo con `published_at` nel futuro è «pubblicato ma non ancora visibile», e compare da solo quando la query di visibilità trova `published_at <= adesso`. Lato React, la dashboard calcola lo stesso stato al volo per dare un feedback immediato all'admin:

```
const isDraft      = item.status === 'draft';
const isScheduled = item.status === 'published' && new Date(item.published_a
↵ t) > new Date();
const isPublished = item.status === 'published' && new Date(item.published_a
↵ t) <= new Date();
```

Il punto delicato è cosa significhi «adesso». Non c'è un'unica risposta giusta: i tre siti lo calcolano in tre modi, e ognuno ha il suo modo di sbagliare.

- **SimonePizziWebSite** forza `date_default_timezone_set('Europe/Rome')` e confronta in PHP. Corretto, ma solo finché il fuso è forzato in *ogni* endpoint (Capitolo 5).

- **SitoRuntime** confronta stringhe `date('Y-m-d H:i:s')` con separatore spazio. La query è giusta, ma se il client invia una data in formato ISO con la `T`, il confronto fra stringhe salta: è l'incidente documentato in `debug_time.php`.
- **DISINTELLIGENZA** delega a `CURRENT_TIMESTAMP` di SQLite, che è in **UTC**, mentre `published_at` è salvato nel fuso del server. Un articolo compare o sparisce con uno scarto di una o due ore.

ATTENZIONE

Chi calcola il presente: PHP o il database? La conversione `datetime-local` (la `T` del browser) verso il formato del DB (lo spazio) non è «lo standard»: è il punto dolente di chi confronta date come stringhe, cioè **SitoRuntime**. Le altre due strategie evitano quel problema ma ne introducono altri (il fuso da forzare ovunque in SPW, lo scarto UTC in DIS). La regola pratica: scegli una sola fonte del presente e usala sempre. Se confronti in PHP, forza il fuso nel prelude condiviso; se confronti nel database, assicurati che `published_at` e `NOW/CURRENT_TIMESTAMP` vivano nello stesso fuso. Mescolarle è la ricetta del post che appare un'ora prima del previsto.

```
// la conversione T <-> spazio è di SitoRuntime, non una regola universale
value={published_at.replace(' ', 'T').slice(0, 16)} // DB -> UI
onChange={e => setPublishedAt(e.target.value.replace('T', ' ') + ':00')} /
↵ / UI -> DB
```

3. Il Contratto di Risposta: dove si chiude il Double Read

Al Capitolo 6 il client leggeva il payload in modo difensivo, perché la forma delle risposte non è uniforme. Il perché è qui, lato server: il contratto non è mai stato versionato, è stato **esteso quando serviva** (aggiungendo la paginazione, un wrapper `success`), e così la busta è diversa per endpoint e per sito.

- **SimonePizziWebSite**: *solo* la lista articoli ritorna `{ data, total, page, limit }`; tutto il resto (progetti, categorie, tag) è array nudo. Il client fa il «Double Read» non perché un endpoint cambi forma, ma perché mescola le due famiglie nei loader.
- **SitoRuntime**: tre buste diverse, `{ success, data, meta }` per le news, `{ success, articles, total }` per l'admin, array nudo per speaker e podcast. Un mosaico per-endpoint.
- **DISINTELLIGENZA**: sempre array o oggetto nudo, la «busta zero».

Anche il conteggio per la paginazione vive in posti diversi. SPW ritorna il `total` grezzo e lascia il calcolo di `hasMore` al client (con un `COUNT(*)` separato sulle stesse condizioni e, dettaglio MySQL obbligatorio, i `LIMIT/OFFSET` bindati con `PARAM_INT`, altrimenti PDO li tratta come stringhe e la query fallisce). **SitoRuntime** pre-calcola `total_pages` lato server e lo mette in `meta`, così il load-more del client è banale. **DISINTELLIGENZA** non dà nessun metadato: il client chiede la pagina successiva alla cieca, finché torna vuota.

NOTA

Estendere un contratto invece di versionarlo: cosa costa Aggiungere `{ data, total }` a un endpoint che prima tornava un array nudo è la cosa più rapida del mondo, e non rompe subito niente. Il costo arriva dopo, e lo paga il client: deve

indovinare quale forma riceverà, e quando sbaglia (un array senza `total` letto come se avesse il conteggio) nasce il `hasMore` errato del Capitolo 6. Versionare un'API costa disciplina; estenderla in-place costa fragilità diffusa. Per un CMS piccolo la seconda è una scelta legittima, ma va fatta sapendo che il prezzo si sposta sul frontend.

4. La Cache dei Contenuti senza Redis

SitoRuntime è l'unico dei tre ad aggiungere una cache di lettura, e lo fa nel modo più thin-stack possibile: file JSON su disco, senza Redis, senza Memcached. La lista delle news viene servita da un file in `.cache/`, rigenerato solo quando è più vecchio del TTL.

```
// SR news.php - cache su file con TTL e header diagnostico
$cacheFile = __DIR__ . "/.cache/news_p{$page}_l{$limit}.json";
if (file_exists($cacheFile) && (time() - filemtime($cacheFile) < 300)) { //
↳ TTL 300s
    header('X-Cache: HIT');
    echo file_get_contents($cacheFile);
    exit;
}
// ...altrimenti interroga il DB, salva il JSON, e:
header('X-Cache: MISS');
```

Due dettagli la rendono sana. L'invalidazione è esplicita: a ogni `save` o `delete` la cartella `.cache/` viene ripulita, così il pubblico non vede mai dati vecchi dopo una modifica. E l'header `X-Cache: HIT/MISS` permette di verificare dall'esterno se la cache sta lavorando. È lo stesso meccanismo che, applicato al SEO, diventa lo scudo anti-bot del Capitolo 11; qui serve solo a non interrogare il database per ogni visita alla stessa pagina di lista.

5. Tassonomie: Categorie e Tag (e quando non servono)

Qui il Modello mostra di nuovo la sua scala, e va corretta un'idea diffusa: il multi-tagging relazionale non è lo standard di tutti, è la scelta di **un** sito quando serve davvero.

SimonePizziWebSite è il blog tassonomico completo: categorie gerarchiche con `parent_id` (una categoria-contenitore filtra anche le sottocategorie, via `IN`), e tag in relazione multi-a-molti su `article_tags`. Ma non è un «passaggio esclusivo» al modello relazionale: la funzione che salva i tag scrive **in parallelo** anche il vecchio campo CSV `articles.tags`, tenuto come cache di retrocompatibilità. È la convivenza del modello vecchio e nuovo durante una migrazione mai chiusa, non un taglio netto.

```
// SPW - doppia scrittura: relazione M:N nuova + campo CSV legacy in parallelo
↳ lo
syncArticleTags($pdo, $articleId, $tagIds); // tabella artic
↳ le_tags (nuovo)
$pdo->prepare("UPDATE articles SET tags = ? WHERE id = ?") // campo CSV (le
↳ gacy, cache retro-compat)
->execute([implode(',', $tagNames), $articleId]);
```

SitoRuntime e **DISINTELLIGENZA** non hanno affatto tag relazionali: la `category` è una stringa libera (in DIS con default `'generale'`), e i «tag», dove esistono, sono un campo `TEXT` o `JSON`. Per un sito che non è un archivio tassonomico, una tabella categorie è peso che non serve: una stringa basta.

CONSIGLIO

Quando NON ti serve una tabella categorie La gerarchia `parent_id` con tag M:N è giusta per un blog con centinaia di articoli da filtrare per argomento. Per una radio con poche categorie fisse, o un festival, una stringa libera fa lo stesso lavoro senza join, senza tabella di relazione, senza editor di tassonomia. Aggiungere la tassonomia relazionale «perché è più corretto» è il genere di complessità che il Modello invita a rimandare finché il contenuto non la chiede.

6. Pubblico contro Admin: il Bypass e il 404 Deliberato

La stessa risorsa serve due pubblici, e la differenza è una condizione di visibilità aggiunta solo a chi non è loggato. SimonePizziWebSite lo realizza con un `AND` condizionale nello stesso endpoint; SitoRuntime con due query in due file diversi (lettura in `news.php`, gestione in `admin.php`); DISINTELLIGENZA con un `if (! $isAdmin)` che aggiunge il `WHERE`. Stessa idea, tre strutture.

Sul singolo articolo non pubblicato, però, c'è un dettaglio di sicurezza che vale la regola: si risponde **404, non 403**.

```
// SPW articles.php?slug=... - il 404 deliberato: non confermare l'esistenza
↳ di una bozza
$article = $stmt->fetch();
if ($article) {
    $is_admin    = isset($_SESSION['user_id']);
    $is_published = $article['status'] === 'published' &&
        (empty($article['published_at']) || strtotime($article['
↳ published_at']) <= $ita_now_time);
    if (!$is_admin && !$is_published) {
        http_response_code(404);                // non 403: un 403 co
↳ nfermerebbe che la bozza esiste
        echo json_encode(['error' => 'Articolo non trovato']);
        exit;
    }
    echo json_encode($article);                // l'admin vede tutto
}
```

Per la lista della dashboard, che deve mostrare anche le bozze, serve un controllo doppio: la sessione e un parametro esplicito.

```
// il parametro ?admin=true da solo sarebbe bypassabile: va sempre legato al
↳ la sessione
$is_admin_dashboard = isset($_SESSION['user_id']) && ($_GET['admin'] ?? '')
↳ === 'true';
if (!$is_admin_dashboard) {
    $conditions[] = "status = 'published'";
    $conditions[] = "(published_at IS NULL OR published_at = '' OR published
↳ _at <= ?)";
    $params[]     = $ita_now_str;
}
```

Il fetch per `id` (non per `slug`) serve solo all'editor in dashboard, e richiede `Auth::check()` obbligatorio, perché deve caricare nel form anche le bozze mai pubblicate. È il terzo modo di accedere allo stesso contenuto, ciascuno con il suo livello di gate.

7. Workflow Editoriale e Integrità

Tre accortezze tengono insieme l'esperienza di redazione:

- **Auto-slug solo alla creazione:** lo slug si genera quando l'articolo nasce, non a ogni modifica del titolo, per non rompere i link già pubblicati e indicizzati.
- **Editor che si ripulisce tra un contenuto e l'altro:** passando dalla modifica di un articolo a un altro, il componente editor va smontato e rimontato (`key={item.id}`), così i buffer interni non trascinano testo da un contenuto al successivo.
- **Anteprima della copertina:** il form mostra la miniatura dell'immagine selezionata, con rimozione immediata prima del salvataggio.

Sul lato dashboard, la tabella di gestione cura le proporzioni (il titolo non oltre il 45% della griglia, una colonna categoria con badge colorati per la scansione rapida, date e stato dinamico ben visibili, le azioni condensate in icone a fine riga): dettagli di SimonePizziWebSite, utili come riferimento di UX più che come prescrizione.

IMPORTANTE **Il Canone**

- Stati bozza/programmato/pubblicato; il confronto `published_at <= NOW` va fatto nello stesso formato e fuso (o delegato a `NOW()`).
- Per i contenuti non pubblici rispondi 404, non 403: non confermare l'esistenza di una bozza.
- Estendi un contratto (es. `{data, total}`) invece di versionarlo, ma mantienilo coerente tra gli endpoint.
- Cache JSON con TTL per i listing pesanti, invalidata su `save/delete`.

**

Prossimo Capitolo: Security & Auth. Gestione delle sessioni, CSRF, ruoli e protezione anti-abuso.

CAPITOLO 10: Security & Auth

La sicurezza è l'unica lente di questo libro in cui i tre siti non scalano in parallelo con l'ingegnerizzazione del backend. Negli altri capitoli vale una regola intuitiva: più un sito è cresciuto, più la sua infrastruttura è ricca. Qui no. Lo stesso scheletro thin-stack (PHP nativo, sessione su cookie, regole Apache nei `.htaccess`) regge tre gradini di difesa decrescenti, ma il gradino più alto non è quello del sito col backend più sofisticato.

SimonePizziWebSite (SPW) ha il perimetro più maturo: cookie `Secure`, anti session-fixation, invalidazione globale delle sessioni, recupero password completo, IP anti-spoofing. SitoRuntime (SR), il sito più ingegnerizzato del trio, quello delle cicatrici di scalabilità, è solo a metà strada: ha ruoli e token CSRF, ma il cookie viaggia senza `Secure`, non rigenera la sessione al login e il suo rate-limit si aggira con un header. DISINTELLIGENZA (DIS) è l'autenticazione di grado zero: niente CSRF, niente rate-limit, cookie ai valori di default di PHP. Eppure è l'unico dei tre a portare due idee di sicurezza proprie e di valore, la difesa anti-frode di un'azione pubblica (il voto del festival) e il backup automatico prima di un'operazione distruttiva.

È la dimostrazione più netta della tesi che attraversa tutto il libro: più ingegnerizzato non significa più sicuro, e più sicuro non significa più completo su ogni singolo punto. Il modo giusto di leggere ogni difesa è come una scala di sottrazione: cosa resta, e cosa si rompe, quando togli un flag dal cookie, un token da una richiesta, un contatore da un endpoint di login.

NOTA

Una nota di metodo. Tutti gli stralci di codice di questo capitolo vengono dallo stato reale dei tre siti, citati come `file:linea`. Quando il capitolo dice «il Modello raccomanda» sta prescrivendo; quando dice «SPW fa» o «SR fa» o «DIS fa» sta fotografando il codice in produzione. Le due cose non sempre coincidono, ed è proprio in quello scarto che si impara.

*

**

1. Il perimetro comune: sicurezza fatta a mano

Prima delle divergenze conviene fissare ciò che i tre siti condividono. Sotto le tre implementazioni c'è lo stesso oggetto, con cinque tratti ricorrenti.

1) Nessuna libreria di autenticazione. Niente Passport, niente pacchetto JWT, nessun framework di sessione. Tutto poggia su primitive PHP native: `session_*`, `password_hash/password_verify`, `random_bytes`, `hash_equals`, e sulle regole Apache nei `.htaccess`. L'auth è fatta

a mano, ed è la scelta fondante che rende ogni differenza tra i siti una scelta visibile, non una configurazione sepolta dentro un vendor.

2) Sessione su cookie più password_verify: lo zoccolo identico. Tutti e tre aprono una sessione PHP nativa, cercano l'utente per username, verificano la password contro un hash PASSWORD_DEFAULT, e popolano \$_SESSION. Il sistema non conosce mai la password in chiaro:

```
// Lo schema comune ai tre siti: nessun segreto in chiaro
$hash = password_hash($password, PASSWORD_DEFAULT); // alla creazione
// ...
if (password_verify($input, $user['password_hash'])) { /* login ok */ }
```

3) Il gate è una manciata di righe in cima all'endpoint. Non c'è middleware né router: la protezione di un endpoint sono poche righe all'inizio del ramo mutativo, che pretendono una sessione valida prima di toccare i dati. È la versione thin-stack del middleware. Ma quanto fa quella manciata di righe (solo sessione? più CSRF? più ruolo? più invalidazione?) è il primo grande asse di divergenza, e lo vediamo al §2.

4) Difesa «JSON-first» anche sugli errori di sicurezza. Un accesso negato non produce una pagina di errore di Apache: si imposta il codice HTTP semanticamente corretto (401/403/429) e si risponde con un oggetto {status|success, message|error}. Il frontend reagisce per codice.

5) Hardening a livello server via .htaccess. Tutti hanno almeno un .htaccess che imposta header di sicurezza e nega l'accesso a file sensibili. È il secondo strato, fuori dall'applicazione, e anche qui la copertura va dal completo (HTTPS forzato, HSTS, CSP, PHP-off) al minimo (deny di due estensioni).

A questi si aggiunge un tratto negativo condiviso: nessuno dei tre tiene un audit-log degli accessi, e in due casi su tre il contatore brute-force del login non vive nemmeno nel database. Il perimetro è sottile per costruzione. La domanda del capitolo è quanto sottile si può andare prima che qualcosa si rompa.

2. Il gate: middleware unico, gate componibile, gate inline

Proteggere un endpoint significa decidere, in cima al codice, se chi chiama ha diritto di proseguire. I tre siti risolvono lo stesso problema con tre architetture diverse, ed è il primo punto in cui la scala si fa visibile.

2.1 SPW: il gate unico Auth::check()

SPW concentra tutto in una classe inclusa da ogni endpoint protetto. Una sola riga, Auth::check(), fa tre cose in sequenza: esige una sessione valida, verifica l'anti-CSRF sui metodi mutativi, controlla l'invalidazione via session_version.

```
// public/api/articles.php:238-239 — il consumatore tipico
elseif ($method === 'POST') {
    Auth::check(); // sessione + CSRF + session_version, tutto qui
    // ... da qui in poi si può scrivere
```

Il vantaggio è che la copertura è automatica: ogni nuovo endpoint che include `auth_helper.php` e chiama `Auth::check()` eredita tutte le difese insieme. Non c'è modo di ricordarsi la sessione e dimenticare il CSRF.

2.2 SR: il gate componibile a funzioni

SR scompone la stessa protezione in mattoni separati, autorizzazione da una parte e anti-CSRF dall'altra, che ogni endpoint compone a mano nell'ordine giusto:

```
// public/api/auth_utils.php - i mattoni
function isLoggedIn() { return isset($_SESSION['user_id']); }
function isAdmin()    { return isLoggedIn() && ($_SESSION['role'] ?? '') ===
    ↵ 'admin'; }

// public/api/upload.php:8-13 - il consumatore li compone
if (!isLoggedIn()) { http_response_code(401); echo json_encode([]); exit;
    ↵ }
// ...
validateCsrf();
```

È più flessibile, e il gate a ruolo (`isAdmin()`) è gratis: arriva quel `admin/editor` che SPW non ha. Ma è anche più facile da dimenticare, perché la protezione dipende dalla disciplina del singolo endpoint.

2.3 DIS: il gate inline grezzo

DIS porta lo stesso schema all'estremo. Niente file di funzioni-mattone: il gate è ricostruito a mano in ogni ramo come un `isset()` nudo.

```
// public/api/reset_votes.php:11 - il gate per intero, inline
if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    http_response_code(401); die(...);
}
```

ATTENZIONE

Middleware o disciplina? Il gate che si dimentica Il gate componibile e quello inline hanno un rischio strutturale che il gate unico non ha: la sicurezza dipende dall'ordine dei rami e dalla memoria di chi scrive. In DIS i rami `participants.php?update_status` e `update_round` sono protetti dal solo `isset($_SESSION['user_id'])`, non da `isAdmin()`. Risultato: un editor, non un amministratore, può approvare o respingere partecipanti, mandare le email e spostare i round del festival. La stessa crepa si vede in SR, dove `list_users` e `create_user` stanno prima del blocco 401 e si autoprotettono solo con il check di ruolo locale. Funziona finché tutti i rami sono scritti con disciplina. Ma un gate unico (`Auth::check()`) non può essere dimenticato su un endpoint: o lo includi, o l'endpoint non parte. È questa la differenza vera tra middleware e convenzione. Le conseguenze sul festival si vedono al CAP 18.

*
**

3. CSRF: tre gradini di difesa

Il Cross-Site Request Forgery è il problema di impedire che un sito terzo invii richieste mutative sfruttando il cookie di sessione che il browser dell'utente allega in automatico. È la difesa portante di questo capitolo, e i tre siti la risolvono su tre livelli distinti.

3.1 SPW: controllo origin/Referer (server-side, zero handshake)

SPW non usa token: dentro `Auth::check()`, sui metodi non-safe, confronta l'host di `Origin/Referer` con quello di `SITE_URL`.

```
// public/api/auth_helper.php:21-37 (estratto)
$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
if (!in_array($method, ['GET', 'HEAD', 'OPTIONS'], true)) {
    $source = $_SERVER['HTTP_ORIGIN'] ?? $_SERVER['HTTP_REFERER'] ?? '';
    if ($source !== '') {
        $sourceHost = parse_url($source, PHP_URL_HOST);
        $allowedHost = parse_url(SITE_URL, PHP_URL_HOST);
        $isLocalDev = in_array($sourceHost, ['localhost', '127.0.0.1'], true);
        if ($sourceHost !== $allowedHost && !$isLocalDev) {
            http_response_code(403);
            echo json_encode(['status' => 'error', 'message' => 'Origine del
la richiesta non valida']);
            exit;
        }
    }
}
```

Non richiede alcun handshake col client, perché il browser invia `Origin/Referer` da solo. E vivendo dentro `Auth::check()`, copre in automatico ogni endpoint che include la guardia.

3.2 SR: token sincronizzato X-CSRF-Token

SR adotta il pattern da manuale: un token casuale per sessione, restituito al client al login, rispedito dal client in un header e validato con `hash_equals` (confronto timing-safe).

```
// public/api/auth_utils.php:20-35
function generateCsrfToken(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
function validateCsrf(): void {
    $incoming = $_SERVER['HTTP_X_CSRF_TOKEN'] ?? '';
    $stored = $_SESSION['csrf_token'] ?? '';
    if (!$stored || !hash_equals($stored, $incoming)) {
        http_response_code(403);
        echo json_encode(['success' => false, 'error' => 'Token di sicurezza
non valido. Ricarica la pagina.']);
        exit;
    }
}
```

3.3 DIS: niente

In DIS le mutazioni sono protette dal solo cookie di sessione. Un grep su tutta `public/api/` non trova nessun token CSRF, nessun check `Origin/Referer`: la difesa è assente.

ATTENZIONE

CSRF nel thin stack: Origin/Referer, token sincronizzato, o niente C'è un sottotesto controintuitivo in questi tre gradini. La soluzione più «da manuale», il token sincronizzato di SR, è anche la più facile da indebolire, perché dipende dalla disciplina per-ramo: basta un endpoint mutativo che dimentichi `validateCsrf()` e il buco è aperto. La soluzione più semplice, il check `Origin/Referer` di SPW, copre di

più proprio perché è centralizzata dentro il gate unico. Più codice non vuol dire più copertura. È la tesi di fondo del libro applicata al CSRF.

**

4. Il cookie di sessione e l'anti session-fixation

NOTA

Un errore comune sui cookie di sessione. Verrebbe da prescrivere un cookie con `cookie_secure = 1` e `cookie_samesite = 'Lax'`. Guardando il codice reale, due cose vanno corrette: il `samesite` giusto, e quello che i siti usano davvero, è `strict`, non `Lax` (sia SPW sia SR); e solo SPW imposta davvero tutti e tre i flag.

I flag vanno impostati prima di `session_start()`, altrimenti non hanno effetto sul cookie corrente. È qui che la scala si vede meglio.

SPW imposta i tre flag in un file condiviso:

```
// public/api/auth_helper.php:7-11
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.cookie_samesite', 'Strict');
session_start();
```

SR omette `Secure`:

```
// public/api/auth_utils.php:4-9
ini_set('session.cookie_httponly', '1');
ini_set('session.cookie_samesite', 'Strict');
session_start(); // NB: cookie_secure NON impostato
```

DIS non imposta nessun flag: il comportamento del cookie dipende interamente dal `php.ini` dell'hosting. Una dipendenza implicita e silenziosa.

A questo si aggiunge l'anti session-fixation, cioè rigenerare l'ID di sessione subito dopo il login per invalidare un ID eventualmente piantato prima dall'attaccante. Solo SPW lo fa:

```
// public/api/auth.php:186-188 (estratto del ramo di successo)
if ($user && password_verify($password, $user['password_hash'])) {
    session_regenerate_id(true); // anti session-fixation: una riga, spe
    ↪ sso assente
    $_SESSION['user_id'] = $user['id'];
    // ...
}
```

SR e DIS non rigenerano: un ID di sessione fissato prima del login resta valido dopo.

ATTENZIONE

I flag del cookie, e perché HSTS non è il redirect HTTPS Togliere `Secure` non è un dettaglio cosmetico. Senza `Secure`, il cookie di sessione può viaggiare anche su HTTP in chiaro. Si potrebbe pensare che HSTS protegga comunque, ma HSTS protegge solo dopo la prima visita HTTPS riuscita. SR applica HSTS ma non forza il redirect 301 a HTTPS: c'è una finestra reale di esposizione su HTTP al primo accesso. SPW

chiude la finestra da entrambi i lati, con `cookie_secure=1` e `CON RewriteCond %{HTTPS} !=on` → 301. La lezione: Secure sul cookie e redirect HTTPS forzato sono due difese distinte; HSTS non sostituisce nessuna delle due. Curiosità storica (SPW): fino alla v1.18 i flag stavano solo in `auth.php`, così un cookie emesso da un altro endpoint nasceva debole. La v1.19.0 ha spostato gli `ini_set` nel file condiviso `auth_helper.php`. La config di sicurezza della sessione va nel file incluso da tutti, non nell'endpoint di login.

5. check_auth e lo stato di sessione

Il frontend non deve mai conservare la password o lo stato di login in `localStorage`: interroga il server per sapere se la sessione è ancora valida.

```
// schema di check_auth (SPW/DIS)
if (isset($_SESSION['user_id'])) {
    echo json_encode([
        'status' => 'success',
        'user'   => ['username' => $_SESSION['username'], 'role' => $_SESSION
↵ N['role']]
    ]);
}
```

NOTA

Lo username non è garantito in sessione. Verrebbe da leggere `$_SESSION['username']` come se fosse sempre presente. In SR non lo è: il login salva `user_id` e `role` ma non `username` (`admin.php:123-124`). La conseguenza è concreta: `check_auth` restituisce `username: null`, e il salvataggio articolo ripiega su `author = $_SESSION['username'] ?? 'Admin'`. In SPW e DIS lo `username` è in sessione e l'esempio regge. Non è un buco di sicurezza, ma un'incoerenza di stato reale: lo stesso campo non è garantito su tutti e tre i siti.

6. Le password e la difesa brute-force

L'hashing è l'unico punto in cui i tre siti sono identici e tutti corretti: `password_hash($pass, PASSWORD_DEFAULT)` alla creazione, `password_verify` alla verifica. Il sistema non conosce mai la password in chiaro, e l'algoritmo può evolvere (da `bcrypt` ad `argon2`) senza che si tocchi una riga di codice.

NOTA

La difesa brute-force non è un `sleep(1)`. Verrebbe da ridurre la difesa brute-force a un `sleep(1)`, ma è una soluzione incompleta e tarata sul solo SR. Il `sleep(1)` è appena un accorgimento; la difesa vera è il `lockout`, e soprattutto conta da quale IP lo si misura.

La domanda non è «come rallento i tentativi» ma dove vive il contatore. Tre risposte. SPW lo tiene in una tabella DB `login_attempts`: dopo 5 tentativi falliti da un IP in 15 minuti scatta il 429, e al login riuscito il contatore si azzerava.

```
// public/api/auth.php:177-211 (estratto)
if ($attempts >= 5) {
    http_response_code(429);
    echo json_encode(['status' => 'error', 'message' => 'Too many failed log
↵ in attempts. Try again in 15 minutes.']);
    exit;
}
// ... su fallimento:
$pdo->prepare("INSERT INTO login_attempts (ip_address) VALUES (?)")->execute
↵ ([$ip_address]);
```

SR lo tiene su file: un JSON per IP in `.cache/ratelimit/<md5(ip)>.json`, finestra 900 secondi, soglia 5, più `sleep(1)` sul fallimento. Niente tabella DB, coerente con lo schema MySQL che non contiene `login_attempts`. DIS non lo tiene da nessuna parte: il login si può martellare quanto si vuole.

CONSIGLIO

Il contatore brute-force: file, DB, o assenza Non c'è una sede giusta in assoluto. La tabella DB di SPW è transazionale e si presta al riuso (lo stesso `login_attempts` serve anche il recovery). Il file di SR evita di gravare sul database ma vive fuori dallo schema, quindi non compare in un dump né in una migrazione: è invisibile finché non lo cerchi nel filesystem. L'assenza di DIS è coerente con un sito-festival dove l'admin è uno solo e l'attrito su un login interno è basso, ma resta un'assenza, non una scelta documentata.

Il punto più sottile, però, è da quale variabile leggi l'IP. È il box-ancora di questo capitolo.

ATTENZIONE

Fidarsi dell'IP: il rate-limit che non limita (*box-ancora, richiamato al CAP 18 e al CAP 20*) Un lockout «5 tentativi per IP» vale esattamente quanto la tua capacità di sapere qual è l'IP. E qui i tre siti divergono in modo istruttivo.

SR si fida dell'header sbagliato. Prende l'IP da `X-Forwarded-For`, per primo e senza validazione:

```
// public/api/admin.php:106-107
$ip = $_SERVER['HTTP_X_FORWARDED_FOR'] ?? $_SERVER['REMOTE_ADDR'] ?? 'unknown
↵ n';
$ip = trim(explode(',', $ip)[0]); // ← l'attaccante controlla questo head
↵ e
```

Quell'header è scritto dal client. Un attaccante lo varia a ogni richiesta e il contatore non si incrementa mai: il lockout 5/15min si aggira.

SPW si fida dell'IP TCP, e dell'header solo dietro proxy interno. La funzione `getClientIp()` usa `REMOTE_ADDR` se è già pubblico, e accetta `X-Forwarded-For` (validato `NO_PRIV_RANGE|NO_RES_RANGE`) solo quando `REMOTE_ADDR` è privato, cioè solo dietro un proxy fidato. Niente spoofing.

DIS usa `REMOTE_ADDR` grezzo, e qui è un pregio. Per la barriera anti-doppio-voto (§12), `REMOTE_ADDR` non è falsificabile a livello TCP, quindi la barriera regge. Il rovescio è il NAT: dietro un proxy o una rete condivisa, molti utenti collassano sullo stesso IP.

La lezione non è «usa sempre `REMOTE_ADDR`». È che l'IP giusto dipende dal modello d'abuso. Un login da forzare (vuoi che l'attaccante non possa cambiare la propria identità, quindi diffidi di `X-Forwarded-For`) è il problema opposto di un voto pubblico da non duplicare; eppure la conclusione è la stessa: l'header controllato dal client non è affidabile. Lo stesso `REMOTE_ADDR` grezzo è un buco in un contesto e una difesa nell'altro.

7. session_version: invalidare le sessioni a costo zero

C'è un problema classico dell'auth su sessione: come disconnetti tutte le sessioni di un utente, per esempio dopo un reset password, se non tieni uno store server-side delle sessioni attive? Solo SPW risolve, con un trucco a costo zero: un intero `session_version` in `users`, copiato in `$_SESSION` al login e confrontato a ogni richiesta protetta.

```
// public/api/auth_helper.php:51-57 - dentro Auth::check()
try {
    $stmt = $pdo->prepare("SELECT session_version FROM users WHERE id = ?");
    $stmt->execute([$$_SESSION['user_id']]);
    $row = $stmt->fetch();
    if (!$row || (int)$row['session_version'] !== (int)$_SESSION['session_v
↵ ersion'] ?? -1) {
        session_destroy();
        http_response_code(401);
        echo json_encode(['status' => 'error', 'message' => 'Sessione scadut
↵ a. Effettua nuovamente il login.']);
        exit;
    }
} catch (PDOException $e) {
    error_log('session_version check failed: ' . $e->getMessage());
    http_response_code(401);    // ← fail-closed: in caso di errore DB, NEGA
    echo json_encode(['status' => 'error', 'message' => 'Sessione non verifi
↵ cabile. Riprova.']);
    exit;
}
```

Basta incrementare `session_version` di un numero (lo fa il reset password, §8) e tutte le sessioni col numero vecchio diventano invalide al primo controllo. Logout-everywhere senza alcuno store di sessioni.

CONSIGLIO

Fail-closed, non fail-open Il dettaglio che distingue una difesa fatta bene è il ramo `catch`. Se il controllo di `session_version` solleva una `PDOException` (DB irraggiungibile), SPW nega l'accesso (401), non lo concede. La regola: quando una verifica di sicurezza non può essere completata, l'esito di default deve essere «negato». SR e DIS non hanno questo meccanismo, e in SR un cambio password non disconnette le altre sessioni; in DIS nemmeno.

8. Recovery e reset password fatti bene

È un'intera sezione che il capitolo prima non aveva, perché solo SPW implementa il recupero password self-service. SR e DIS hanno solo `change_password` autenticato, dove devi già essere dentro: password dimenticata uguale nessun rientro.

Il flusso di SPW ha quattro accortezze che vale la pena guardare una per una.

La prima è il token monouso casuale, con scadenza.

```
// public/api/auth.php:98-106
$token = bin2hex(random_bytes(32)); // 32 byte casuali
$expires = date('Y-m-d H:i:s', strtotime('+1 hour'));
$pdo->prepare("DELETE FROM password_resets WHERE user_id = ?")->execute([$user_id]); // invalida i precedenti
$pdo->prepare("INSERT INTO password_resets (user_id, token, expires_at) VALUES (?, ?, ?)")->execute([$user_id, $token, $expires]);
```

La seconda è il link costruito da URL canonico, non da `HTTP_HOST`. È la difesa contro il *password-reset poisoning*: se il link nell'email si costruisse da `HTTP_HOST` (controllabile con un header `Host` falsificato), l'attaccante potrebbe far recapitare alla vittima un link che punta al suo dominio e intercettare il token.

```
// public/api/auth.php:227-230
$link = SITE_URL . "/admin/reset-password/{$token}"; // SITE_URL hardcoded
// mai HTTP_HOST
```

La terza è l'essere enumeration-safe. La richiesta di recupero risponde sempre con lo stesso messaggio generico («se l'account esiste, riceverai un'email»), che l'utente esista o no, così non si può usare il form per scoprire quali email sono registrate. Lo stesso contatore `login_attempts` viene riusato qui con namespacing della chiave ('rec:' + sha256(IP)), per limitare anche gli abusi del recovery.

La quarta è che il reset invalida le sessioni. Applicando la nuova password, `session_version` viene incrementato, e per quanto visto al §7 tutte le sessioni aperte cadono:

```
// public/api/auth.php:127-149 (estratto)
if (strlen($newPassword) < 12) { /* 400: "almeno 12 caratteri" */ }
// ... verifica token non scaduto ...
$hash = password_hash($newPassword, PASSWORD_DEFAULT);
$pdo->prepare("UPDATE users SET password_hash = ?, session_version = session_version + 1 WHERE id = ?")->execute([$hash, $reset['user_id']]);
$pdo->prepare("DELETE FROM password_resets WHERE token = ?")->execute([$token]); // token monouso
```

NOTA

Lo schema che non c'è. La tabella `password_resets` è usata ovunque in `auth.php`, ma nessun file la crea: lo script di migrazione che la generò è stato cancellato dopo l'esecuzione in produzione. Lo stesso vale per la colonna `session_version`, aggiunta a caldo con un `ALTER TABLE` idempotente dentro `auth.php`. È un debito di tracciabilità: lo schema vero non è ricostruibile da un singolo file. Il tema delle migrazioni usa-e-getta torna al CAP 15 (Database Evolution).

9. Credenziali di default: random, hardcoded, omessa

Il primo amministratore è un problema di sicurezza spesso sottovalutato. I tre siti lo risolvono in tre modi, e qui si chiude un punto anticipato al CAP 5 (Backend Logic).

ATTENZIONE

Il seeding dell'admin: cosa NON fare SPW la genera random e la stampa una volta. La password del primo admin è casuale e mostrata una sola volta in fase di setup. È la scelta corretta. SR la hardcode a `runtime2026`. Il file di manutenzione `fix_users_table.php` ricrea l'utente admin con `password_hash('runtime2026', ...)` se la tabella è vuota, e quella password è committata nel repo. La mitigazione reale è il `.htaccess`, che nega l'accesso HTTP a `fix_users_table.php` (§10); ma non esiste alcun flusso che obblighi a cambiare la default, e se nessuno usa `change_password` l'account admin resta indovinabile. DIS la omette del tutto. Non c'è nessun seeding: `init_db.php` ha eliso la creazione dell'admin, e `users.php` può creare utenti solo se sei già admin. Risultato: niente default indovinabile (il vantaggio), ma il primo admin vive solo nel file `.sqlite` e non è ricostruibile dal repo (lo svantaggio, con il classico problema dell'uovo e la gallina al primo deploy pulito). Tre posizioni sulla stessa scala: la random è la più sicura, l'omessa è la più spartana ma non insicura, l'hardcoded è l'unica davvero pericolosa. E non perché la password esista, ma perché niente costringe a cambiarla.

10. Proteggere il database-a-file e gli script di manutenzione

Quando il database è un file (`.sqlite`), o quando nel docroot vivono script potenti (migrazioni, fix, reset), il `.htaccess` diventa parte del perimetro di sicurezza.

Il primo compito è negare l'accesso diretto al DB-a-file. DIS tiene il `.sqlite` in una cartella `.data/` generata a runtime e protetta:

```
# DIS: deny dei file di dati e backup
<FilesMatch "\.(sqlite|bak)$">
    Require all denied
</FilesMatch>
```

La regola di base è quella che il capitolo già insegnava (file fuori dalla root pubblica, oppure `Require all denied` più nomi non prevedibili). La novità è che DIS la accoppia a una cartella `.data/` creata a runtime dal bootstrap, di cui parliamo al CAP 5.

Il secondo compito è negare gli script di manutenzione, ed è qui che SR ha il `.htaccess` più interessante dei tre, perché blocca un'intera famiglia di file per prefisso, prima ancora che PHP li veda:

```
# public/.htaccess:80-81 (SR)
<FilesMatch "^(debug_|test_|emergency_|migrate_|fix_|init_|rebuild_|setup_|o
↳ ptimize_)">
    Order allow,deny
    Deny from all
</FilesMatch>
# blocca anche gli URL tipici degli scanner, prima di PHP
```

```
RewriteRule ^(wp-|wordpress|xmlrpc|\.env|\.git|cgi-bin|phpmyadmin) - [R=404,  
↳ L]
```

È questo deny a rendere non eseguibile via browser la `fix_users_table.php` con la password `runtime2026` del §9.

Il terzo compito riguarda gli upload. Se un attaccante riesce a caricare un `.php`, l'esecuzione va spenta a livello di cartella, come fa SPW:

```
# public/uploads/.htaccess (SPW)  
<IfModule mod_php.c>  
  php_flag engine off  
</IfModule>  
<FilesMatch "\.(php|phtml|php[0-9]|phps|cgi|pl|py|sh)$">  
  Require all denied  
</FilesMatch>
```

ATTENZIONE

Il deny che manca dove serve di più Attenzione a non leggere queste regole come una checklist uniforme: la copertura è disomogenea. SR ha il deny by-prefix più sofisticato dei tre, ma non ha un `uploads/.htaccess` che spenga PHP nella cartella di caricamento. DIS protegge il `.sqlite` e i `.bak`, ma i suoi script `update_db_*` non rientrano in nessun pattern di deny e restano raggiungibili. Ogni sito ha blindato la porta che aveva visto, lasciandone aperta un'altra. La protezione dell'upload pubblico, e la catena d'abuso che ne nasce in DIS, è il cuore del CAP 7.

*
**

11. Errori parlanti per l'utente, opachi per l'attaccante

Un errore PHP non gestito che finisce nell'output può rivelare percorsi, query, struttura del database (è il cosiddetto *path/information disclosure*). Lo standard del Modello: codici HTTP semanticamente corretti, messaggi generici al client, dettaglio tecnico solo nel log.

```
// SPW: il dettaglio va nel log, non al client  
{ catch (PDOException $e) {  
  error_log('auth: ' . $e->getMessage()); // solo qui  
  http_response_code(500);  
  echo json_encode(['status' => 'error', 'message' => 'Errore interno. Rip  
↳ rova.']);  
}
```

ATTENZIONE

Non rimandare l'eccezione al client DIS fa l'opposto: `auth.php`, `users.php`, `participants.php` rispediscono `$e->getMessage()` direttamente al client.

```
// DIS auth.php:48 (anti-pattern)  
{ catch (PDOException $e) {  
  echo json_encode(['status' => 'error', 'message' => $e->getMessage()]);  
↳ // leak dei dettagli DB  
}
```

È information disclosure gratuita: un attaccante legge nomi di tabelle e colonne dai messaggi d'errore. La regola «parlante per l'utente, opaco per l'attaccante» non costa nulla, basta non saltarla.

*
**

12. Le azioni distruttive e pubbliche

L'ultimo fronte è il più scivoloso: cosa succede quando un'azione è protetta da login ma anche distruttiva, oppure è pubblica (nessun login) ma deve comunque difendersi dall'abuso.

12.1 Il reset a un clic: perché «gated» non basta

In DIS gli endpoint `reset_system.php` e `reset_votes.php` richiedono il login admin ma non hanno CSRF. Una POST cross-site verso `reset_system.php`, innescata mentre l'admin è loggato in un'altra scheda, cancella tutti i partecipanti, i voti e gli audio. L'unica mitigazione è il `SameSite` di default del cookie (non impostato, come visto al §4), che dipende dalla versione di PHP e non copre ogni caso.

ATTENZIONE

Perché un'azione gated ha comunque bisogno del CSRF «È protetta da login» e «è protetta dal CSRF» sono garanzie diverse. Il login dice chi sei; il CSRF dice se sei stato tu a volerlo. Un'azione distruttiva ha bisogno di entrambe: senza CSRF, è la sessione legittima dell'admin a essere usata contro di lui. E una `confirm` JavaScript («sei sicuro?») non sostituisce il CSRF, perché gira sul client e l'attaccante la salta.

12.2 Il backup just-in-time: la difesa che DIS ha e SR no

E qui arriva la sorpresa che rompe la scala. Lo stesso DIS che non ha CSRF sul reset fa una cosa che il flagship degli incidenti, SR, non fa: copia il database prima di toccarlo.

```
// public/api/reset_votes.php:18-21 - backup prima del distruttivo
$dbPath = __DIR__ . '/.data/database.sqlite';
if (file_exists($dbPath)) {
    copy($dbPath, __DIR__ . '/.data/backup_votes_' . date('Ymd_His') . '.sql
↳ ite.bak');
}
$pdo->exec("DELETE FROM votes");
```

È esattamente la prevenzione che mancava a SR (al CAP 15 la chiamiamo «cura senza prevenzione»): il sito più debole sull'identità è l'unico a fare il backup giusto-in-tempo. Più sicuro non significa più completo su ogni punto.

12.3 Anti-frode di un'azione pubblica, e privacy

Il voto del festival è un'azione che non è autenticata: è il pubblico a votare. DIS la difende a strati, con una sola barriera reale, la `REMOTE_ADDR/24h` vista al box IP del §6. Il trattamento completo dell'anti-frode voto (master switch, cookie cosmetico, `vote_count` denormalizzato) vive al CAP 18; qui interessa solo un punto trasversale: come si conserva l'identità di chi compie un'azione pubblica.

CONSIGLIO

Votare in anonimato: hash invece di IP in chiaro (*ponte al CAP 20*) DIS salva `ip_address` e `user_agent` in chiaro nella tabella `votes`. Funziona per l'anti-doppio-voto, ma conserva dati personali senza necessità. SPW, per le reazioni anonime (CAP 20), ottiene la stessa garanzia anti-frode memorizzando solo `voter_hash = SHA256(IP + UA)`: il confronto regge, ma il dato personale non viene mai scritto. Due posture privacy opposte sulla stessa esigenza funzionale. Nota però che l'hash di SPW non è salato e usa input a bassa entropia (IP e UA): è anti-collisione, non anonimato crittografico forte, perché un IP candidato si può ancora verificare per forza bruta. Il confronto pieno tra le due filosofie (sanitizzazione write-time contro render-time, e identità anonima) è al CAP 20.

Sul versante anti-abuso, c'è un ultimo dettaglio: lo stesso `login_attempts` di SPW viene riusato come rate-limit a due strati per le reazioni (per-hash 20/min più solo-IP 30/min): il primo strato si aggira ruotando lo User-Agent, il secondo è l'argine vero. Anche qui il dettaglio è al CAP 20.

✱
✱ ✱

13. Il lato client e una lezione sui bot

L'area Admin in React è protetta da un `AdminLayout` che, nel suo `useEffect` principale, interroga `check_auth`: se il server risponde 401, il client distrugge lo stato locale e reindirizza al login, così non lampeggia contenuto sensibile. La sidebar e le rotte si generano in base al `role` ricevuto dal server (admin contro editor in SR e DIS).

NOTA

Il client non è la difesa. Il logout client-side e il check 401 sono esperienza utente, non sicurezza: nascondono ciò che non devi vedere, ma è il gate server-side (§2) a impedirti di toccarlo. La differenza tra «logout sul client» e l'invalidazione vera via `session_version` (§7) è esattamente questa. L'architettura completa del pannello admin (le tre dashboard, le tre architetture di guardia, il posizionamento dei backup) è al CAP 14.

C'è infine una lezione che viene da un incidente reale e resta pertinente qui, anche se il caso completo è migrato altrove. Nel febbraio 2026 Runtime Radio è stato sommerso da bot che simulavano i crawler dei social (Telegram, Facebook, X) per colpire l'entry-point SEO in PHP, che a ogni richiesta interrogava il database. Lo User-Agent dei bot è falsificabile.

ATTENZIONE

Lo User-Agent non è un gatekeeper Non si prendono mai decisioni di sicurezza in base allo User-Agent, perché si falsifica in un campo header. Lo si può usare per ottimizzare (servire una cache ai bot riconosciuti), mai come barriera d'accesso. Il racconto completo dell'attacco DDoS-da-bot e la soluzione (cache statica precom-

pilata, percorso bot separato da quello umano) sono al CAP 11, perché il vettore è proprio l'entry-point SEO. Qui resta solo la massima.

**

In sintesi

La sicurezza è la lente che smentisce l'intuizione «più grande uguale più robusto». SPW, che non è il sito più ingegnerizzato, ha il perimetro più maturo. SR, il più ricco, lascia tre buchi reali: cookie senza `Secure`, niente anti-fixation, rate-limit che si aggira. DIS, il grado zero, porta comunque due idee che agli altri mancano, l'anti-frode pubblica robusta e il backup pre-distruttivo. Letta come una scala di sottrazione, ogni difesa insegna due cose insieme: cosa fa, e cosa si rompe quando la toglie.

IMPORTANTE Il Canone

- Sessioni con cookie `HttpOnly` + `SameSite=Strict` + `Secure` su HTTPS; password con `password_hash()`.
- Token CSRF su tutte le mutazioni: un `confirm()` non è una difesa CSRF.
- Autorizzazione per **ruolo** (`isAdmin`), non solo per login (`isLoggedIn`).
- Lockout brute-force misurato su un IP affidabile, non su header controllati dal client (`X-Forwarded-For`).
- `session_version` per invalidare le sessioni al cambio password; nessuna credenziale di default nel codice.

**

Prossimo Capitolo: SEO Pre-rendering con PHP Entry-Point. Il motore SEO invisibile che trasforma una SPA in un sito indicizzabile, e il vettore dell'attacco DDoS-da-bot del febbraio 2026.

CAPITOLO 11: SEO Pre-rendering con PHP Entry-Point

Una Single Page Application ha un problema di nascita con i motori di ricerca. Il bot di Google, o il crawler di Telegram che genera l'anteprima di un link, riceve dal server un `index.html` quasi vuoto: un `<div id="root"></div>` e un bundle JavaScript. Il contenuto vero arriva solo dopo che quel JavaScript gira, e molti crawler non lo eseguono. Il risultato è una pagina che per il bot non ha né titolo sensato, né descrizione, né immagine di anteprima, né testo da indicizzare.

Il Modello risolve senza un framework SSR, senza Next.js, senza Node. Mette un file PHP davanti alla SPA, gli fa interrogare il database e gli fa iniettare nei posti giusti i meta tag corretti, prima che l'HTML arrivi al bot. È la versione thin-stack del server-side rendering: qualche centinaio di righe di PHP e zero infrastruttura nuova.

Ma questo capitolo ha tre fili da tenere insieme. Il primo è una storia di tentativi: il pattern vincente non è il primo che è stato provato, e i due flagship hanno scartato per strada una soluzione che a prima vista sembra la più naturale. Il secondo è una scala a tre gradini: i due flagship fanno un Dynamic Rendering completo, DIS si ferma a un proxy di anteprime social, e il gradino più capace porta con sé tre debiti. Il terzo è il filo della sicurezza aperto al CAP 8: il prerender è l'emettitore del `content` dove il buco XSS dei «quattro emettitori» è **ancora vivo**.

NOTA

Una tentazione da evitare: lo Static Prerendering (SSG). Per l'indicizzazione profonda verrebbe da aggiungere, oltre alla sola iniezione dei meta, uno **Static Prerendering** in build (tipo `vite-plugin-prerender`). Ma quella è **esattamente la SSG con Puppeteer che SimonePizziWebSite ha provato e poi abbandonato (§1)**: oggi nel suo repo è solo codice morto, con tanto di post-mortem. La soluzione realmente adottata dai flagship è il **Dynamic Rendering**. Attenzione anche a un dettaglio che inganna: una forma con un file `.sqlite` e connessione diretta non è il motore di SPW, che gira su MySQL, ma il proxy di DIS.

*
**

1. Il problema, e tre tentativi di risolverlo

La storia del prerendering in SPW è un percorso a tre tappe, e raccontarla spiega come si è arrivati al pattern finale.

La prima tappa (v1.7.3) iniettava soltanto titolo, descrizione e Open Graph nell'<head>. Bastava per le anteprime social, ma non per Google: il crawler trovava i meta corretti e un <body> ancora vuoto, e indicizzava pochissime pagine.

La seconda tappa fu la **SSG con Puppeteer**: uno script Node che, in fase di build, prerenderizzava ogni rotta in un file HTML statico su disco. Tecnicamente funzionava. Ma era concettualmente sbagliata per un CMS, e il post-mortem lo dice chiaro: rompeva il flusso «modifica online, pubblica» (serviva una build locale e un upload FTP a ogni articolo), congelava i dati al momento della build, e si portava dietro una catena di complicazioni (un bridge anti-CORS, un buffer `dist-static`, gli eventi di render). Fu abbandonata.

La terza tappa, quella attuale, è il **Dynamic Rendering**: niente build, niente file statici. Un solo `index.php`, in tempo reale, decide cosa servire a seconda di chi chiede.

CONSIGLIO

Il codice che resta dopo che la strategia è cambiata Della SSG abbandonata, in SPW, è rimasta l'impronta fossile: `prerender.php` è deprecato (risponde solo un avviso), `prerender.js` e `prerender-routes.js` sono codice morto (il `postbuild` esegue `clean-dist.js`, non loro), e una costante `IS_PRERENDERING` protegge dei rami che nessuno definisce più. SR, arrivato dopo, è andato dritto al Dynamic Rendering e non ha fossili. È un piccolo promemoria archeologico: quando si cambia strategia, il codice della strategia vecchia raramente sparisce: resta lì, inerte, finché qualcuno non si fida abbastanza da cancellarlo.

✱
✱✱

2. Dynamic Rendering: tutto in un solo `index.php`

Il pattern vincente concentra tutto in `public/index.php`, che `.htaccess` mette davanti a `index.html` e a cui dirotta ogni URL virtuale di React Router. Per ogni richiesta, in tempo reale, il file fa quattro cose: deduce dal percorso che tipo di pagina è, interroga il database per i dati reali, e poi sceglie tra due strade in base a chi sta chiedendo.

La scelta passa da `isCrawler()`, che annusa lo User-Agent:

```
// public/index.php:46-86 (SPW) - lo snodo crawler vs umano
function isCrawler(): bool {
    $ua = $_SERVER['HTTP_USER_AGENT'] ?? '';
    if (empty($ua)) return false;
    $crawlers = ['Googlebot', 'Bingbot', 'facebookexternalhit', 'Twitterbot'
    ↵ ,
        'TelegramBot', 'WhatsApp', 'Discordbot', /* ... */];
    foreach ($crawlers as $bot) { if (stripos($ua, $bot) !== false) return t
    ↵ rue; }
    return false;
}
$isCrawler = isCrawler();
if ($isCrawler && $pageType !== 'admin') { /* HTML completo server-side per
    ↵ il bot */ }
else { /* index.html di Vite + meta iniettati nell'head; il body lo fa React
    ↵ */ }
```

All'utente vero arriva l'`index.html` di Vite con i meta e il JSON-LD iniettati nell'`<head>`, e il `<body>` lo costruisce React come sempre. Al crawler, invece, arriva un HTML già completo, con titolo, meta e il **corpo dell'articolo** scritto direttamente nel `<body>`, così il bot indicizza senza eseguire una riga di JavaScript. Il file rivendica esplicitamente che non è cloaking: il contenuto servito al bot è lo stesso che l'utente vede dopo l'idratazione di React.

Perché funzioni, il PHP deve conoscere le rotte quanto le conosce React Router, e qui sta il primo costo del metodo:

```
// public/index.php:133-154 (SPW) - routing server-side speculare a React Ro
< uter
if (count($uri_parts) === 0)           $pageType = 'homepage';
elseif ($uri_parts[0] === 'admin')     $pageType = 'admin';      // niente
< SEO per l'area admin
elseif ($uri_parts[0] === 'tutti-i-progetti') $pageType = 'projects';
elseif (count($uri_parts) === 2) { $pageType = 'article'; $catSlug = $uri_p
< arts[0]; $slug = $uri_parts[1]; }
elseif (count($uri_parts) === 1) { $pageType = 'category'; $catSlug = $uri_p
< arts[0]; }
```

ATTENZIONE

Il prezzo di non avere un framework SSR: la doppia verità delle rotte Le rotte ora vivono in due posti: in `App.tsx` per React, e in `index.php` per i bot. Aggiungere una pagina pubblica significa toccarle entrambe. Una rotta nuova che il PHP non conosce ricade nel ramo di default, e il bot riceve i meta sbagliati senza che nessuno se ne accorga. È il compromesso del Dynamic Rendering fatto a mano: nessuna infrastruttura, ma una mappa da tenere allineata a mano in due linguaggi diversi.

L'iniezione vera e propria è un'operazione chirurgica sull'HTML compilato da Vite: si rimuove il `<title>` generato dal build e si inserisce il blocco SEO prima di `</head>`.

```
// public/index.php:576-614 (SPW) - iniezione meta + rimozione del title di
< Vite
$seoInjection = '<title>' . $metaTitle . '</title>'
. '<meta name="description" content="' . esc($metaDesc) . '" />'
. '<link rel="canonical" href="' . esc($canonicalUrl) . '" />'; // + 0
< G/Twitter
if ($jsonLd) { $seoInjection .= '<script type="application/ld+json">' . json
< _encode($jsonLd) . '</script>'; }
$htmlContent = preg_replace('/<title>.*?</title>/s', '', $htmlContent, 1);
< // evita il title doppio
$htmlContent = str_replace('</head>', $seoInjection . '</head>', $htmlConten
< t);
```

Ogni valore che arriva dal database passa per un `esc()` (cioè `htmlspecialchars`) prima di finire in un attributo. Tutti i valori, con un'eccezione che è il cuore del §4: il corpo dell'articolo.

✱
✱✱

3. La scala a tre gradini: Dynamic Rendering o OG-proxy

I due flagship fanno la stessa cosa. SR ha copiato il motore di SPW quasi alla lettera: stesso `isCrawler()`, stessi helper (`esc`, `truncateText`, `absImageUrl`), stessa iniezione. La diffe-

renza più visibile è cosmetica (SR deriva `baseUrl` da `$_SERVER['HTTP_HOST']`, SPW usa il suo `SITE_URL` canonico) e il tipo di dati strutturati: SPW emette `Article` e `CollectionPage`, SR aggiunge un `RadioStation` adatto al suo dominio. Ma l'impianto è lo stesso.

DIS sta su un altro gradino. Il suo `index.php` non è un Dynamic Rendering engine: è un proxy di anteprime social, e lo dichiara. Non annusa lo `User-Agent`, quindi serve a tutti lo stesso HTML (niente rischio di cloaking) e inietta soltanto i meta, lasciando il corpo a React. E i meta li scrive passando ogni valore per `htmlspecialchars`:

```
// public/index.php:93-109 (DIS) - iniezione meta sicura per escape, nessun
↳ corpo prerenderizzato
function injectTag($html, $tag, $content, $property = null) {
    if (!$content) return $html;
    if ($tag === 'title')
        return preg_replace('/<title>.*?</title>/s', "<title>".htmlspecialchars
↳ hars($content)."</title>", $html);
    $attr = $property ? "property" : "name";
    return /* inserisce */ '<meta ' . $attr . '=' . $tag . ' content="' . htmlspecialchars
↳ lchars($content) . '" />';
}
```

CONSIGLIO

Dynamic Rendering completo o OG-proxy leggero Sono due risposte diverse alla stessa domanda. Il Dynamic Rendering (SPW, SR) indicizza meglio sui crawler che non eseguono JavaScript, perché serve loro anche il corpo dell'articolo; in cambio è più complesso, mantiene una doppia mappa di rotte, e (lo vedremo) riapre un buco di sicurezza. L'OG-proxy (DIS) è semplice e onesto: niente cloaking, niente corpo da rimettere, solo meta escaped; in cambio la SEO testuale per i bot no-JS è debole. Non c'è un vincitore assoluto: un sito-festival che vive di anteprime social su Telegram sta benissimo col proxy leggero; un sito di contenuti che vuole posizionarsi su Google ha bisogno del corpo prerenderizzato. La scelta segue cosa devi far vedere, e a chi.

✱
✱ ✱

4. Il prerender riapre il buco XSS: la falla viva dei quattro emettitori

Al CAP 8 abbiamo fissato il quadro dei quattro emettitori del `content`: lo stesso HTML salvato grezzo nel database esce verso il mondo da quattro punti diversi, e la sanitizzazione che vive nel render React non copre gli altri tre. Il prerender SEO è il punto dove quel buco è **ancora aperto**.

Per servire al bot un corpo indicizzabile, il ramo crawler riemette il `content` dell'articolo. Ma non lo passa per `DOMPurify` (che vive solo nel render React): lo passa per `strip_tags` con una `allowlist` di tag.

```
// public/index.php:403-405 (SPW) - il corpo per il crawler: strip_tags allo
↳ wlist, NON DOMPurify
<div>' . strip_tags($article['content'],
    'p><br><h2><h3><h4><ul><ol><li><strong><em><a><blockquote><pre><cod
↳ e>') . '</div>
```

La differenza non è teorica. `strip_tags` con `allowlist` rimuove i tag pericolosi a livello di elemento: `<script>`, `<iframe>`, `<svg>` non sono in lista, quindi spariscono. Ma `strip_tags` **non guarda gli attributi** dei tag che lascia passare. Un `<a href="javascript:...">` o un `<p onmouseover="...">` sopravvive intatto. `DOMPurify` quei due li avrebbe rimossi; `strip_tags` no.

E il ramo crawler è raggiungibile da chiunque, perché `isCrawler()` si fida solo dello `User-Agent`. Basta presentarsi con `User-Agent: Googlebot` per ricevere il corpo passato dal solo `strip_tags`. La superficie è stretta (perché di norma è la vittima a dover avere un UA da crawler, e i bot non eseguono JavaScript), ma è un secondo percorso di render del contenuto utente che non condivide la difesa del primo.

ATTENZIONE

Quando copi un pattern, copi anche la sua falla SR-C7 è SPW-C7 quasi verbatim: stesso `isCrawler`, stessi helper, **stesso identico** `strip_tags` con la stessa `allowlist`. Copiando il pattern-firma del Dynamic Rendering, SR ne ha copiato anche il buco. E nel copiarlo ha perso un pezzo (la regola di visibilità, §5) introducendo un bug nuovo. DIS, che non prerenderizza il corpo, è l'unico immune: non per una difesa migliore, ma perché emette di meno. È il rovescio esatto della sua mancanza di `DOMPurify` vista al CAP 8: là il «non difendere» faceva male, qui il «non emettere» salva. La lezione chiude il filo aperto al CAP 8: quando la difesa XSS vive in un solo render, ogni altro emettitore deve ri-sanitizzare per conto suo, e prima o poi uno se ne dimentica. La soluzione giusta non è aggiungere `strip_tags` qui e `DOMPurify` là, ma **una sola funzione di sanitizzazione lato server**, usata da tutti gli emettitori PHP del `content`. Il filo si chiuderà al CAP 12 (il feed, che escapa) e al CAP 13 (la newsletter, che non emette).

*
**

5. La SEO che indicizza le bozze

C'è una seconda regola che il prerender dovrebbe condividere con il resto del sistema, e che SR e DIS dimenticano. Al CAP 9 il ciclo di vita stabilisce che un contenuto è pubblico solo se `status = 'published'` e la sua data di pubblicazione è passata. SPW riusa questa regola nelle query SEO e perfino nella sitemap. SR e DIS filtrano solo sulla data:

```
// public/index.php:130-135 (SR) – manca il filtro su status
$stmt = $pdo->prepare(
    "SELECT id, title, slug, summary, content, cover_image, published_at
    FROM news WHERE slug = ? AND published_at <= ? LIMIT 1"); // ← niente
↳ "AND status = 'published'"
```

La conseguenza è una crepa tra due idee di «pubblico». Un articolo in bozza con una data passata è invisibile nell'API e nella lista pubblica, ma trapela nei meta, nell'HTML servito al crawler, nel blocco «ultime notizie» della homepage e, in SR, perfino nella sitemap, che lo consegna a Google.

ATTENZIONE

Due idee di «pubblico» nello stesso sito «Pubblico per l'utente» e «pubblico per il bot» dovrebbero essere la stessa cosa, ma quando la regola di visibilità è scritta a mano in ogni query, è facile che un percorso la applichi e un altro no. SPW la tiene allineata in tre file (API, prerender, sitemap); SR la perde proprio nel file che la dà in pasto ai motori di ricerca. Anche qui la radice è la stessa del buco XSS: una regola di dominio che non vive in un punto solo finisce per divergere tra i suoi consumatori.

6. La seo-cache che sopravvive al suo lettore

SR, a differenza di SPW, ha una cache SEO: a ogni salvataggio scrive un file `.cache/seo_news_<md5(slug)>.json`, lo rigenera in blocco con uno script dedicato, lo cancella quando elimini il contenuto. Tutto perfettamente mantenuto, tranne un dettaglio: **nessuno la legge**.

Una ricerca su tutto il codice non trova un solo punto che apra quei file. Il motore v3.0 interroga il database direttamente. E la prova del delitto è scritta nel codice stesso: il banner di `index.php` dichiara di aver sostituito «il proxy cache-file della v2», e uno script di rigenerazione contiene un commento che tradisce il lettore scomparso:

```
// public/api/rebuild_seo_cache.php:67-79 (SR)
$seoData = [
    'title' => $speaker['name'],
    'description' => $speaker['role'] . ' - ' . substr($speaker['bio'] ?? ''
    ↪ , 0, 150) . '...',
    'name' => $speaker['name'], // Injected for consistency with index.php r
    ↪ eader ← il reader non esiste più
];
file_put_contents($cacheDir . '/seo_speaker_' . md5($speaker['id']) . '.json'
    ↪ , json_encode($seoData));
```

La versione 2 era un proxy che leggeva quei JSON; la v3.0 ha riscritto il motore con la query diretta e ha rimosso il lettore senza spegnere gli scrittori. Ogni salvataggio paga ancora il costo di scrivere un file che nessuno aprirà. È l'opposto di SPW, che la cache SEO non l'ha mai avuta, per scelta.

CONSIGLIO

La cache che sopravvive al suo lettore È un modo specifico in cui il codice marcisce: riscrivi il consumatore di qualcosa e dimentichi di spegnerne i produttori. La cache resta, viene aggiornata, invalidata, rigenerata, e non serve a niente. Vale anche come correzione a un punto del CAP 7, che citava lo script `rebuild_seo_cache` come utile per le migrazioni: oggi rigenera una cache morta. Il prossimo paragrafo dà a questa cache un risvolto meno innocente di così.

7. Quando l'entry-point diventa un bersaglio

C'è un motivo per cui una cache che serve i bot senza toccare il database non è solo un'ottimizzazione. Lo ha imparato Runtime Radio, e la lezione è abbastanza importante da avere una casa in questo capitolo: il vettore dell'attacco è esattamente l'entry-point SEO che abbiamo appena descritto.

Tra il 23 e il 27 febbraio 2026 il sito ha vissuto due crisi sovrapposte. La prima fu il collasso del database SQLite sotto un carico cresciuto per mesi, risolto con una migrazione d'emergenza a MySQL in meno di un giorno (la racconta il CAP 15). La seconda arrivò mentre l'infrastruttura era ancora instabile: un'ondata di richieste che restituivano errori 503 e 500, con picchi di traffico verticali, impossibili da spiegare con la crescita organica.

Il vettore era elegante. Migliaia di bot ostili **simulavano i crawler dei social**, mandando uno User-Agent da Telegram o Facebook. E ogni richiesta con quell'UA colpiva `index.php`, che per costruzione interrogava il database per estrarre i meta. Il motore SEO, progettato per far apparire bene i link condivisi, era diventato una leva: una richiesta a costo quasi zero per l'attaccante forzava una query sul database appena migrato e ancora fragile.

```
Bot ostile → User-Agent: TelegramBot → richiesta a /news/uno-slug  
→ index.php → query al DB per i meta → risposta  
→ il bot scarta e ripete, mille volte al secondo  
→ il DB cede → 503/500
```

La risposta separò il percorso dei bot da quello degli utenti. Le risposte per i crawler social vennero servite da **file JSON statici precompilati**, scritti al momento della pubblicazione di un articolo: il bot riceve i meta in pochi millisecondi, senza che il database venga mai interrogato. Solo gli utenti veri, con un browser che esegue JavaScript, percorrono la strada normale. E il servizio core, lo streaming audio, restò attivo dietro una pagina di manutenzione statica anche mentre il resto era offline.

Quei file JSON precompilati hanno un nome familiare: `.cache/seo_*.json`. Sono la stessa seo-cache del paragrafo precedente. È difficile non leggere i due fatti insieme: la cache nacque come scudo anti-DDoS, un percorso per i bot che non toccava il database; poi la riscrittura v3.0 del motore tornò alla query diretta e rimosse il lettore, lasciando la cache scritta-ma-mai-letta. Lo scudo è ancora lì, viene ancora lucidato a ogni salvataggio, ma non è più imbracciato da nessuno.

ATTENZIONE

Ogni endpoint pubblico che interroga il DB è un bersaglio; lo User-Agent non è un gatekeeper

Tre cose da portarsi a casa. La prima: qualunque endpoint non autenticato che, per rispondere, interroga il database, è una leva d'attacco volumetrico. Se la risposta a una richiesta ripetitiva può venire da una cache statica, quella cache non è solo performance, è un layer di sicurezza. La seconda, già incontrata al CAP 10: i bot social si riconoscono dallo User-Agent, e lo User-Agent si falsifica. Lo si può usare per *ottimizzare* (servire una cache ai bot riconosciuti), mai come barriera d'accesso. La terza, la più scomoda: una difesa che funziona può essere smontata

senza farlo apposta, in una riscrittura che guarda solo alle funzionalità. La seo-cache di Runtime Radio non è stata «tolta»: è stata orfanata, e con lei la protezione che dava.

8. sitemap, robots e dati strutturati

Resta l'infrastruttura SEO di contorno, che è facile liquidare come «già gestita dall'.htaccess». In realtà c'è un pattern preciso: `sitemap.xml` e `robots.txt` non sono file fisici, ma rewrite verso `sitemap.php` e `robots.php`, generati in tempo reale.

```
# public/.htaccess (SPW) - niente file fisici: sitemap e robots sono PHP
DirectoryIndex index.php index.html          # il SEO Engine prima di
↳ index.html
RewriteRule ^sitemap\.xml$ sitemap.php [L,NC]
RewriteRule ^robots\.txt$ robots.php [L,NC]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_URI} !^/api/ [NC]
RewriteRule ^(.*)$ /index.php [L,QSA]      # ogni URL virtuale Rea
↳ ct → SEO Engine
```

Così la sitemap è sempre fresca senza prerendering, e il suo `baseUrl` si deriva dall'host della richiesta, quindi lo stesso file funziona su produzione e staging. Il `robots.php` di SR ci aggiunge una piccola personalità editoriale: blocca i crawler SEO commerciali (Ahrefs, Semrush, DotBot) e impone un `Crawl-delay`, per non sprecare banda con strumenti che non portano lettori.

Sui dati strutturati, entrambi i flagship costruiscono il JSON-LD come array PHP, scegliendo il tipo in base alla pagina: `Article` o `NewsArticle` per gli articoli, `CollectionPage` per gli elenchi, e in homepage un `@graph` che descrive il sito e l'autore (o, per Runtime Radio, una `RadioStation`). DIS, fedele al suo gradino, non emette JSON-LD: si ferma ai meta.

In sintesi

Indicizzare una SPA senza un framework SSR si può, con un PHP che fa da front controller, annusa il bot e gli serve un HTML completo. Ma il Dynamic Rendering non è gratis: duplica la mappa delle rotte, e soprattutto riapre il buco XSS dei contenuti, perché riemette il `content` con uno strumento (`strip_tags`) più debole del `DOMPurify` del render. SPW lo fa bene, SR ne ha copiato anche i difetti e ne ha aggiunti di propri (le bozze indicizzate, la cache orfana), DIS evita i problemi facendo di meno. E la storia del DDoS di febbraio chiude il cerchio: l'entry-point che rende il sito visibile è lo stesso che, sotto sforzo, lo fa cadere, e la cache che lo salvò è oggi un attrezzo dimenticato in un angolo.

IMPORTANTE
Il Canone

- Indicizza la SPA con Dynamic Rendering da un entry-point PHP (UA-sniff), non con una SSG fragile.
- Il prerender riemette il `content`: sanitizzalo con la **stessa** difesa del render, perché `strip_tags` con `allowlist` lascia passare gli attributi (`onerror`, `href="javascript:"`).
- Rispetta lo `status` anche nel ramo crawler: non indicizzare le bozze.
- `sitemap/robots` dinamici; lo User-Agent non è un gatekeeper di sicurezza.

✱
✱ ✱

Prossimo Capitolo: RSS Feed & Syndication. Il feed è l'emettitore più sicuro del contenuto, quello che chiude il filo dei quattro emettitori, e insieme il luogo dove si annidano un proxy CORS e un po' di teatro della sicurezza.

CAPITOLO 12: RSS Feed & Syndication

Il feed RSS è il canale con cui un sito consegna i propri contenuti a chi non viene a leggerli sul sito: aggregatori, lettori di notizie, app podcast, bot di Telegram. Nel Modello è sempre lo stesso gesto thin-stack: un endpoint PHP che, a ogni richiesta, interroga il database e stampa l'XML al volo. Niente cron, niente file `.xml` materializzato su disco, niente libreria di serializzazione. Un articolo pubblicato compare nel feed al primo refresh.

Ma questo capitolo ha una lente precisa, ed è la chiusura di un filo. Al CAP 8 abbiamo aperto il quadro dei quattro emettitori del `content`: lo stesso HTML salvato grezzo nel database esce verso il mondo da quattro punti, ognuno con la propria difesa. Al CAP 11 abbiamo visto il prerender lasciare il buco aperto. Il feed è l'emettitore che lo **chiude**: in tutti e tre i siti, o non emette affatto il `content`, o lo escapa per intero. È il punto più sicuro della catena.

E proprio perché sulla sicurezza i tre siti convergono, è interessante quanto divergano su tutto il resto. La geografia va da un file unico a un trittico di file con ruoli opposti, fino a un feed che non è nemmeno di notizie ma di podcast. E sulla disciplina del GUID si vede una piccola storia di regressione: una buona idea risolta da un sito e dimenticata dagli altri due.

NOTA

Tre malintesi diffusi su questi feed. Tre punti, spesso dati per scontati, sono in realtà sbagliati o fuorvianti. Il **feed podcast non è di SiteRuntime ma di DISINTELLIGENZA**: SR non genera un feed podcast, *consuma* quelli esterni con un proxy (§4). Il **catch vuoto** viene a volte insegnato come «fallback silenzioso» virtuoso: è invece un anti-pattern, perché serve un feed troncato con HTTP 200 (§7). E **feed.php non è un «alias»** di `rss.php`: è il feed podcast di DIS, un endpoint con tutt'altro scopo (§3).

**

1. L'anatomia comune di un feed

Sotto le differenze, i feed dei tre siti condividono la stessa ossatura. Un solo file, una GET, l'XML stampato in diretta dal database. L'esempio è il `rss.php` di SimonePizziWebSite, il più lineare dei tre:

```
// public/api/rss.php (SPW) - il feed news, real-time dal DB
header('Content-Type: application/rss+xml; charset=utf-8');
$pdo = Database::connect();
date_default_timezone_set('Europe/Rome'); // fuso forzato ne
```

```

< l singolo file
$ita_now_str = date('Y-m-d H:i:s');
define('RSS_FEED_LIMIT', 50);

echo '<?xml version="1.0" encoding="UTF-8" ?>' . "\n<rss version=\"2.0\">\n<
< channel>\n";
echo '  <title>' . htmlspecialchars($site_title) . "</title>\n";
echo '  <link>' . htmlspecialchars($base_url) . "</link>\n";
echo '  <language>it-IT</language>' . "\n";

```

Tre attenzioni ricorrono in tutti e tre. Ogni campo dinamico che entra nell'XML passa per htmlspecialchars: titolo, link, URL dell'immagine, nessuno finisce crudo nel feed. Il fuso Europe/Rome viene rinforzato dentro il file stesso, senza fidarsi del timezone impostato altrove, perché ogni file che confronta date col database deve difendersi da solo da un server in un fuso sbagliato (la lezione del CAP 9). E la data di pubblicazione esce sempre in formato RFC-822, quello che RSS 2.0 pretende, tramite la costante DATE_RSS: mai ISO 8601, che molti lettori rifiutano.

C'è anche una piccola asimmetria condivisa. Mentre sitemap.xml e robots.txt sono serviti da URL puliti tramite una rewrite (lo abbiamo visto al CAP 11), il feed si raggiunge sempre all'URL grezzo del file PHP: /api/rss.php, /api/feed_news_rss.php, /api/feed.php. Nessuno dei tre siti gli dà un /feed.xml pulito. Cura SEO sugli altri asset, nessuna cura qui.

*
**

2. Il feed chiude il filo dei quattro emettitori

Riprendiamo la tabella aperta al CAP 8. Lo stesso content, salvato grezzo, esce da quattro emettitori, e l'unica difesa che vale davvero è quella che ognuno mette per conto suo. Il render usa DOMPurify (tranne DIS). Il prerender usa strip_tags con allowlist, che lascia passare gli attributi: è la falla viva del CAP 11. Il feed, terzo emettitore, la chiude.

#	Emettitore	Cosa emette del content	Difesa	Esito
1	Render React (CAP 8)	content pieno	DOMPurify (SPW, SR) / niente (DIS)	choke-point reale; DIS scoperto
2	Prerender SEO (CAP 11)	content pieno	strip_tags allowlist (solo tag)	buco sugli attributi (SPW, SR)
3	Feed RSS (questo capitolo)	niente (SPW, DIS) / preview escapata (SR)	htmlspecialchars / strip_tags+htmlspecialchars	sicuro
4	Newsletter (CAP 13)	(la chiude il prossimo capitolo)	—	—

Il feed è sicuro, ma per due strade diverse. SPW lo è per sottrazione: il suo rss.php carica la colonna content nella query ma non la stampa mai; la <description> dell'item usa solo l'excerpt, escapato.

```

// public/api/rss.php (SPW) — content SELECTato ma MAI emesso; description =
< solo excerpt escapato
echo '  <description>' . htmlspecialchars($article['excerpt']) . "</descri

```

```

↳ ption>\n"; // ← excerpt, non content
// (la colonna content è nella SELECT ma non c'è un solo echo di $article['c
↳ ontent'])

```

SR invece **rompe la sottrazione**: quando il riassunto manca, ripiega sui primi 500 caratteri del content. Ma lo neutralizza con uno strip_tags senza allowlist, che toglie *tutti* i tag, seguito da htmlspecialchars:

```

// public/api/feed_news_rss.php (SR) - emette una preview del content, ma es
↳ capata per intero
$descriptionText = $article['summary'] ?: $article['content_preview'] . '...
↳ '; // usa il content se manca summary
$description = htmlspecialchars(strip_tags($descriptionText));
↳ // strip_tags SENZA allowlist + escape

```

La differenza con il prerender del CAP 11 è tutta qui: là strip_tags aveva una allowlist e lasciava i tag (e i loro attributi pericolosi); qui non ha allowlist, quindi non resta nessun tag, e quel poco testo viene anche trasformato in entità. DIS, dal canto suo, non emette un feed di notizie: il suo è un feed podcast, e il content delle news non lo tocca nessuno.

IMPORTANTE

Due modi di rendere sicuro un feed: non emettere, o escappare Il feed dimostra in positivo la tesi che attraversa CAP 8, 11 e 13: quando la difesa XSS vive in un solo render, ogni altro emettitore deve cavarsela da sé, e il feed ci riesce. SPW non emette il campo pericoloso (sicuro per sottrazione: robusto, ma perde informazione, niente articolo completo nel feed). SR lo emette ma lo escapa per intero (sicuro per escape: più informativo, ma più fragile, perché basterebbe rimettere un'allowlist a strip_tags per riaprire il buco, com'è successo nel prerender). Esiti uguali, filosofie opposte. La conclusione resta quella del filo: la sanitizzazione dovrebbe vivere una volta sola, lato server, non essere reinventata da ogni emettitore. Il quadro si chiude del tutto al CAP 13, dove nessuno emette il content.

3. La geografia: un file, un trittico, un feed podcast

Sulla forma, i tre siti non potrebbero essere più diversi.

SPW è un file solo, rss.php: feed di notizie, minimale e rigoroso. SR è un trittico, e qui sta la sua complessità: feed_news_rss.php genera il feed news del sito, rss.php (stesso nome di SPW, ruolo opposto) è un proxy che *consuma* feed esterni, e feed_config.php è un dispenser che serve all'admin l'indirizzo del feed. DIS è un caso a parte: niente feed di notizie, solo feed.php, un feed **podcast** in RSS 2.0 con il namespace iTunes, sottoscrivibile da un'app come Apple Podcasts.

NOTA

Produrre un feed podcast, o consumarne uno: non è la stessa cosa È un punto facile da confondere. Il sito che *genera* un feed podcast è DIS, con feed.php: legge la tabella podcasts ed emette gli episodi con <enclosure> audio e i tag <itunes:*>. SR fa l'opposto: non genera nessun feed podcast, ma con rss.php *scarica* i feed podcast

altrui (Spreaker, il suo AzuraCast) per mostrarli sul sito. Produrre e consumare un feed sono due operazioni speculari, ed è proprio il contrasto tra questi due siti a renderlo evidente. Lo vediamo al paragrafo seguente.

*
**

4. Il proxy CORS inbound: consumare feed altrui

Un browser non può leggere un feed ospitato su un altro dominio se quel dominio non manda gli header CORS, e i feed podcast esterni di solito non li mandano. SR risolve con un proxy server-side: `rss.php` scarica il feed altrui e lo restituisce `same-origin`. La parte interessante è la difesa, perché un proxy che scarica qualunque URL gli passi è un *open proxy*, una porta spalancata verso attacchi SSRF.

```
// public/api/rss.php (SR) - proxy inbound: allowlist + https-only + stale f
↳ allback
$allowedHosts = ['www.spreaker.com', 'spreaker.com', 'player.runtimeradio.co
↳ m'];
if ($scheme !== 'https' || !in_array($host, $allowedHosts, true)) {
    http_response_code(400);
    echo json_encode(['success' => false, 'error' => 'Feed URL not allowed']);
↳ ); exit; // niente open proxy
}
// ... cache su disco rss_<md5>.xml, TTL 30' ...
if ($xml === false || $httpCode !== 200) {
    if (file_exists($cacheFile)) { header('X-Cache: STALE'); readfile($cache
↳ File); exit; } // una cache scaduta è meglio di niente
}
```

Due scelte sane: una allowlist di host (il proxy scarica solo da quei domini) e l'obbligo di HTTPS chiudono la porta SSRF; una cache su disco con fallback «stale» fa sì che, se l'upstream è giù, si serva l'ultima copia buona invece di fallire. C'è però un dettaglio meno sano sul lato client: se il proxy del sito non risponde, il codice frontend ripiega su proxy pubblici di terzi (CodeTabs, AllOrigins), che l'allowlist non la conoscono. La resilienza si compra con una dipendenza da terzi non dichiarata.

ATTENZIONE

Un proxy che scarica URL è un bersaglio SSRF Qualunque endpoint che, su richiesta, scarica un URL fornito dall'esterno va trattato come una potenziale leva per raggiungere risorse interne (metadati cloud, servizi su `localhost`, IP privati). Le due difese di `rss.php` sono il minimo corretto: una allowlist esplicita di host fidati e il rifiuto di tutto ciò che non è HTTPS. La regola è la stessa del CAP 10 sull'IP: non fidarti di un valore che arriva dal client, in questo caso l'URL da scaricare. E attenzione ai fallback: il ripiego del client su proxy pubblici aggira l'allowlist e va saputo.

*
**

5. feed_config.php: il lucchetto sulla porta accanto

Nel trittico di SR c'è un terzo file che vale un box a sé, perché è un caso da manuale di sicurezza apparente. `feed_config.php` è protetto da `isAdmin()` e il suo commento promette molto: «Restituisce l'URL del feed RSS privato solo ad admin autenticati. Il token non viene mai esposto nel bundle JavaScript pubblico».

```
// public/api/feed_config.php (SR) - gata l'URL, ma l'endpoint è pubblico
require_once 'auth_utils.php';
if (!isAdmin()) { http_response_code(403); echo json_encode(['success'=>false, 'error'=>'Forbidden']); exit; }
$origin = (/* https */ . '://' . $_SERVER['HTTP_HOST']);
echo json_encode(['success' => true, 'feed_url' => $origin . '/api/feed_news_rss.php']);
```

Due cose non tornano. Non esiste alcun token: l'URL restituito è nudo. E soprattutto il feed che quell'URL raggiunge, `feed_news_rss.php`, è **completamente pubblico**: non include `auth_utils.php`, non chiama `isAdmin()`, chiunque può leggerlo senza login. Il gate `isAdmin()` protegge l'atto di *scoprire* un indirizzo che è comunque pubblico e indovinabile.

ATTENZIONE

Il lucchetto sulla porta accanto Mettere un gate sul dispenser di un URL non rende privato l'endpoint che quell'URL raggiunge. È sicurezza per oscurità, e l'oscurità qui non c'è nemmeno, perché il nome del file è prevedibile. Quasi certamente è il fossile di un'intenzione mai realizzata: un feed news autenticato con token per il bot Telegram (coerente con il `TELEGRAM_BOT_TOKEN` che vive nei segreti, CAP 10), progettato e mai costruito. La lezione: se un endpoint deve essere privato, l'autenticazione va su *quell'endpoint*, non sul foglietto che ne riporta l'indirizzo.

**
**

6. Il GUID che ripubblica

Ogni <item> di un feed ha un <guid>, l'identificatore con cui i consumatori riconoscono se un contenuto è nuovo o già visto. Sbagliarlo ha una conseguenza concreta e fastidiosa: se il GUID di un articolo cambia, gli aggregatori e i bot lo trattano come un articolo nuovo e lo ripubblicano. Su un'integrazione Telegram, significa spammare di nuovo tutti gli iscritti.

SPW ha risolto il problema in modo netto: il GUID è un URN costruito sull'ID di database, disaccoppiato dall'URL.

```
// public/api/rss.php (SPW) - GUID stabile, indipendente dall'URL
$stable_guid = 'urn:simonepizzi:article:' . (int)$article['id'];
echo ' <guid isPermaLink="false">' . $stable_guid . "</guid>\n";
```

Così un cambio di slug o di categoria non tocca l'identità dell'articolo: per i consumatori resta lo stesso contenuto. È la stessa preoccupazione che, lato URL, giustifica i redirect 301: tenere stabile l'identità di una pagina anche quando il suo indirizzo cambia.

Gli altri due siti questa idea l'hanno persa. SR usa il permalink come GUID, con `isPermaLink="true"`: al primo cambio di slug, l'articolo si ripubblica (e SR non ha nemme-

no i 301 a fare da rete). DIS scende ancora più giù e usa l'URL del file audio: basta che `migrate_media` sposti l'audio e ogni episodio torna «nuovo» nelle app podcast.

CONSIGLIO

L'identità stabile di un articolo: un URN, non il permalink Il GUID a URN è la raccomandazione corretta. Quello che va aggiunto è che è una best practice che due siti su tre **non** seguono: SPW la applica, SR è regredito al permalink, DIS all'URL audio (il meno stabile di tutti). La regola, in una riga: l'identità di un item nel feed deve dipendere da qualcosa che non cambia mai (l'ID di database), mai dall'indirizzo, che cambia eccome.

*
**

7. Il catch vuoto che nasconde un database giù

Resta un punto spesso scambiato per virtù, e a torto. Sia SPW sia DIS avvolgono la query in un `try/catch` con il `catch` **vuoto**:

```
// public/api/rss.php (SPW) - il catch vuoto: l'header e <channel> sono già
< uscirli con HTTP 200
} catch (Exception $e) {
    // fallback silenzioso: nessun log, nessun 5xx
}
echo "</channel></rss>";
```

Il problema è la sequenza. Quando l'eccezione scatta, l'header `Content-Type` e l'apertura del `<channel>` sono già stati stampati, con codice di risposta 200. Il `catch` vuoto inghiotte l'errore, e la chiusura `</channel></rss>` esce comunque: il client riceve un feed perfettamente ben formato e **vuoto**, indistinguibile da «non ci sono novità». Nessun log, nessun 500, nessun segnale. Un database giù diventa invisibile.

SR fa meglio, anche se non perfettamente: intercetta l'eccezione e risponde con un vero errore HTTP.

```
// public/api/feed_news_rss.php (SR) - almeno segnala il guasto
} catch (PDOException $e) {
    http_response_code(500);
    echo "<error>Database Error</error>"; // (copre solo PDOException: un
< errore non-PDO sfuggirebbe)
}
```

ATTENZIONE

Il fallback silenzioso che nasconde un guasto Un errore mascherato da risposta valida è peggio di un errore visibile: nessuno lo nota finché un utente non si lamenta che il feed «non aggiorna da una settimana». La regola è di non aprire la risposta (header e primo output) prima di aver eseguito la parte che può fallire, oppure di gestire il fallimento con un codice HTTP onesto, come prova a fare SR. Il `catch` vuoto non è un fallback: è un guasto silenziato.

*
**

8. Cicatrici minori

Alcuni dettagli più piccoli, ma reali. Il filtro di visibilità del CAP 9 (`status = 'published'`) viene dimenticato anche qui: il feed news di SR filtra solo sulla data, quindi una bozza con data passata trapela nel feed, terzo file dopo `l'index.php` e la sitemap a dimenticare la stessa regola. L'`<enclosure>` dell'immagine dichiara `type="image/jpeg"` cablato nel codice, ma l'upload converte le cover in WebP (CAP 7): i lettori più pignoli scartano una miniatura il cui MIME non corrisponde. E in entrambi i flagship la configurazione del canale (titolo, descrizione) è hardcoded, con un commento che promette di leggerla «dai settings»: una configurabilità annunciata e mai cablata, di cui DIS è la versione estrema (legge chiavi `podcast_*` che nessuno popola, quindi gira sempre sui default).

Su DIS c'è anche un dettaglio che è quasi un reperto. Il `feed.php` contiene commenti in cui il codice ragiona ad alta voce con sé stesso:

```
// public/api/feed.php (DIS) — il codice che discute i propri dubbi, in prod
↳ uzione
// During init_db.php step I created a 'podcasts' table? Let's check init_db
↳ .php if I can.
// Fallback: ... create it via SQL here? No, bad practice on GET.
// I'll assume it exists or use NEWS with a category.
```

NOTA

Quando il codice racconta i suoi dubbi Non è un bug, ma è una cicatrice di processo: il ritratto di un codebase generato in conversazione con un assistente, e mai riletto prima del deploy. Lo stesso tono si ritrova in altri file di DIS (`l'init_db.php`, `l'api.ts`). In produzione il codice dovrebbe affermare, non interrogarsi: questi monologhi vanno tolti nella rilettura, perché rivelano, a chiunque apra il sorgente, esattamente cosa l'autore non aveva verificato.

✱
✱✱

9. Annunciare e consegnare il feed

Un feed serve a poco se nessuno lo trova. Va annunciato nell'`<head>` dell'HTML, così browser e lettori RSS lo scoprono da soli:

```
<link rel="alternate" type="application/rss+xml" title="Feed Notizie" href="
↳ /api/rss.php" />
```

E va consegnato ai distributori. SPW ha un bottone «Copia RSS» nell'area admin, che mette l'indirizzo negli appunti con un click; SR mostra l'URL in dashboard (è ciò a cui serve davvero `feed_config.php`, una volta tolta la pretesa di privacy). Piccola UX redazionale: dare a chi pubblica un modo per passare il feed a un aggregatore o a un bot senza trascriverlo a mano.

✱
✱✱

In sintesi

Il feed è la parte più tranquilla del giro del contenuto: l'emettitore che chiude il buco XSS, per sottrazione o per escape. Ma intorno a questa quiete i tre siti raccontano tre

storie diverse: SPW è minimale e disciplinato (GUID stabile, regola di visibilità rispettata, errori a parte); SR è più ricco e più frammentato, con un proxy inbound ben difeso, un dispenser che finge una privatezza che non c'è, e qualche regressione (le bozze nel feed, il GUID che ripubblica); DIS fa solo podcast, sui default, con i dubbi dell'autore ancora scritti nel codice. Il filo dei quattro emettitori, intanto, ha un'ultima casella da riempire: la newsletter.

IMPORTANTE **Il Canone**

- Feed RSS 2.0 valido: header corretto, date RFC-822, URL assoluti, `content` escapato (o non emesso affatto).
- Il feed è un emettitore del `content`: o lo escapi del tutto o non lo metti dentro.
- GUID stabile con `isPermaLink="false"` (un URN), non il permalink mutevole.
- Niente `catch` vuoto che maschera un guasto: meglio un 5xx esplicito. Un proxy inbound va protetto con allowlist host + https-only (anti-SSRF).

✧
✧ ✧

Prossimo Capitolo: Newsletter & Email System. L'ultimo emettitore del contenuto, che chiude del tutto il filo, e una scala di quanto si può semplificare un sistema di posta prima che diventi pericoloso.

CAPITOLO 13: Newsletter & Email System

Una lista email è uno dei pochi asset che un sito possiede davvero: non passa da un algoritmo, non rischia lo shadowban, non sparisce se una piattaforma chiude. I tre siti del Modello hanno tutti una newsletter fatta in casa, senza servizi esterni, e tutti la costruiscono sullo stesso scheletro thin-stack: un endpoint PHP che gestisce iscrizione e invio, e l'email composta come stringa HTML al volo.

Questo capitolo chiude un filo iniziato cinque capitoli fa. La newsletter è il **quarto e ultimo emettitore** del content: dopo il render (CAP 8), il prerender (CAP 11) e il feed (CAP 12), è l'ultimo posto da cui lo stesso HTML salvato grezzo potrebbe uscire verso un browser, quello del client di posta. La buona notizia, che vedremo alla fine, è che nessuno dei tre siti emette il content nell'email: il filo si chiude senza riaprire il buco XSS.

E proprio perché sul rischio XSS i tre convergono, la lente vera del capitolo diventa un'altra: **quanto si può semplificare un sistema di posta** prima che la semplificazione diventi pericolosa. Si va da un sistema completo, con double opt-in e protezione anti-abuso, fino a una `mail()` nuda che chiunque può usare per iscrivere o disiscrivere chiunque. E, come già al CAP 10, il sito col backend più ricco non è il più sicuro.

NOTA

Tre punti su cui è facile sbagliare. Tre aspetti di questo sistema vengono spesso fraintesi. Il primo è il **double opt-in**, la feature-cardine di due siti su tre: ometterlo, e mostrare al suo posto un'iscrizione attiva da subito con disiscrizione per sola email, significa adottare il modello di DIS, il più debole. Quella disiscrizione per sola email **non è «GDPR-Compliant»**: è la versione insicura (chiunque disiscrive chiunque). E un `usleep` **non è «Rate Limiting»**: è un'altra cosa (§4).

1. Il ciclo di vita di un iscritto

Sotto le differenze, l'anatomia è condivisa. L'endpoint smista per `?action=`, con le azioni pubbliche servite prima di un gate centrale e quelle admin dopo. L'email viene sempre validata lato server, senza fidarsi del form, e la ri-iscrizione si gestisce in modo reattivo: si prova l'INSERT e si cattura la violazione del vincolo UNIQUE, restituendo un successo neutro che non rivela chi è già in lista.

```
// public/api/newsletter.php (DIS) - validazione server-side + idempotenza p
↳ er cattura del duplicato
```

```

$email = filter_var($input['email'] ?? '', FILTER_VALIDATE_EMAIL);
if (!$email) { echo json_encode(['status'=>'error','message'=>'Email non val
↳ ida']); exit; }
try {
    getDB()->prepare("INSERT INTO subscribers (email) VALUES (?)")->execute(
↳ [$email]);
    echo json_encode(['status'=>'success','message'=>'Iscrizione completata'
↳ ]);
} catch (PDOException $e) {
    if ($e->getCode() == 23000) { // violazione UNIQUE → già iscritto, ma
↳ non lo riveliamo come errore
        echo json_encode(['status'=>'success','message'=>'Sei già iscritto!'
↳ ]);
    } else { echo json_encode(['status'=>'error','message'=>'Errore database
↳ ']); }
}
}

```

La disiscrizione è «morbida»: nessuno cancella il record, l'iscritto viene marcato `is_active = 0`, così si preserva la storia e si evitano re-iscrizioni accidentali. E l'email finale, qualunque sia il sito, è costruita come stringa HTML con layout a tabelle e CSS inline (i client di posta non leggono fogli di stile esterni), con le immagini rese assolute e un placeholder nel link di disiscrizione, sostituito per ogni destinatario.

NOTA

Lo schema minimale non è quello reale. La tabella `subscribers` a quattro colonne (`id`, `email`, `is_active`, `created_at`) è il modello di DIS. Lo schema reale di SPW e SR ha invece i campi del double opt-in: lo stato (`pending/confirmed/unsubscribed`), uno o più token, la data di conferma. In SR quello schema esteso convive con due versioni più vecchie create da altri script, una delle quali è un fossile SQLite che su MySQL si romperebbe: il runtime presuppone lo schema esteso e una query fallisce finché la migrazione giusta non è stata eseguita. È la stessa storia di «una tabella, più verità» del CAP 15.

*
**

2. Il double opt-in e il segreto del link di disiscrizione

Il double opt-in è la garanzia che chi iscrive un'email **possiede** quella casella: invece di attivare subito l'indirizzo, si crea un record in attesa e si manda un'email con un link di conferma; solo dopo il click l'iscrizione diventa attiva. È la feature più importante dell'intero sistema, e i tre siti la implementano su tre gradini.

SPW la fa nel modo da manuale, con **due token distinti**: uno di conferma, monouso, azzerato dopo l'uso; uno di disiscrizione, casuale e stabile, separato dal primo.

```

// public/api/subscribers.php (SPW) – due token con scopi diversi
$confirmToken = $forceConfirm ? null : bin2hex(random_bytes(32)); // m
↳ onouso
$unsubscribeToken = bin2hex(random_bytes(32)); // s
↳ tabile, separato
$status = $forceConfirm ? 'confirmed' : 'pending';
if (!$forceConfirm) { sendConfirmEmail($email, $name ?: 'Amico', $confirmTok
↳ en); }

```

SR la fa con **un solo token** che serve a entrambi gli scopi, conferma e disiscrizione, e che non viene mai azzerato né scade. Ne derivano due piccole conseguenze. L'email di conferma promette che «il link scade dopo il primo utilizzo», ma è falso: il token sopravvive all'uso. E poiché lo stesso token finisce nell'URL di disiscrizione di *ogni* newsletter, chi inoltra una mail consegna a un altro il potere di disiscrivere quell'utente.

```
// public/api/newsletter.php (SR) - un token, due scopi
// confirm: attiva l'iscrizione MA non azzerà il token
"UPDATE subscribers SET is_active = 1, confirmed_at = NOW() WHERE confirmati
↳ on_token = ?"
// send: l'URL di disiscrizione espone lo stesso confirmation_token
$unsubscribeUrl = 'https://runtimeradio.com/unsubscribe?token=' . urlencode($sub['
↳ confirmation_token']);
```

DIS non ha né l'uno né l'altro. L'iscrizione è attiva subito, e la disiscrizione avviene per sola email, senza alcun token:

```
// public/api/newsletter.php (DIS) - disiscrizione per sola email, via GET,
↳ senza token
if ($action === 'unsubscribe') {
    $email = filter_var($_GET['email'] ?? '', FILTER_VALIDATE_EMAIL);
    getDB()->prepare("UPDATE subscribers SET is_active = 0 WHERE email = ?")
↳ ->execute([$email]);
    echo "<h1>Disiscrizione completata</h1>"; // chiunque conosca l'email
↳ può disiscriverla
}
```

ATTENZIONE

Il link di disiscrizione ha bisogno di un segreto Questa disiscrizione per sola email viene spesso spacciata per «GDPR-compliant». È l'opposto: senza un token segreto, chiunque conosca o indovini l'indirizzo di un iscritto può disiscriverlo. E poiché è una GET, è anche *prefetch-able*: un client di posta che precarica i link può disiscrivere l'utente solo passandoci sopra. La versione corretta è quella di SPW, con un `unsubscribe_token` casuale e stabile (e, idealmente, una conferma con POST dalla pagina di atterraggio). Il double opt-in protegge l'ingresso; un token di disiscrizione protegge l'uscita. Servono entrambi.

**

3. Spedire: `mail()` nativa o SMTP autenticato

Il trasporto è il punto in cui SR si stacca dagli altri due. SPW e DIS usano la `mail()` nativa di PHP, che si appoggia al `sendmail` di sistema: zero configurazione, ma deliverability fragile, perché senza SPF e DKIM le email finiscono facilmente nello spam (DIS ha persino un commento «Fake domain?» accanto al mittente). SR usa invece PHPMailer con SMTP autenticato e STARTTLS, leggendo le credenziali dai segreti d'ambiente.

```
// public/api/newsletter.php (SR) - SMTP autenticato via PHPMailer
$mail = new \PHPMailer\PHPMailer\PHPMailer(true);
$mail->isSMTP();
$mail->Host = $cfg['SMTP_HOST']; $mail->SMTPAuth = true;
$mail->Username = $cfg['SMTP_USER']; $mail->Password = $cfg['SMTP_PASS'];
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS; $mail->Port = $cfg['SMTP
↳ _PORT'];
$mail->setFrom($cfg['SMTP_USER'], $cfg['SMTP_FROM_NAME']);
```

C'è però una contraddizione interna: SR usa l'SMTP autenticato solo per la newsletter. Il form contatti, `contact.php`, è rimasto sulla `mail()` nativa. Lo stesso sito ha così due meccaniche di posta diverse: è facile pensare all'SMTP come a una scelta «per il futuro, a volumi maggiori», quando in realtà in SR è già in produzione, accanto alla `mail()` che non ha mai dismesso.

**

4. Il form che spara email a nome tuo

Arriva qui il difetto più istruttivo del capitolo, e nasce da una confusione comune tra due difese che sembrano simili e non lo sono.

Un **throttle** rallenta l'invio in uscita, per non sovraccaricare il mail server e non farsi mettere in greylisting: è una pausa ogni tot email. Un **rate-limit** limita le richieste in ingresso, per impedire a un estraneo di abusare di un endpoint: è un tetto di tentativi per IP. Sono difese ortogonali, su lati opposti del sistema. Quello che spesso viene chiamato «Rate Limiting» è in realtà un throttle:

```
// public/api/newsletter.php (SR) - questo è un THROTTLE in uscita, non un r
↳ ate-limit in ingresso
if ($count % 10 == 0) { usleep(500000); } // mezzo secondo ogni 10 email:
↳ protegge il mail server
```

Il throttle protegge il *tuo* server di posta. Non protegge da chi martella il form di iscrizione. E qui sta il buco di SR: la sua `subscribe` non ha alcun rate-limit. L'IP del richiedente viene perfino registrato, ma solo memorizzato, mai usato come limite, e per giunta letto da `X-Forwarded-For` grezzo (falsificabile, come al CAP 10). Chiunque può quindi mandare richieste di iscrizione con email arbitrarie, e a ognuna parte una vera email di conferma via SMTP verso un terzo che non ha chiesto nulla. È un vettore di mail-bombing che brucia la reputazione del dominio e consuma la quota SMTP.

SPW invece il vettore lo chiude, riusando per la newsletter la stessa tabella `login_attempts` del login (con un prefisso diverso sulla chiave, per non mischiare i contatori):

```
// public/api/subscribers.php (SPW) - rate-limit anti-mail-bombing che ricic
↳ la login_attempts
$rl_key = 'sub:' . substr(hash('sha256', $_SERVER['REMOTE_ADDR']) ?? 'unknown
↳ ', 0, 40);
$stmtRl = $pdo->prepare("SELECT COUNT(*) FROM login_attempts WHERE ip_addres
↳ s = ?");
$stmtRl->execute([$rl_key]);
if ((int)$stmtRl->fetchColumn() >= 3) { http_response_code(429); exit; } /
↳ / max 3 iscrizioni / 15 min
```

ATTENZIONE

Rate-limit in ingresso ≠ throttle in uscita SR ha il throttle ma non il rate-limit; SPW ha il rate-limit ma non il throttle. Hanno difese opposte, e quella che manca a SR è quella che conta di più sulla sicurezza: senza un tetto in ingresso, il form di iscrizione diventa un'arma che spara email a nome tuo verso chiunque. È il ribaltamento già visto al CAP 10: SR, il sito più ingegnerizzato, lascia aperto proprio il buco che SPW, più semplice, aveva chiuso. E la beffa è che l'infrastruttura per

limitare (la stessa `.cache/ratelimit/` del login) in SR esiste già: semplicemente non è stata riusata qui.

C'è poi un problema che accomuna tutti e tre, su una scala di rozzezza decrescente: l'invio è un `foreach` bloccante dentro la richiesta HTTP. Su una lista grande, la richiesta va in `max_execution_time` e la consegna si interrompe a metà, senza che nessuno lo sappia. SR è il meno peggio (ha il `throttle` e un `try/catch` per destinatario che conta gli errori); SPW conta solo il valore di ritorno di `mail()`; DIS lo ignora del tutto, e il contatore delle campagne registra i *tentativi*, non i successi. Nessuno dei tre ha una coda o un cron: è lo stesso anti-pattern del «lavoro pesante dentro la richiesta» visto con la conversione delle immagini al CAP 7.

5. Header injection dal campo nome

Un'ultima trappola, ed è in DIS, nel form contatti. Il nome inserito dall'utente viene ripulito con `strip_tags` prima di finire nel database, e questo va bene per il database. Ma quello stesso nome viene anche messo nell'oggetto dell'email di notifica all'admin, e lì `strip_tags` non basta:

```
// public/api/contact.php (DIS) - strip_tags toglie i tag, NON gli a-capo
$name = strip_tags($input['name'] ?? ''); // saniti
↳ zazione per il DB
$subject = "Nuovo Messaggio da $name - Disintelligenza"; // ...ma
↳ $name finisce nell'header Subject
$headers = "From: no-reply@...r\nReply-To: $email\r\n";
mail('runtimeradio@gmail.com', $subject, $body, $headers);
```

`strip_tags` rimuove i tag HTML, ma non i caratteri di a-capo `\r\n`. Un nome che li contenga può iniettare header aggiuntivi nell'email, per esempio un `Cc` o un `Bcc` verso indirizzi scelti dall'attaccante. L'email del mittente, che finisce nel `Reply-To`, è invece al sicuro perché passata da `FILTER_VALIDATE_EMAIL`: il vettore è il nome, l'unico campo testuale libero che entra in un header.

ATTENZIONE

Sanitizzare per il database non è sanitizzare per gli header email La sanitizzazione ha sempre un contesto. `strip_tags` neutralizza l'HTML, che è il pericolo quando il dato verrà mostrato in una pagina; ma quando lo stesso dato entra in un'intestazione di posta, il pericolo cambia forma, sono i `\r\n` a contare. La regola: per un header email, rimuovi o rifiuta i caratteri di controllo, e non mettere mai input dell'utente nelle intestazioni se puoi evitarlo. SR, non a caso, costruisce il `From` da un valore fisso, non dall'input.

6. Il filo dei quattro emettitori si chiude

Torniamo un'ultima volta alla tabella del CAP 8. La newsletter è la quarta casella, e in tutti e tre i siti la query d'invio seleziona titolo, riassunto, immagine e link, **mai il content**. L'email rimanda all'articolo con un «Leggi tutto», e il campo HTML grezzo difeso solo a render-time non viene mai toccato.

#	Emettitore	Cosa emette del content	Difesa	Esito
1	Render React (CAP 8)	content pieno	DOMPurify (SPW, SR) / niente (DIS)	choke-point reale; DIS scoperto
2	Prerender SEO (CAP 11)	content pieno	strip_tags allowlist (solo tag)	buco sugli attributi (SPW, SR)
3	Feed RSS (CAP 12)	niente / preview escapata	htmlspecialchars / strip_tags+escape	sicuro
4	Newsletter (questo capitolo)	niente (solo titolo, riassunto, intro)	htmlspecialchars (SR, DIS) / grezzo dietro Auth (SPW)	sicuro

Si potrebbe leggere la query senza `content` come una semplice «ottimizzazione di payload», per tenere le email leggere. È vero, ma è soprattutto la chiusura del filo: non emettere il campo grezzo è ciò che impedisce all'email di diventare un quinto vettore XSS.

C'è una simmetria curiosa tra i due flagship. In SR la newsletter è l'emettitore *più* sicuro dei quattro: non tocca il `content` e per giunta escapa ogni altro campo. In SPW è invece il *meno* sanitizzato, perché il testo introduttivo scritto dall'admin viene emesso grezzo, senza alcun `htmlspecialchars`: è sicuro solo perché chi lo scrive è autenticato, non perché ci sia una difesa. Due posizioni opposte sulla stessa scala.

IMPORTANTE

Il quadro completo: una sanitizzazione, quattro render-path Con la newsletter il filo dei quattro emettitori si chiude. Feed e newsletter non riaprono il buco, perché o non emettono il `content` o lo escapano. L'unica falla che resta viva in tutto il quadro è il prerender del CAP 11, con il suo `strip_tags` ad allowlist che lascia passare gli attributi. La conclusione, ripetuta da cinque capitoli, è sempre la stessa: la sanitizzazione del contenuto dovrebbe vivere una volta sola, lato server, condivisa da tutti gli emettitori, invece di essere reinventata (o dimenticata) da ognuno. Il fatto che il buco sia rimasto aperto in un solo punto su quattro non è merito dell'architettura: è fortuna, più la disciplina di chi ha scritto gli altri tre.

✱
✱ ✱

7. Il consenso, e dove sparisce

Resta il lato GDPR dell'iscrizione, che non è solo una questione di token ma di consenso. Sul form, SPW chiede il doppio assenso esplicito (trattamento dei dati e dichiarazione

di maggiore età); SR ne chiede uno solo, e ha una variante «minimal» del form, pensata per il footer, che non ha alcun checkbox. DIS, sul form, valida e basta.

Ma il caso più interessante è in DIS, e non passa nemmeno dal form. Un'email può entrare nella lista anche da una seconda porta: quando un partecipante al festival viene approvato, il suo indirizzo viene aggiunto agli iscritti con un `INSERT OR IGNORE`, **senza un consenso esplicito alla newsletter**. I commenti nel codice mostrano lo stesso sviluppatore in dubbio se sia corretto.

ATTENZIONE

Il consenso come effetto collaterale Iscrivere qualcuno a una lista perché ha fatto *un'altra* cosa (candidarsi a un festival) è una raccolta di consenso che il GDPR non considera valida: il consenso deve essere specifico per quella finalità. È una trappola facile, perché il codice che lo fa sembra innocuo: una riga di `INSERT` in coda all'approvazione. La regola: ogni porta d'ingresso a una lista email deve avere il suo consenso, esplicito e separato. Il trattamento completo del workflow del festival è ai capitoli che lo riguardano; qui basta la lezione.

**

In sintesi

La newsletter mostra una scala di semplificazione che è anche una scala di rischio. SPW è il gradino completo: double opt-in con due token distinti, rate-limit contro il mail-bombing, link di disiscrizione con segreto, consenso doppio. SR aggiunge il trasporto più serio, l'SMTP autenticato, ma toglie il rate-limit (e apre il vettore mail-bombing) e fonde i due token in uno solo. DIS toglie anche il double opt-in e ogni token, e lascia un'iniezione di header nel nome del form contatti, pur conservando due buone abitudini d'igiene (la validazione dell'email ovunque e la pulizia in scrittura). Sul `content`, però, tutti e tre fanno la cosa giusta: non lo emettono, e il filo dei quattro emettitori si chiude. La difesa che manca più spesso non è quella contro l'XSS, che qui è risolta per disciplina: è quella contro l'abuso del proprio stesso form.

IMPORTANTE

Il Canone

- Double opt-in con due token distinti (conferma monouso + disiscrizione stabile): il link di disiscrizione ha bisogno di un segreto, non basta l'email in chiaro.
- Rate-limit (tetto per IP) sull'iscrizione contro il mail-bombing; non confonderlo col throttle, che regola solo la cadenza d'invio.
- Sanitizza per gli header email (`i \r\n`), non solo per il DB: il nome è un vettore di header injection.
- Consenso GDPR esplicito; l'invio di massa, meglio se asincrono o in coda.

**

Prossimo Capitolo: Admin Dashboard & Panels. Il pannello di controllo che lega insieme i sistemi visti finora, e i tre modi molto diversi in cui i siti decidono cosa un amministratore può vedere e fare.

CAPITOLO 14: Admin Dashboard & Panels

Ogni sistema visto finora ha un retro. I contenuti del CAP 9, i media del CAP 7, la newsletter del CAP 13, le reazioni del CAP 20, il festival dei capitoli che seguono: tutti si affacciano, prima o poi, su un pannello da cui un amministratore li guarda e li governa. L'area admin è il tessuto che lega gli altri cluster, il punto in cui diventano numeri da leggere e azioni da compiere. È una superficie che il festival rende facile sottovalutare: l'unica «dashboard» a cui si pensa di solito è quella del concorso, che è invece un caso particolare (lo ritroviamo al CAP 19). Qui ci occupiamo dell'area amministrativa generale, di cui quella del festival è solo una specializzazione.

I tre siti costruiscono la loro console su due domande indipendenti. La prima è *come è fatta*: una struttura a rotte con una guardia dichiarativa, un singolo mega-componente, o una via di mezzo. La seconda è *quanto misura*: un cruscotto analitico con grafici, un menu che non conta nulla, o numeri veri scritti in testo. Le due domande non vanno a braccetto, e la loro combinazione dà tre admin molto diversi.

C'è un ribaltamento che vale la pena anticipare, perché è il filo conduttore. SitoRuntime è il sito delle cicatrici di scalabilità, quello che ha vissuto il crash notturno e la migrazione d'emergenza del CAP 15. Eppure è quello con l'admin meno attrezzato: non misura niente e non ha backup. Cioè non ha né l'occhio per accorgersi di un problema, né la rete per sopravvivergli. È la conferma, ancora una volta, che più ingegnerizzato non significa più protetto.

*
**

1. L'admin è un aggregatore, non un'applicazione

Prima delle differenze, quattro tratti che i tre siti condividono.

L'area riservata non è un'applicazione separata: è un telaio che monta dentro di sé i pannelli degli altri cluster. L'editor del CAP 8, la libreria media del CAP 7, il compositore newsletter del CAP 13, la gestione del festival vivono come pagine di una console comune. Quello che è proprio dell'admin sono il telaio, la guardia, e poche superfici sue: la dashboard, le impostazioni, la gestione utenti.

La guardia, appunto, copre tutta l'area in un punto solo, non pagina per pagina. È il rovescio del backend, dove ogni endpoint chiama il proprio `Auth::check()` (CAP 10): qui il frontend protegge l'intero sotto-albero riservato con un'unica barriera, e tutte le pagine figlie ne ereditano la protezione. È il principio «una guardia, N pagine».

Ogni pannello degrada con grazia per conto suo: se una fonte dati cade (le metriche assenti, una tabella non ancora migrata), quel pannello si nasconde o mostra zero, ma il resto della console resta usabile. La dashboard non è mai una pagina bianca. E in tutti e tre il cambio password admin vive dentro le impostazioni, spesso l'unica voce davvero «di configurazione» presente.

**

2. Come è costruita: tre architetture di guardia

Su questo asse, la scala va dal dichiarativo all'imperativo.

SPW usa un *route-guard* dichiarativo: la rotta che monta il layout admin ha un loader, e tutte le figlie ereditano il suo redirect. La sessione viene verificata prima che la pagina si monti; se manca, l'utente non vede nemmeno un lampo di contenuto riservato.

```
// SPW App.tsx:239-258 + loaders.ts:10-20 - una guardia, N pagine
{ element: <AdminLayout />, loader: adminAuthLoader, children: [ /* dashboar
  ↪ d, settings, ... */ ] }

export const adminAuthLoader = async () => {
  const session = await api.checkSession();
  if (!session || !session.user) return redirect('/admin/login'); // mai
  ↪ un dato admin servito senza sessione
  return session;
};
```

SR sta all'estremo opposto: un solo componente, *Admin.tsx*, quasi seicento righe, che cambia «sezione» in memoria invece di navigare tra rotte. La guardia è un controllo dentro il componente, eseguito al montaggio. DIS prende una via di mezzo che è quasi un ibrido curioso: ha la struttura di SPW (un *AdminLayout* con sidebar e rotte figlie via *outlet*), gira perfino sull'infrastruttura data-router che permetterebbe i loader, ma poi protegge l'area con un controllo dentro il componente, come SR.

```
// DIS AdminLayout.tsx:13-25 - struttura come SPW, guardia come SR
useEffect(() => {
  api.checkAuth().then(u => {
    if (!u) navigate('/admin/login'); // protezione client-side,
  ↪ nessun loader
    else setUser(u);
  });
}, [navigate]);
if (!user) return null; // NB: nessun controllo rol
↪ e === 'admin' (vedi sotto)
```

Il confronto pieno tra guardia-loader e guardia-componente, con i suoi compromessi, è al CAP 6. Qui conta una conseguenza di sicurezza che si vede proprio in DIS.

ATTENZIONE

Proteggere l'area non basta: serve il ruolo La guardia di DIS verifica che ci sia un utente loggato, non che sia un amministratore. Un *editor*, quindi, vede l'intera console: iscrizioni, voto, reset, gestione utenti. È il backend a dover respingere le azioni che l'editor non può compiere, ma lo fa in modo incoerente (CAP 10): alcuni endpoint sono riservati agli admin, altri accettano qualunque utente loggato. Il risultato è che un editor può davvero approvare partecipanti e cambiare i round dalla

UI. È la versione estesa all'intera area del problema «il gate nasconde il contenuto ma non il pulsante» che si vede anche in SR. La regola: nascondere una pagina è esperienza utente; impedire un'azione è sicurezza, e va fatta sul ruolo, sia sul client sia (soprattutto) sul server.

**
**

3. Quanto misura: analitica, console, testuale

L'altro asse è ortogonale al primo, e separa i tre siti in modo netto.

SPW ha un cruscotto analitico vero: sette card di totali, una decina di mini-statistiche con trend percentuale, sei grafici (con un selettore di periodo a 7, 30 o 90 giorni). SR non misura nulla: la sua dashboard è fatta di card che sono pulsanti di navigazione, non contatori. È un menu travestito da cruscotto. DIS sta nel mezzo, ed è la lezione più utile dei tre: misura davvero, ma in testo. Contatori di iscritti e voti, lo spazio su disco diviso per cartella, la classifica provvisoria, gli ultimi arrivati. Niente grafici, niente motore di tracciamento, solo i COUNT giusti.

CONSIGLIO

Quanto cruscotto ti serve davvero La dashboard di DIS dimostra che si può dare valore informativo a un amministratore senza Chart.js e senza un motore di analytics: bastano le query giuste e una pagina che le scrive in chiaro. Il cruscotto analitico di SPW è più ricco, ma costa un intero sistema di tracciamento da mantenere; il «menu» di SR è il gradino sotto al minimo, perché non risponde nemmeno alla domanda più semplice (« quanti iscritti ho oggi? »). Tra il troppo e il niente, il cruscotto testuale è spesso il punto giusto: misura ciò che serve decidere, e nient'altro.

**
**

4. Misurare senza terze parti

Quando un sito vuole sapere quante visite riceve senza affidare i dati dei propri lettori a Google Analytics, deve costruirsi un analytics in casa. SPW lo fa con `analytics.php`, un file a doppia personalità: il ramo che *registra* un evento è pubblico (lo chiama il sito live a ogni visita), il ramo che *legge* i report è riservato all'admin.

```
// SPW analytics.php:16-77 - un file, due pubblici
if ($method === 'POST') {
    // tracking 'view'/'click' dal sito live, anonimo: nessun Auth
    // 'view' deduplicata per IP-hash + articolo + giorno; 'click' a rate-li
    ↪ mit
}
elseif ($method === 'GET') {
    Auth::check(); // solo la reportistica è privata
    // ~20 aggregazioni per la dashboard
}
```

Due accortezze rendono questo tracking sano. Le visualizzazioni sono deduplicate per IP e giorno, così un lettore che ricarica dieci volte conta una volta sola: i numeri non si gonfiano. E i click, oltre la soglia di frequenza, ricevono una risposta neutra invece di un errore:

```
// SPW analytics.php:62 – oltre il limite, risposta neutra: il client non di
↳ stingue "registrato" da "scartato"
echo json_encode(['status' => 'ok']); // non un 429
```

È lo stesso analytics.php a consumare le reazioni del CAP 20, trasformandole in «articoli più amati» e «reazioni per tipo». Un sito che conta da sé i propri lettori tiene i dati in casa (un vantaggio di privacy concreto) e decide da sé cosa conta come visita vera. Il prezzo è un endpoint pubblico in più da difendere dall'abuso, ed è il motivo della deduplica e del rate-limit.

*
**

5. La rete di salvataggio: il backup, e il paradosso di SitoRun-time

Qui i tre siti si separano nel modo più istruttivo. SPW ha il pattern d'oro: il backup automatico viene scritto **fuori dalla document root**, dove nessuna richiesta web può raggiungerlo. E c'è un dettaglio che nasce da una cicatrice reale.

```
// SPW backup.php:192-213 – il backup va fuori dalla docroot; il fallback si
↳ difende a runtime
$outside = dirname(realpath(__DIR__ . '/../')) . '/db_backups_simonepizzi';
if (!is_dir($outside)) @mkdir($outside, 0700, true);
if (is_dir($outside) && is_writable($outside)) {
    $backup_dir = $outside;
} else {
    // Fallback dentro la docroot: ma clean-dist.js (postbuild) elimina .dat
↳ a/ dalla dist,
    // quindi l'.htaccess di deny committato nel repo NON arriva sul server
↳ → ricreolo a runtime
    $backup_dir = __DIR__ . '/.data/backups';
    if (!is_dir($backup_dir)) mkdir($backup_dir, 0700, true);
    @file_put_contents($backup_dir . '/.htaccess', "Require all denied\n");
}
$filename = "auto_backup_" . date('Y-m-d_H-i-s') . '_' . bin2hex(random_byte
↳ s(8)) . ".sql"; // nome non indovinabile
@chmod($backup_dir . '/' . $filename, 0600);
```

ATTENZIONE

Il build può tradire la tua difesa statica: difenditi a runtime Il .htaccess CON Require all denied che protegge la cartella dei backup è committato nel repo. Ma lo script di build (clean-dist.js) rimuove la cartella .data/ dalla distribuzione, per non spedire database di sviluppo in produzione. Effetto collaterale: il file di deny non arriva mai sul server. SPW lo ricrea a runtime, la prima volta che scrive un backup. La lezione è generale: una difesa che vive in un file statico vale solo se quel file arriva fin dove serve; se la tua pipeline di build lo tocca, la difesa va ristabilita a runtime. Conviene sempre chiedersi cosa il build *toglie*, non solo cosa aggiunge.

Il backup di SPW è anche schedulabile da un cron esterno senza login, ma protetto da un segreto confrontato in modo timing-safe, e *fail-closed*: se il segreto non è configurato, il ramo cron è raggiungibile solo da un admin loggato.

```
// SPW backup.php:156-166 – cron senza login, ma protetto; nega se il secret
↳ non è definito
if (!$is_admin && (empty($configured_secret) || !hash_equals($configured_sec
↳ ret, $secret))) {
    http_response_code(403); die("Accesso negato.");
}
```

SR, di tutto questo, non ha niente. Nessun backup, nessun export, nessun cron, e, come abbiamo visto al §3, nessuna metrica.

ATTENZIONE

Cura senza prevenzione: il sito che non si fa il backup È il paradosso al cuore di questo capitolo. SitoRuntime ha vissuto un crash notturno del database e una migrazione d'emergenza (CAP 15), e si porta dietro uno script di «revert d'emergenza» del WAL: ha la *cura*. Ma non ha la *prevenzione*: nessun backup automatico, nessuna metrica che lo avverta prima che le cose peggiorino. Il sito mappato proprio per i suoi incidenti è quello meno attrezzato a vederli arrivare e a rimediare. Avere lo script di emergenza ma non il backup è come tenere l'estintore e non l'allarme antincendio.

✱
✱

6. Le azioni potenti, e come (non) sono protette

Un'area admin contiene le leve più pericolose del sito, e i tre siti le proteggono in modo diseguale.

SR ha quella che si può chiamare una **console nascosta**: dentro il suo `admin.php` vivono azioni potenti, come un `ALTER TABLE` o la riconversione di tutte le immagini, raggiungibili via `GET` digitando l'URL. Sono protette dal solo login, non dal ruolo, non hanno protezione CSRF (sono `GET`), e soprattutto nessun pulsante della UI le invoca: chi non conosce il nome dell'azione non sa nemmeno che esistono. È manutenzione fatta a mano, senza interfaccia e senza confine di ruolo (la meccanica di quelle migrazioni è al CAP 15).

DIS espone i reset distruttivi in un pannello, e li circonda di conferme. Ma le conferme sono solo esperienza utente:

```
// DIS Settings.tsx:155-163 – doppio confirm CLIENT, fetch POST SENZA token
↳ CSRF
if (confirm('⚠ RESET EDIZIONE: cancellerà TUTTI i partecipanti/voti/audio...'))
↳ ) {
    if (confirm('ULTIMA CONFERMA: i dati saranno persi per sempre. Procedere?'))
↳ ) {
    await fetch('/api/reset_system.php', { method: 'POST',
        body: new URLSearchParams({ action: 'confirm_reset' }) }); // nessun
↳ CSRF
    }
}
```


ATTENZIONE

Il `confirm()` non è una difesa di sicurezza Un doppio `window.confirm` riduce l'errore umano: è utile contro il clic distratto. Ma non ferma una richiesta forgiata da un altro sito mentre l'admin è loggato, perché quella richiesta non passa dal dialogo del browser. La difesa contro quel vettore è il token CSRF (CAP 10), che qui manca. Confondere la conferma con la protezione è un errore comune: la prima parla all'utente onesto, la seconda all'attaccante. Servono entrambe, e fanno cose diverse.

SPW è il più disciplinato anche qui, ma non immune da noi. Il suo cambio password incrementa il `session_version` lato server, invalidando le altre sessioni aperte (CAP 10); però il salvataggio delle impostazioni accetta qualunque chiave, senza una lista di quelle ammesse. È un rischio basso, perché tutto è dietro la guardia admin, ma è il genere di porta che conviene chiudere prima che qualcuno ci appoggi qualcosa di sensibile.

7. Dati senza consumatore: la tabella che nessuno legge

Un difetto più sottile, e tutto di DIS, riguarda i messaggi del form contatti. Vengono salvati nella tabella `contacts`, e poi non li legge nessuno: non esiste un pannello che li mostri, né un endpoint che li recuperi. L'unico modo in cui l'admin «vede» un messaggio è l'email di notifica che parte al momento dell'invio; la copia nel database resta lì, scritta e mai consultata.

ATTENZIONE

Raccogliere e dimenticare Persistere un dato senza avere un posto dove leggerlo è un costo nascosto, e per i dati personali è anche un rischio. Quella tabella accumula nomi, email e messaggi (con l'indirizzo IP) senza uno scopo applicativo e, soprattutto, senza un punto dove cancellarli quando andrebbero cancellati. La regola è semplice e spesso ignorata: se salvi un dato, devi avere un consumatore che lo usa e un modo per dismetterlo. Una tabella di sola scrittura non è un archivio, è un debito.

8. Il festival è un caso particolare di questo capitolo

Tutto quello che abbiamo visto (il telaio con la guardia, il cruscotto, le impostazioni come interruttori, la valutazione delle candidature) ha una specializzazione nel modulo festival, che ha pannelli suoi (l'ascolto delle tracce dei partecipanti, i master switch di registrazione e voto, la classifica). Quella dashboard è il CAP 19, e va letta come l'istanza-festival del pattern generale descritto qui: la struttura, la guardia e il backup appartengono a questo capitolo; i master switch e i KPI del concorso restano là.

In sintesi

L'area admin si misura su due assi indipendenti, e i tre siti li occupano in modo rivelatore. SPW è alto su entrambi: cruscotto analitico e architettura dichiarativa, con il backup fuori docroot a fare da rete. SR è basso su entrambi: un mega-componente che non misura nulla e non salva nulla, il paradosso del sito che ha sofferto di più senza essersi attrezzato per vederlo arrivare. DIS sta nel mezzo in modo istruttivo: la struttura del primo, la guardia (e i buchi) del secondo, e un cruscotto testuale che dimostra quanto si può misurare con poco. La lezione del capitolo non è «aggiungi più grafici»: è che un buon admin ti fa *vedere* lo stato del sistema e ti dà la *rete* per quando qualcosa va storto. L'apparato conta meno della domanda a cui risponde.

IMPORTANTE Il Canone

- Una sola guardia per l'intera area riservata (route-guard o controllo al montaggio), con verifica di **ruolo**, non solo di login.
- Backup automatico fuori docroot, con cron protetto da un segreto timing-safe e fail-closed; ricrea a runtime le difese che il build strippa.
- Le azioni potenti passano per POST + token CSRF, non per GET nascoste; nessuna tabella di sola scrittura (un dato salvato ha un consumatore e un modo per cancellarlo).
- Misura ciò che serve a decidere: anche un cruscotto testuale è abbastanza.

**

Prossimo Capitolo: Database Evolution, da SQLite a MySQL. La notte di febbraio in cui un database è caduto, e la migrazione d'emergenza raccontata ora per ora.

P A R T E V

I Casi Reali

Dove la teoria incontra la produzione. Pattern estratti da progetti reali, con le loro cicatrici.

CAPITOLO 15: Database Evolution - Da SQLite a MySQL

Il 23 febbraio 2026, alle 2:25 di notte, un commit di SitoRuntime porta un messaggio asciutto: «Risolto crash server e ottimizzate performance con switch SQLite WAL->DELETE». Il giorno dopo, il 24, il sito abbandona SQLite e passa a MySQL. Le due date, a un giorno di distanza, raccontano da sole la storia di questo capitolo: la migrazione non è stata un upgrade pianificato, è stata una fuga.

È la differenza che cambia tutto. Si potrebbe descrivere il passaggio da SQLite a MySQL come una soglia di crescita: più traffico, più scritture, più contesa sul file, e a un certo punto si cambia motore. È una linea guida ragionevole, e più avanti la diamo anche noi. Ma non è la storia vera di SitoRuntime, e raccontarla così farebbe perdere la lezione più preziosa. SitoRuntime è migrato perché si era fatto male, e questo capitolo non parla di «come si migra» ma di **cosa una migrazione lascia indietro** e di chi è attrezzato a sopravvivere alla prossima notte storta.

I tre siti del Modello hanno tre rapporti diversi con la stessa tecnica. DISINTELLIGENZA gira **ancora oggi su SQLite, in produzione, senza essere mai caduta**: è la prova vivente che il problema non era il motore. SimonePizziWebSite è migrato a MySQL in silenzio, senza un incidente in git, e con una rete di salvataggio sotto. SitoRuntime è migrato in fuga, di notte, e si è lasciato dietro sei fossili e nessun backup. È lo stesso ribaltamento che attraversa tutto il libro: il sito che ha sofferto di più è quello meno preparato a soffrire di nuovo.

*
**

1. La notte del WAL: quando l'ottimizzazione è il disastro

Prima della migrazione, SitoRuntime era su SQLite e voleva andare più veloce. SQLite, di suo, può usare un journaling chiamato **WAL** (Write-Ahead Logging): più rapido in scrittura, perché accoda le modifiche in un file collaterale invece di riscrivere subito il database. Sulla carta è un'ottimizzazione. In pratica, su un hosting condiviso Apache/PHP, il WAL crea due file di servizio (`.sqlite-wal` e `.sqlite-shm`) e ha bisogno di un locking che molti hosting condivisi non gestiscono bene. I lock non si rilasciano, le richieste vanno in timeout, il sito cade.

La cura vive dentro il codice che la spiega, ed è doppia. Uno script forza il ritorno al journaling classico:

```
// SitoRuntime optimize_db.php:17-22 - l'"ottimizzazione" che ha innescato i
↳ 1 crash, è codice SQLite
// WAL è performante ma spesso causa blocchi su hosting condivisi.
// Impostiamo DELETE per massima stabilità.
$pdo->exec("PRAGMA journal_mode=DELETE;"); // ← PRAGMA: direttiva solo SQL
↳ ite
```

E un secondo script è il pulsante rosso, da premere dal browser se il sito è già in timeout:

```
// SitoRuntime emergency_revert_wal.php:19-43 - il kit di pronto soccorso
$result = $pdo->query("PRAGMA journal_mode=DELETE"); // rimuove i file .wa
↳ l/.shm
$mode = $result->fetchColumn();
if (strtoupper($mode) === 'DELETE') echo "SUCCESSO: ..ripristinato..";
$pdo->exec("VACUUM;"); // ricostruzione del
↳ file DB
// nel catch: "potresti dover eliminare manualmente i file .wal e .shm via F
↳ TP"
```

ATTENZIONE

Quando l'ottimizzazione è la causa del disastro Il WAL non è un errore: in molti contesti è la scelta giusta. Ma su hosting condiviso, dove il filesystem e il locking sono fuori dal tuo controllo, toccare il `journal_mode` di un database a file è un cambiamento ad alto rischio mascherato da miglitoria. È il motivo per cui il Capitolo 3 prescrive `journal_mode=DELETE` e non WAL: non è una preferenza estetica, è una lezione pagata in produzione, di notte. La regola generale: un'ottimizzazione che cambia il modo in cui il motore scrive su disco va trattata come una modifica rischiosa, con un backup prima e un piano di rientro pronto, non come un interruttore da premere e dimenticare.

La diagnosi corretta non è «SQLite non regge». È «toccare il WAL su hosting condiviso non regge». La prova arriva dal sito accanto.

*
**

2. DISINTELLIGENZA: SQLite vivo, in produzione, senza cicatrici

Mentre SitoRuntime fuggiva, DISINTELLIGENZA restava. Gira oggi su SQLite, con il database in un file dentro `.data/`, sullo stesso tipo di hosting condiviso, e non è mai caduta. Tutto ciò che in SitoRuntime è un fossile inerte (`PRAGMA`, `sqlite_master`, `AUTOINCREMENT`, il database a file) qui è il motore reale e corrente. La differenza non è il motore: è che DISINTELLIGENZA non ha mai toccato il journal mode per spremere prestazioni che non le servivano, e fa un backup `.bak` prima di ogni operazione distruttiva (CAP 10).

Questo qualifica la regola pratica, invece di smentirla. SQLite resta perfetto per i progetti leggeri o in sviluppo: zero configurazione, un file solo, deploy in pochi secondi. La soglia oltre cui conviene valutare MySQL esiste davvero, e si riconosce da segnali concreti più che da un numero:

- **scritture concorrenti frequenti:** più utenti che scrivono nello stesso istante significano più contesa sul lock del file;

- **hosting che limita o blocca SQLite** per policy interne;
- **bisogno di accesso esterno al database**, per esempio da phpMyAdmin o da strumenti di gestione visuale, che parlano con un server, non con un file;
- **query complesse su tabelle grandi**, dove MySQL gestisce join e aggregazioni meglio.

Come linea di massima, finché un sito sta sotto le poche decine di scritture all'ora e non mostra lock ricorrenti, SQLite gli basta. DISINTELLIGENZA è in quella fascia e ci resta bene. SitoRuntime ne è uscito non perché avesse superato la soglia di traffico, ma perché un'ottimizzazione sbagliata gli aveva fatto venire voglia di non avere più un database a file. Sono due strade diverse verso MySQL, e solo una delle due è quella dei manuali.

**
**

3. Far evolvere uno schema senza strumenti di migrazione

C'è un tratto che i tre siti condividono, ed è il vero contesto di tutto il capitolo: **nessuno di loro ha uno strumento di migrazione**. Niente cartella migrations/, niente tabella schema_version, niente runner che tenga il conto di cosa è già stato applicato. Lo schema evolve con un solo attrezzo, lo stesso ovunque.

L'attrezzo è ALTER TABLE ... ADD COLUMN dentro un try/catch, reso idempotente. Si aggiunge la colonna, e se l'errore dice che esiste già la si tratta come successo. Su MySQL il segnale è il codice Duplicate column:

```
// SitoRuntime migrate_status.php:6-16 - la micro-migrazione che si auto-rip
↪ ara
$pdo->exec("ALTER TABLE news ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT
↪ 'published'");
// ...
} catch (PDOException $e) {
    if ($e->getCode() == '42S21' || strpos($e->getMessage(), 'Duplicate colu
↪ mn') !== false) {
        echo "OK: colonna 'status' già presente.\n"; // ri-esecuzione = ne
↪ ssun danno
    } else { echo "ERRORE: " . $e->getMessage() . "\n"; }
}
```

Su SQLite la stessa filosofia cambia dialetto: si interroga PRAGMA table_info e si aggiunge la colonna solo se manca.

```
// DISINTELLIGENZA update_db_v0.4.2.php:10-17 - stesso pattern, dialetto SQL
↪ ite
$columns = $pdo->query("PRAGMA table_info(users)")->fetchAll(PDO::FETCH_COLU
↪ MN, 1);
if (!in_array('role', $columns)) {
    $pdo->exec("ALTER TABLE users ADD COLUMN role TEXT DEFAULT 'editor'");
    $logs[] = "Added 'role' column to users table.";
} else {
    $logs[] = "'role' column already exists in users table.";
}
```

Il prezzo di questa semplicità è che **nessun file rappresenta lo schema corrente**. La versione è implicita, scritta nei nomi dei file: DISINTELLIGENZA ha una catena update_db_0_1_3 → 0_1_4 → v0.4.2 → v0.5.4, SitoRuntime ha apply_v291 e apply_v293. Per sapere

com'è fatta davvero una tabella oggi, non basta leggere un file: bisogna interrogare il database, oppure ricostruire la storia dai nomi e dall'ordine in cui sono stati eseguiti.

ATTENZIONE

Senza un registro, la verità dello schema vive solo nel database Versionare lo schema con i nomi dei file funziona finché c'è una persona sola che ricorda l'ordine. Ma non esiste un percorso pulito per ricreare il database da zero: in SitoRuntime servirebbe `init_mysql.php` più la catena `apply_v29x` applicata nell'ordine giusto, e l'`init_db.php` di partenza è un fossile fermo a una versione vecchia. È il debito «schema-as-code» del thin stack: il prezzo da pagare per non aver adottato un sistema di migrazioni. Se il progetto cresce, una tabella `schema_version` con un elenco di migrazioni applicate costa poco e ripaga al primo disaster recovery.

A questo si aggiunge una doppiezza di consegna. La stessa migrazione spesso esiste in due forme: come file usa-e-getta da caricare e cancellare, e come azione persistente dentro la console admin. La colonna `status` di SitoRuntime, per esempio, vive sia in `migrate_status.php` sia come azione `?action=apply_v291_status` dentro `admin.php`:

```
// SitoRuntime admin.php:459-469 - la stessa migrazione, ma self-healing die
↳ tro il login
if ($action === 'apply_v291_status' && $_SERVER['REQUEST_METHOD'] === 'GET')
↳ {
    try {
        getDB()->exec("ALTER TABLE news ADD COLUMN status VARCHAR(20) NOT NU
↳ LL DEFAULT 'published'");
        sendSuccess(['message' => "OK: colonna 'status' aggiunta..."]);
    } catch (PDOException $e) {
        if (strpos($e->getMessage(), 'Duplicate column') !== false) sendSucc
↳ ess(["/* ... */"]);
        else sendError('Errore DB: ' . $e->getMessage(), 500);
    }
}
```

Due filosofie convivono: lo script da FTP-are e poi rimuovere, e la console di manutenzione dietro login. Sono entrambe legittime, ma senza un registro nessuna delle due ti dice, guardandola, se quella migrazione è già passata su un certo database. Sono GET protette dal solo `isLoggedIn`, non dal ruolo né da un token CSRF: la loro innocuità dipende dall'essere idempotenti, non da una vera barriera (è lo stesso buco di gate visto al CAP 14).

*
**

4. Il trasloco di motore: il pattern a tre script

Quando la migrazione di motore è stata decisa, SitoRuntime l'ha eseguita con tre script dedicati, ciascuno con un ruolo preciso. È un ETL fatto a mano, senza strumenti, ed è un pattern positivo e citabile.

Il primo è il file dei segreti, tenuto fuori dal version control:

```
// db_credentials.php - da aggiungere SUBITO al .gitignore, mai committare c
↳ redenziali reali
<?php
return [
```

```

'DB_HOST' => 'mysql.tuohoster.com',
'DB_NAME' => 'nome_database',
'DB_USER' => 'utente_mysql',
'DB_PASS' => 'password_sicura',
'DB_PORT' => 3306,
];

```

Il secondo è il connettore MySQL, che sostituisce la versione SQLite di db.php:

```

// db.php (versione MySQL) - il singleton di connessione
$config = require __DIR__ . '/db_credentials.php';
$dsn = sprintf("mysql:host=%s;dbname=%s;port=%d;charset=utf8mb4",
    $config['DB_HOST'], $config['DB_NAME'], $config['DB_PORT']);
self::$pdo = new PDO($dsn, $config['DB_USER'], $config['DB_PASS'], [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false, // prepared statement nativi: s
    ↪ u SQLite era il default, qui va forzato
    PDO::ATTR_TIMEOUT => 5, // SQLite leggeva un file loca
    ↪ le; MySQL è in rete, il timeout serve
    PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8mb4 COLLATE utf8mb4_unico
    ↪ de_ci"
]);

```

Tre dettagli del passaggio meritano attenzione. `EMULATE_PREPARES => false` attiva i prepared statement nativi, che su SQLite erano impliciti. Il `charset=utf8mb4` nel DSN più il `SET NAMES` garantiscono che emoji e accenti sopravvivano. E il `PDO::ATTR_TIMEOUT` compare per la prima volta: con un file locale non serviva, con un server in rete è essenziale. Spariscono invece tutte le direttive `PRAGMA`: `journal_mode`, `busy_timeout`, `foreign_keys` erano linguaggio SQLite e su MySQL non hanno senso.

Il terzo script, `init_mysql.php`, ricrea lo schema da zero in dialetto MySQL, va eseguito una volta sola, protetto da autenticazione e poi rimosso. E il quarto, il vero trasloco, è l'ETL: apre due connessioni, copia tabella per tabella in modo idempotente, e chiude con una verifica esplicita.

```

// SitoRuntime migrate_to_mysql.php:37-72,199-209 - due PDO, copia idempoten
↪ te, verifica conteggi
$sqlite = new PDO("sqlite:" . $sqlitePath); // sorgente: il file
$mysql = Database::connect(); // destinazione: MySQL
$rows = $sqlite->query("SELECT * FROM news ORDER BY id ASC")->fetchAll();
$stmt = $mysql->prepare("INSERT INTO news (id, slug, title, summary, content
↪ , ...)
VALUES (?, ?, ?, ?, ?, ...)
ON DUPLICATE KEY UPDATE title=VALUES(title), ...");
↪ // ri-eseguibile
foreach ($rows as $r) { $stmt->execute([$r['id'], $r['slug'], ...]); $count++;
↪ }
// alla fine, la prova del nove tabella per tabella:
foreach (['news', 'users', 'subscribers', 'speakers', 'podcasts'] as $t) {
    $c = $mysql->query("SELECT COUNT(*) FROM $t")->fetchColumn();
    echo " $t: $c record in MySQL\n";
}

```

CONSIGLIO

L'ETL del thin stack: due PDO e un COUNT Per spostare i dati tra due motori non serve uno strumento dedicato: bastano due connessioni PDO, un loop che copia con `ON DUPLICATE KEY UPDATE` (così la copia si può ripetere senza creare doppioni) e un `COUNT(*)` finale che conferma quanti record sono arrivati a destinazione. È la

versione minimale, leggibile e verificabile di una migrazione di dati. Lo script poi va caricato sul server solo per il tempo necessario e cancellato insieme al file `.sqlite`: restare in giro è un rischio, non una comodità.

5. Cosa cambia, riga per riga, tra i due motori

Il passaggio tocca decine di micro-decisioni. La tabella le riassume.

Aspetto	SQLite	MySQL
Connessione	file locale	rete (host:port)
Direttive di motore	PRAGMA journal_mode, busy_timeout, foreign_keys	non applicabili
Charset	configurabile per file	utf8mb4 per emoji e accenti
Auto-increment	INTEGER PRIMARY KEY AUTOINCREMENT	INT AUTO_INCREMENT PRIMARY KEY
Boolean	INTEGER (0/1)	TINYINT(1)
JSON	salvato come TEXT	tipo nativo JSON
Prepared statement	nativi di default	nativi con EMULATE_PREPARES=false
Scritture concorrenti	lock sul file (problematico)	locking a livello di riga, gestito dal demone
Backup	copia del file <code>.sqlite</code>	mysqldump o strumenti dedicati

L'ultima riga è quella che fa più male a SitoRuntime, e ci arriviamo tra poco.

6. I sei fossili: l'igiene del repo dopo una migrazione

Dopo lo switch, lo strato SQLite non è stato rimosso. Restano in un repository MySQL **sei file scritti per un motore che non esiste più**: inerti nel migliore dei casi, rotti nel peggiore.

Fossile	Meccanismo SQLite	Stato su MySQL
init_db.php	AUTOINCREMENT, seed dei 24 speaker	crea tipi sbagliati, parzialmente rotto
fix_users_table.php	sqlite_master, PRAGMA, datetime('now')	rotto (PRAGMA dà errore di sintassi)
fix_newsletter_table.php	sqlite_master, PRAGMA, schema a 4 colonne	rotto e obsoleto
setup_podcasts.php	AUTOINCREMENT	parzialmente innocuo
optimize_db.php	PRAGMA journal_mode, CREATE INDEX IF NOT EXISTS	rotto
emergency_revert_wal.php	PRAGMA, VACUUM	inerte (il WAL non esiste su MySQL)

Il fossile più velenoso è il primo. `fix_users_table.php` interroga lo schema con dialetto SQLite, su un database che ormai è MySQL:

```
// SitoRuntime fix_users_table.php:12,20 - codice che parla la lingua del mo
↳ tore sbagliato
$stmt = $pdo->query("SELECT name FROM sqlite_master WHERE type='table' AND n
↳ ame='users'"); // SQLite!
$stmt = $pdo->query("PRAGMA table_info(users)"); // su MySQL → errore
↳ di sintassi
$pdo->exec("UPDATE users SET created_at = datetime('now') WHERE created_at I
↳ S NULL"); // datetime() = SQLite
```

L'unica rete che li tiene a freno è il deny by-prefix di `.htaccess`, che blocca l'esecuzione via HTTP di tutti gli script con un nome di manutenzione:

```
# SitoRuntime public/.htaccess:32-35 - l'unico argine ai fossili
RewriteRule ^(debug_|test_|emergency_|migrate_|fix_|init_|rebuild_|setup_|op
↳ timize_) - [F,L]
```

ATTENZIONE

Una migrazione di motore va completata anche cancellando Quegli script sono ancora sul server e ancora nel repo. Producono tre danni: rumore (chi apre il progetto non sa quali file contano), confusione sulla verità dello schema, e una superficie d'attacco latente, perché se un giorno il deny dell'.htaccess saltasse, sarebbero endpoint potenti raggiungibili. Migrare di motore non è finito quando i dati sono passati: è finito quando il codice del motore vecchio è stato rimosso. Tenere i fossili «per sicurezza» è il contrario della sicurezza.

✱
✱✱

7. Una tabella, tre CREATE diversi

Il debito «nessun file rappresenta lo schema» ha un volto concreto nella tabella `subscribers` di SitoRuntime, che nel codice esiste in **tre definizioni divergenti**:

1. `init_mysql.php` la crea con quattro colonne, lo schema base, precedente al double opt-in.

2. `fix_newsletter_table.php`, il fossile SQLite, ricrea le stesse quattro colonne, ed è rotto su MySQL e comunque insufficiente.
3. `apply_v293_newsletter`, dentro `admin.php`, è la **vera** migrazione: aggiunge le colonne del double opt-in (`confirmation_token`, `confirmed_at`, `subscribed_at`, `subscribed_ip`) e conferma retroattivamente gli iscritti storici.

Solo la terza è allineata al runtime. Il codice della newsletter (CAP 13) **presuppone** lo schema esteso, quindi su un database che ha solo lo schema base ogni query fallisce finché `apply_v293` non gira. È una dipendenza d'ordine non dichiarata: lo schema giusto esiste, ma solo se hai eseguito la migrazione giusta, che nessun file ti dice di eseguire.

ATTENZIONE

Dove vive la verità di una tabella Quando la stessa tabella ha tre `CREATE` sparsi in tre file, e solo uno è quello buono, la fonte di verità non è il codice ma il database in esecuzione. È un sintomo dello schema versionato per nomi-file: la definizione corretta esiste, ma è nascosta nella catena delle migrazioni invece che in un punto solo. Lo si paga il giorno in cui qualcuno ricrea il database dal file sbagliato e tutto sembra a posto finché il runtime non comincia a fallire.

*
**

8. Il bug della data come stringa

Le migrazioni lasciano cicatrici anche nei dati, non solo nello schema. SitoRuntime conserva `debug_time.php`, la diagnostica di un incidente di fuso orario. La regola di visibilità degli articoli confronta `published_at` <= adesso **come stringhe**: se il fuso del server e il formato della data non combaciano, gli articoli pubblicati spariscono dal sito.

```
// SitoRuntime debug_time.php:23-24 - la "fix" che non è mai arrivata in pro  
↳ duzione  
// FIX: Use T separator to match DB  
$now = date('Y-m-d\TH:i:s'); // separatore 'T'
```

Il dettaglio rivelatore è che questa fix non è mai stata applicata al runtime. Il codice di produzione (`news.php`) usa lo spazio, non la `T`, perché lo spazio è il formato corretto del DATETIME di MySQL. La «soluzione» con la `T` è rimasta solo nel file di diagnostica, documento di un bug del passato e di una correzione che il sito reale, alla fine, non ha adottato.

CONSIGLIO

Confrontare date come stringhe è un bug in agguato Due date confrontate come testo coincidono solo se hanno esattamente lo stesso formato e lo stesso fuso. Basta una `T` al posto di uno spazio, o un server impostato su UTC mentre i contenuti sono scritti in ora locale, perché un articolo «pubblicato» resti invisibile. La difesa è forzare il fuso in modo esplicito (`date_default_timezone_set('Europe/Rome')`) e confrontare valori nello stesso formato del tipo data del database, oppure delegare il confronto al database con `NOW()`.

*
**

9. La cura senza la prevenzione

Resta la riga più dolorosa della tabella del §5: il backup. SitoRuntime ha il defibrillatore, l'`emergency_revert_wal.php`, lo script che rianima il sito dopo l'incidente. Ma non ha il check-up: nessun backup automatico, nessun cron, nessun `mysqldump` schedulato. L'unica rete è una cartella `_BACKUP_BEFORE_OPTIMIZATION/`, uno snapshot manuale del progetto committato nel repo prima dell'intervento rischioso. È un gesto, non un sistema, e per giunta è rumore versionato.

Il contrasto con gli altri due è netto. SimonePizziWebSite ha il backup automatico scritto fuori dalla document root, con rotazione e un cron protetto (CAP 14). DISINTELLIGENZA fa un `.bak` prima di ogni reset distruttivo (CAP 10). Se il MySQL di SitoRuntime si corrompe, non c'è un dump recente da cui ripartire.

ATTENZIONE

Avere il defibrillatore ma non l'allarme antincendio È il paradosso al cuore del capitolo. Il sito mappato proprio per i suoi incidenti è quello senza la rete più elementare: un backup. Ha imparato a *curare* il disastro, lo script di emergenza è la prova che il dolore è stato vero, ma non ha imparato a *prevenirlo*. Curare un'emergenza è reattivo: ti tiene in vita per questa volta. Un backup automatico è preventivo: ti tiene in vita per la prossima, quella che non hai previsto. La checklist qui sotto prescrive un backup, ed è il dover essere; SitoRuntime racconta cosa succede a saltarlo.

**

10. Checklist di migrazione

La sequenza, in ordine, per un passaggio SQLite → MySQL fatto bene. Il primo punto è anche quello che SitoRuntime non aveva: non è un dettaglio, è la rete.

- ☐ Fare un backup completo del file `.sqlite` di produzione, e archivarlo fuori dal repo.
- ☐ Creare il database MySQL sul provider di hosting.
- ☐ Caricare ed eseguire `init_mysql.php` per creare lo schema, poi rimuoverlo.
- ☐ Caricare il `.sqlite` e `migrate_to_mysql.php` sul server.
- ☐ Eseguire `migrate_to_mysql.php` e verificare i conteggi nel riepilogo.
- ☐ Sostituire `db.php` con la versione MySQL e caricare `db_credentials.php` con le credenziali reali.
- ☐ Aggiungere `db_credentials.php` al `.gitignore` (prima del primo commit del progetto).
- ☐ Testare tutte le API in produzione, comprese quelle che presuppongono migrazioni successive allo schema base.
- ☐ Eliminare dal server `migrate_to_mysql.php` e il file `.sqlite`.
- ☐ **Rimuovere dal repo i fossili del vecchio motore** (`init_db.php` SQLite, `fix_*`, `optimize_db.php`, `emergency_revert_wal.php`): la migrazione non è finita finché restano.
- ☐ Configurare un backup automatico del MySQL, fuori dalla document root: la cura non sostituisce la prevenzione.

*
**

In sintesi

La migrazione di motore non è un gradino di scaling che prima o poi tutti salgono: è un evento, e il *perché* conta quanto il *come*. SitoRuntime è scappato da SQLite dopo una notte storta, e ha portato MySQL con sé senza ripulire la casa, lasciandosi dietro sei fossili, tre versioni della stessa tabella e nessun backup. DISINTELLIGENZA dimostra che SQLite in produzione regge benissimo, finché non gli si chiede un'ottimizzazione che non sa dare sul suo terreno. SimonePizziWebSite ha fatto lo stesso viaggio in silenzio, con la rete sotto. La lezione non è «migra a MySQL» e nemmeno «resta su SQLite»: è che ogni evoluzione di schema, e ancora di più ogni cambio di motore, va completata fino in fondo, con un registro di cosa hai fatto e un backup di cosa avevi prima. Quello che una migrazione lascia indietro pesa più di quello che porta avanti.

IMPORTANTE Il Canone

- Migra di motore solo per un vincolo concreto (lock ricorrenti, hosting, accesso remoto), non per scaramanzia; fai il backup **prima**, fuori dal repo.
- ETL idempotente (ON DUPLICATE KEY UPDATE) con verifica dei conteggi a fine trasloco.
- A migrazione conclusa, rimuovi i fossili del vecchio motore dal repo.
- Una sola definizione per ogni tabella e un registro dello schema (`schema_version`); la cura d'emergenza non sostituisce la prevenzione, cioè il backup automatico.

*
**

Prossimo Capitolo: Portfolio & Projects Module. Il modulo universale per portfolio e showcase, con riordinamento drag-and-drop e visibilità a interruttore.

CAPITOLO 16: Portfolio & Projects Module

Il modulo Portfolio è un'entità distinta da News/Article, pensata per un sito personale, un'agenzia, uno showcase. Mappato su **SimonePizziWebSite**, porta pattern propri: visibilità granulare, ordinamento manuale, pulsanti d'azione multipli, gestione per categorie. È il riferimento per qualunque sito che debba esporre un catalogo di lavori, prodotti o progetti.

1. La Differenza con il Modulo News/Articles

Caratteristica	News/Articles	Projects/Portfolio
Identificatore URL	slug (testo parlante)	id (numerico)
Visibilità	status (draft/published)	is_visible (boolean)
Programmazione temporale	published_at	non prevista
Rich text body	sì (HTML)	opzionale (description breve)
Ordinamento	per data (automatico)	sort_order (manuale)
CTA	nessuna	button_a + button_b (URL esterni)
Categorizzazione	category + tag	solo category

La riga sull'identificatore conta più di quanto sembri: gli articoli vivono su URL parlanti (slug), i progetti su un id numerico. I progetti, quindi, non generano slug, e la logica di slug avanzata (con la mappa degli accenti italiani) vive una volta sola al Capitolo 5, dove serve agli articoli.

2. Schema Database

```
CREATE TABLE IF NOT EXISTS projects (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL DEFAULT 'Nuovo Progetto',
  description  TEXT DEFAULT '',
  category    TEXT NOT NULL DEFAULT 'progetti-software',
  cover_image TEXT DEFAULT '',
  button_a_label TEXT DEFAULT 'Scopri',
  button_a_url  TEXT DEFAULT '',
  button_b_label TEXT DEFAULT '',
  button_b_url  TEXT DEFAULT '',
  is_visible   INTEGER NOT NULL DEFAULT 1,    -- 1=visibile al pubblico, 0
↳ =nascosto
  sort_order  INTEGER NOT NULL DEFAULT 0,    -- ordinamento manuale per c
↳ ategoria
  created_at   DATETIME DEFAULT (datetime('now'))
);
```

```
CREATE INDEX IF NOT EXISTS idx_projects_category ON projects(category);
CREATE INDEX IF NOT EXISTS idx_projects_sort_order ON projects(sort_order ASC
↳ C);
```

3. L'API projects.php: tutti e cinque i verbi

Il modulo usa l'intera gamma dei metodi HTTP, e PATCH è quello che lo distingue: è il verbo giusto per le operazioni di visibilità e riordino, che cambiano un solo campo.

GET: lista con bypass admin. Il pubblico vede solo i visibili; l'admin vede tutto. Stesso pattern del Capitolo 9, qui su `is_visible` invece che su `status`.

```
$is_admin = isset($_SESSION['user_id']);
if (!$is_admin) $conditions[] = "is_visible = 1";          // il pubblico ve
↳ de solo i visibili
if ($category) { $conditions[] = "category = ?"; $params[] = $category; }
$query .= " ORDER BY category ASC, sort_order ASC, created_at ASC";
```

POST: creazione con auto-sort. Alla creazione, `sort_order` diventa `MAX(sort_order) + 1` nella stessa categoria, così il nuovo progetto compare in fondo alla sua lista.

```
$stmtMax = $pdo->prepare("SELECT COALESCE(MAX(sort_order), 0) FROM projects
↳ WHERE category = ?");
$stmtMax->execute([$category]);
$sort_order = (int)$stmtMax->fetchColumn() + 1;
```

PATCH: aggiornamenti parziali. Non invia l'intero oggetto, solo il campo che cambia: il toggle di visibilità, o il nuovo `sort_order` arrivato dal drag-to-sort del frontend.

```
if (isset($data['is_visible'])) {
    $pdo->prepare("UPDATE projects SET is_visible=? WHERE id=?")->execute([(
↳ int)$data['is_visible'], $id]);
}
if (isset($data['sort_order'])) {
    $pdo->prepare("UPDATE projects SET sort_order=? WHERE id=?")->execute([(
↳ int)$data['sort_order'], $id]);
}
```

La semantica HTTP è chiara: POST crea, PUT sostituisce l'intero oggetto, PATCH modifica un pezzo. Usare PATCH per il toggle e il riordino comunica l'intento meglio di un POST generico.

4. I Pulsanti CTA (button_a / button_b)

Ogni progetto può avere fino a due pulsanti verso risorse esterne: uno principale («Scopri», «Visita il sito», «Gioca ora») e uno secondario opzionale («GitHub», «Case Study», «App Store»).

```
{project.button_a_url && (
    <a href={project.button_a_url} target="_blank" rel="noopener noreferrer" c
↳ lassName="btn-primary">
        {project.button_a_label || 'Scopri'}
    </a>
)}
{project.button_b_url && (
    <a href={project.button_b_url} target="_blank" rel="noopener noreferrer" c
↳ lassName="btn-secondary">
        {project.button_b_label}
```

```
</a>
  } }
```

Il `rel="noopener noreferrer"` sui link `target="_blank"` è obbligatorio: impedisce alla pagina di destinazione di accedere alla `window.opener` di quella di partenza (il tabnapping).

4.1 Lo switch Web / Email

Una rifinitura introdotta nella gestione dei CTA (SimonePizziWebSite v1.7.x) è un toggle «tipo di link» nell'editor: spesso un pulsante non punta a un sito ma deve aprire il client di posta. Se l'autore sceglie «Email», l'editor antepone da solo `mailto:` alla stringa salvata, ignorando l'`https://`. È una piccola UX a prova di distrazione: il redattore non deve ricordarsi il protocollo giusto.

5. Ricerca Unificata: un Solo Endpoint per Articoli e Progetti

Articoli e progetti sono entità diverse, ma per chi cerca sul sito sono la stessa cosa: contenuti. SimonePizziWebSite lo riconosce con un endpoint di ricerca unico, `search.php`, che interroga entrambe le tabelle con un `LIKE` e marca ogni risultato con un campo `type` perché il frontend sappia come renderlo.

```
// SPW search.php - una query per famiglia, risultati uniti e marcati con `t
↳ ype`
$like = '%' . $q . '%';
$articles = $pdo->prepare("SELECT id, title AS name, slug, 'article' AS type
↳ FROM articles
                                WHERE status='published' AND (title LIKE ? OR con
↳ tent LIKE ?)");
$projects = $pdo->prepare("SELECT id, name, NULL AS slug, 'project' AS type
↳ FROM projects
                                WHERE is_visible=1 AND (name LIKE ? OR descriptio
↳ n LIKE ?)");
// ...si eseguono entrambe, si concatenano i risultati, il client smista per
↳ `type`
```

È una ricerca onesta nei suoi limiti: `LIKE '%q%'`, non un motore full-text. Per un portfolio o un blog personale è più che sufficiente, e il campo `type` evita di costruire due ricerche separate nel frontend. Questo endpoint vive a metà strada tra il ciclo di vita dei contenuti (Capitolo 9) e questo modulo: è il punto in cui le due entità tornano a parlare la stessa lingua.

6. Frontend React: i Componenti Chiave

- **PortfolioGrid.tsx**: la griglia pubblica, che filtra per categoria lato client, mostra le copertine con lazy loading, rende i due CTA in modo condizionale e i badge di categoria.
- **ProjectEditor.tsx** (admin): upload immagine via `MediaPicker` (Capitolo 8), le due coppie label+URL dei pulsanti, il toggle `is_visible`, la categoria da dropdown.
- **ProjectsList.tsx** (admin): drag-to-sort che invia un `PATCH` a ogni riposizionamento, toggle di visibilità con `PATCH` istantaneo (l'icona occhio aperto/chiuso), filtro per categoria.

7. Strategie di Categoria: da Statiche a DB-driven

Fino alla v1.6, le categorie erano stringhe fisse nel codice React (`PROJECT_CATEGORIES`). Un portfolio che cresce ha bisogno di più libertà, e dalla v1.7.10 le categorie diventano DB-

driven: una tabella `categories` interrogata dal frontend all'avvio (`GET /api/categories.php`), così l'admin può rinominarle o aggiungerne dal pannello senza una nuova build di Vite.

Il multi-tagging molti-a-molti, invece, resta una feature degli **articoli**, non dei progetti (che hanno una sola `category`): la sua trattazione, con il doppio binario verso il campo CSV legacy, è al Capitolo 9. Qui basta la lezione di disponibilità: spostare le categorie dal codice al database le rende modificabili a caldo, e questo per un catalogo editoriale è ciò che conta.

NOTA

Il pattern `auth_helper.php` Anche `projects.php`, come ogni endpoint protetto, si appoggia all'`auth_helper.php` che incapsula `session_start()`, gli header JSON e la classe `Auth` (il dettaglio è al Capitolo 5). Concentrare quelle chiamate in un solo include, invece di ripeterle in ogni file, riduce gli errori da «headers already sent».

IMPORTANTE

Il Canone

- Tabella `projects` con `sort_order`, `is_visible` e i campi CTA; endpoint con `PATCH` per toggle di visibilità e riordino.
- URL dei progetti su id numerico, non slug; categoria singola (il multi-tag M:N è degli articoli, Capitolo 9).
- Ricerca unificata con un campo `type` che smista i risultati lato client.

*
**

Prossimo Capitolo: Festival Logic, Iscrizioni e Workflow di Approvazione. Il ciclo di gestione dei concorrenti per DISINTELLIGENZA e FDCA.

CAPITOLO 17: Festival Logic - Iscrizioni e Workflow Approvazione

Una premessa di onestà, prima del modulo. Il «festival» non è parte del core del miniCMS: è un **modulo opzionale**, presente in **un solo sito su quattro** (DISINTELLIGENZA), che FDCA eredita immutato perché ne è un fork con il backend byte-identico. I tre capitoli che seguono (questo, le votazioni, la dashboard) raccontano un concorso a voto pubblico reale, con i suoi pregi e le sue crepe, non un componente standard da dare per scontato.

Questo primo capitolo copre la porta d'ingresso: come un partecipante si iscrive, come l'admin lo valuta, e i due punti in cui il modulo idealizzato diverge dal codice (l'iscrizione alla newsletter e il caricamento pubblico delle tracce).

1. Il Workflow del Partecipante

L'iscrizione segue una pipeline a tre stati, gestita non da una macchina a stati ma da una colonna `status` che l'admin fa avanzare.

- **pending**: stato iniziale. Il partecipante ha inviato i dati e la traccia, ma non è visibile sul sito.
- **approved**: validato. Riceve l'email di conferma, entra nel concorso ed (vedi §4) viene iscritto alla newsletter.
- **rejected**: scartato, con una notifica di cortesia.

C'è però un dettaglio di sicurezza che il workflow «pulito» nasconde: l'azione che cambia lo stato (`update_status`) è protetta solo da un controllo di sessione, non di ruolo. Significa che **anche un editor**, non solo un amministratore, può approvare o respingere i partecipanti.

ATTENZIONE

Il gate che confonde «loggato» con «admin» Approvare un partecipante è una decisione che pesa: lo fa entrare nel concorso, gli invia un'email a tuo nome, lo iscrive alla newsletter. Eppure il backend la concede a chiunque sia loggato, senza verificare che sia un amministratore. È lo stesso «gate role-blind» che attraversa più punti del sito (Capitolo 10): nascondere una voce di menu a un editor è esperienza utente, ma impedirgli un'azione è sicurezza, e va fatto sul ruolo, lato server. Qui non è fatto.

2. Le Email Transazionali

A ogni cambio di stato il backend invia un'email con un template HTML coerente col branding del festival: una conferma tecnica alla ricezione, e una comunicazione formale (positiva o negativa) all'esito. Il trasporto è `mail()` nativa, *fire-and-forget*: il record nel database è la fonte di verità, l'email è un canale best-effort (la meccanica completa, con i suoi limiti, è al Capitolo 13).

3. Gli Asset dei Partecipanti: l'Upload Pubblico che Apre una Porta

Le tracce audio caricate dai partecipanti finiscono in una cartella isolata (`uploads/audio/participants/`) con nome univoco, e l'admin le pre-ascolta nel Media Center prima di decidere. Fin qui la versione comoda. La versione vera è che quel caricamento, per abbassare l'attrito dell'iscrizione, **non richiede login**, ed è il fronte da cui parte la catena RCE descritta al Capitolo 7.

ATTENZIONE

L'upload pubblico delle tracce cambia tutte le regole Un form di iscrizione che accetta file senza autenticazione è una superficie d'attacco aperta a Internet. In DISINTELLIGENZA quattro debolezze si sommano: il caricamento è pubblico, la validazione si fida del Content-Type dichiarato dal browser, il nome del file conserva l'estensione, e la cartella non spegne PHP. Il risultato, verificato, è l'esecuzione di codice remoto (la catena completa, anello per anello, è al Capitolo 7). La lezione per chi costruisce un'iscrizione senza attrito: l'apertura al pubblico è una scelta di prodotto legittima, ma va pagata con le difese che quell'apertura richiede (validare i byte reali, neutralizzare il nome, spegnere PHP nella cartella), non con la loro assenza. FDCA, essendo un fork, ha ereditato la stessa porta aperta intatta.

4. L'Iscrizione alla Newsletter all'Approvazione

All'approvazione, l'indirizzo del partecipante viene inserito nella tabella della newsletter con un `INSERT OR IGNORE`. Il modulo idealizzato lo presenta come una «strategia di crescita» del database marketing, che garantirebbe una lista di soli utenti reali e validati. È il framing da rovesciare.

ATTENZIONE

Iscriversi a un concorso non è acconsentire al marketing Chi invia la propria traccia acconsente a partecipare al festival, non a ricevere una newsletter: sono due basi giuridiche diverse (Capitolo 13). Iscrivere d'ufficio i partecipanti approvati alla mailing list, senza un consenso esplicito e separato, è un problema di conformità GDPR, non un pregio di prodotto. I commenti nel sorgente tradiscono il dubbio dello stesso sviluppatore. La soluzione è semplice e va nella direzione opposta a quella «comoda»: un consenso marketing distinto, opt-in, raccolto al momento dell'iscrizione e registrato; l'approvazione al concorso non deve trascinarsi dietro l'iscrizione alla newsletter come effetto collaterale.

IMPORTANTE Il Canone

- Workflow pending → approved/rejected con master switch `registration_active`.
- L'upload pubblico delle tracce è il fronte della catena RCE: gatelo e validalo come ogni upload (Capitolo 7).
- L'approvazione è un'azione admin: gate per **ruolo**, non solo per login (niente gate role-blind).
- Iscrivere a un concorso non è consenso al marketing: niente sync della newsletter senza opt-in esplicito e separato.

Prossimo Capitolo: Festival Logic, Votazioni e Protezione Anti-Frode. Il voto pubblico, le difese che contano davvero e quelle solo cosmetiche.

CAPITOLO 18: Festival Logic - Votazioni e Protezione Anti-Frode

Il voto è il cuore del concorso, ed è anche il punto in cui un modulo apparentemente robusto rivela quali delle sue difese contano davvero e quali sono solo decorazione. Questo capitolo distingue le une dalle altre, e mette in chiaro due fragilità che il testo idealizzato taceva: la classifica può derivare in silenzio, e il reset non è protetto.

1. La Sessione di Voto

Un visitatore può esprimere da una a tre preferenze in una sola chiamata. Il backend valida che ogni partecipante votato sia davvero in gara (`status = 'approved'` e `in_current_round = 1`), poi registra il voto in una transazione che fa due cose insieme: inserisce la riga in `votes` e incrementa il contatore sul partecipante.

```
// ogni voto è una transazione: INSERT + incremento del contatore denormaliz
↳ zato
$pdo->beginTransaction();
$pdo->prepare("INSERT INTO votes (participant_id, ip, user_agent) VALUES (?,
↳ ?, ?)")
    ->execute([$pid, $ip, $ua]);
$pdo->prepare("UPDATE participants SET vote_count = vote_count + 1 WHERE id
↳ = ?")->execute([$pid]);
$pdo->commit();
```

Quel `vote_count` non è una comodità: è la **fonte di verità della classifica**. L'ordinamento dei partecipanti si fa per `vote_count`, non contando le righe di `votes`. È una scelta di performance ragionevole (la classifica diventa una `SELECT ... ORDER BY vote_count`), ma ha un prezzo che il §5 mette a fuoco.

2. Le Difese Anti-Abuso, in Ordine di Efficacia Reale

Il modulo presenta tre difese contro il voto multiplo. Il punto, però, è che non sono affatto equivalenti: una regge, una è cosmetica, una non difende nulla. Distinguerle conta, perché confonderle dà una falsa sicurezza.

- **La barriera reale: IP per ventiquattr'ore.** Il backend registra l'indirizzo IP del votante e rifiuta nuovi voti dallo stesso IP per le ventiquattr'ore successive. È l'unica difesa server-side che un attaccante non aggira dal proprio browser.
- **La difesa cosmetica: il cookie.** Dopo il voto viene impostato un cookie (`dis_voted`) con scadenza a trenta giorni. Vive sul client, quindi chiunque lo cancella, apre una finestra anonima o cambia browser e rivota. Migliora l'esperienza dell'utente onesto (gli ricorda che ha già votato), non ferma chi vuole barare.

- **Nessuna difesa, solo registro: lo User-Agent.** Ogni voto memorizza lo User-Agent, ma non viene usato per bloccare niente: serve solo a un'eventuale analisi a posteriori.

ATTENZIONE

Tre difese, una sola conta Elencare cookie, IP e User-Agent come se fossero tre lucchetti equivalenti è un errore comune e pericoloso, perché chi legge crede di avere una difesa a tre strati quando ne ha una sola. Il cookie e lo User-Agent sono sotto il controllo del client: il primo si cancella, il secondo si falsifica. La sola barriera che vive sul server, e che quindi può davvero limitare l'abuso, è quella basata sull'IP. Conoscere quale difesa regge davvero è ciò che evita di lasciare un concorso indifeso credendolo blindato.

Sull'IP c'è un'osservazione controintuitiva (Capitolo 10): il modulo usa il `REMOTE_ADDR` grezzo, non un helper che legge gli header di forwarding. Per un'autenticazione dietro proxy sarebbe un difetto, ma per un voto pubblico è un **pregio**: l'header `X-Forwarded-For` lo scrive il client e si falsifica, mentre il `REMOTE_ADDR` no. Il rovescio è la collisione NAT: dietro una stessa rete aziendale o universitaria, molti votanti legittimi condividono un IP e si bloccano a vicenda.

NOTA

Il contrappunto privacy: l'identità hashata delle reazioni Il voto del festival salva IP e User-Agent **in chiaro** nella tabella `votes`: dati personali persistiti senza offuscamento. Le reazioni agli articoli (Capitolo 20) affrontano lo stesso problema in modo più attento, derivando uno pseudonimo `SHA256(IP+UA)` invece di salvare l'IP nudo. Nessuno dei due è anonimato perfetto (quell'hash è reversibile, vedi Capitolo 20), ma è la differenza tra «ho offuscato il dato» e «ho conservato l'indirizzo di tutti i votanti in chiaro». Per un modulo che raccoglie voti dal pubblico, salvare l'IP grezzo è una scelta da pesare anche sul piano GDPR, non solo dell'anti-frode.

3. I Round a Interruttore

Un partecipante compare nella pagina di voto solo se è `approved` e `in_current_round = 1`. Le fasi del concorso (eliminatorie, semifinali, finale) non sono entità con una storia: sono questo flag, acceso o spento dall'admin su gruppi di partecipanti. È la semplicità del modulo, e anche il suo limite.

ATTENZIONE

reset_votes cancella la storia del turno Far avanzare il concorso significa azzerare i voti del turno e riaccendere il flag sul gruppo successivo. Ma `reset_votes` **cancella** i voti del turno precedente e riporta a zero il contatore: non esiste una storicizzazione per-turno, quindi i risultati delle eliminatorie spariscono, salvo la copia `.bak` che il sistema fa prima delle operazioni distruttive (Capitolo 10). Gestire le fasi con un solo flag booleano è comodo, ma se vuoi conservare i risultati di ogni turno devi archivarli prima del reset: il modulo, da solo, non lo fa.

4. Il Master Switch del Voto

La possibilità di votare è regolata da un interruttore globale (`voting_active`) nella tabella `settings`: se è spento, il backend rifiuta ogni voto con un 403. La tabella `settings` è leggibile pubblicamente (il frontend la consulta per mostrare o nascondere il form) e scrivibile solo dall'admin.

Un dettaglio rivela una piccola incoerenza interna: la lettura del flag accetta sia `'1'` sia `'true'`, perché un punto del codice salva i booleani come `'1'` e un altro come la stringa `'true'`. La lettura difensiva (`=== '1' || === 'true'`) compensa il fatto che la scrittura non ha mai fissato una convenzione. Funziona, ma è il sintomo da riconoscere: quando il lettore deve indovinare come ha scritto lo scrittore, manca un accordo a monte.

5. La Classifica che può Derivare

Resta la fragilità più sottile, e la più importante per l'equità del concorso. Siccome la classifica si ordina per `vote_count` (il contatore denormalizzato del §1) e non per il conteggio reale delle righe di `votes`, i due valori possono divergere. La transazione li tiene allineati nel funzionamento normale, e il reset li rialzera insieme, ma non esiste una **reconciliation**: nessun controllo periodico verifica che `vote_count` corrisponda davvero al numero di voti registrati.

ATTENZIONE

Denormalizzare un contatore: velocità contro verità Tenere il totale dei voti in una colonna rende la classifica istantanea, ma la rende anche fragile: basta una transazione interrotta a metà, un import manuale, una correzione fatta a mano sul database, e il contatore non corrisponde più ai voti reali. Il guaio è che la divergenza è **silenziosa**: la classifica mostra un ordine che sembra giusto, e nessuno si accorge che è sbagliato finché qualcuno non conta i voti a mano. Quando un numero denormalizzato decide un esito (una classifica, un premio), serve una query di riconciliazione che periodicamente confronti il contatore con `COUNT(votes)` e segnali gli scostamenti. Il modulo non ce l'ha, ed è la prima cosa da aggiungere prima di affidargli un concorso vero.

C'è infine il reset stesso: le azioni che azzerano voti e contatori (`reset_votes`, `reset_system`) sono potenti e distruttive, ma **non hanno protezione CSRF** (Capitolo 10). Le circonda solo una conferma nel browser, che ferma il clic distratto ma non una richiesta forgiata da un altro sito mentre l'admin è loggato. La copia `.bak` pre-distruttiva è la rete che attenua il danno; la difesa che manca è il token.

IMPORTANTE

Il Canone

- L'anti-frode reale è il vincolo IP + finestra temporale (24h); il cookie è cosmetico, lo User-Agent solo un indizio.
- Per il voto pubblico l'IP grezzo è un pregio anti-spoof (a differenza dell'auth dietro proxy, Capitolo 10).

- Se denormalizzi un contatore (`vote_count`), prevedi una riconciliazione periodica con `COUNT(votes)`, oppure accetti il drift silenzioso.
- I reset distruttivi passano per un token CSRF più la copia `.bak`; la conferma nel browser non basta.

**

Prossimo Capitolo: Festival Logic, Dashboard Admin, Settings e Reporting. Il pannello di controllo del concorso, e il report finale che non è mai stato acceso.

CAPITOLO 19: Festival Logic - Dashboard Admin, Settings e Reporting

Questa dashboard è il quadro di comando del concorso, ma non è una console a sé: è la **specializzazione festival** dell'area admin generale del Capitolo 14. La struttura che la regge (la guardia che protegge l'area, il layout, il backup fuori docroot) appartiene a quel capitolo; qui restano le cose proprie del festival, gli interruttori delle fasi, i KPI del concorso, l'approvazione, la classifica e il report. Leggere questo capitolo come l'istanza-festival di un pattern generale, e non come un pannello isolato, evita di duplicare ciò che il Capitolo 14 ha già spiegato.

E come gli altri due capitoli del modulo, anche questo allinea il testo idealizzato al codice reale, dove due funzioni promesse non sono ciò che sembrano: il report finale e i finalisti.

1. I Master Switch

La tabella `settings`, una coppia chiave/valore, è il quadro elettrico del festival: `registration_active` apre o chiude il form di iscrizione, `voting_active` la sessione di voto, `current_round` indica la fase. Un interruttore qui apre o chiude un'intera fase per tutti.

La tabella è leggibile pubblicamente (il frontend la consulta per sapere cosa mostrare) e scrivibile solo dall'admin. Un dettaglio già incontrato al Capitolo 18 torna qui: la lettura dei flag accetta sia `'1'` sia `'true'`, perché la scrittura non ha mai fissato una convenzione unica. È una toppa difensiva che compensa un'incoerenza a monte, non un design.

2. I KPI del Concorso

L'area mostra gli indicatori in tempo reale: i partecipanti suddivisi per stato (pending, approved), il volume totale dei voti, una stima dei votanti unici. Su quest'ultima serve un'avvertenza, perché il modulo la calcola «per IP e cookie». Il cookie, però, è una difesa cosmetica (Capitolo 18): si cancella, quindi conta poco come misura di unicità. Il segnale affidabile è uno solo, l'IP nella finestra delle ventiquattr'ore; il «votante unico» va letto come «IP unico», con tutte le imprecisioni del caso (la rete aziendale che collassa molti votanti su un indirizzo solo).

3. Approvazione e Classifica

L'admin vede i partecipanti in una tabella, ascolta la traccia e decide l'esito; l'approvazione è l'unica azione che fa partire l'email di conferma ufficiale. Quella stessa

azione, però, è concessa a chiunque sia loggato, non solo agli amministratori: il gate role-blind del Capitolo 17 vale anche da qui.

La classifica si ordina per `vote_count`, ed è veloce proprio perché legge un contatore già pronto invece di contare i voti. Ne paga però la fragilità: senza una riconciliazione periodica, quel contatore può divergere in silenzio dai voti reali (il box del Capitolo 18 spiega perché, e cosa aggiungere). C'è poi una promessa del testo che il codice non mantiene.

ATTENZIONE

Lo stato `finalist` racconta un piano mai realizzato Il modulo idealizzato parla di «selezionare i finalisti da spostare nel round successivo», come se esistesse uno stato `finalist` che l'admin assegna. Nell'enum dei partecipanti quello stato c'è, ma **nessuna riga di codice lo imposta mai**: le fasi del concorso si gestiscono accendendo il flag booleano `in_current_round` su gruppi di partecipanti, non promuovendoli a uno stato. Quel `finalist` è uno schema che documenta un'intenzione abbandonata, non una funzione. È utile saperlo riconoscere: uno stato che esiste nel database ma che nessuno scrive è un fossile, e descriverlo come attivo confonde il lettore su come gira davvero il concorso.

4. Il Reporting che non è mai stato Acceso

Alla chiusura del voto, dice il modulo idealizzato, il backend invia allo staff un'email di report finale: voti totali, Top 20 dei più votati, qualche statistica geografica sugli IP. La funzione esiste, si chiama `sendVotingReport`, ed è scritta per intero. C'è solo un problema.

ATTENZIONE

Una feature costruita e disabilitata: «Phase 2» `sendVotingReport` è interamente implementata ma **disabilitata nel codice**, commentata con l'etichetta «Phase 2». È vero che il backend *contiene* il report; è falso che lo *invii*. È il gemello dello stato `finalist` del paragrafo precedente: codice presente, funzione dormiente. Presentare una capacità commentata come operante è uno dei modi più facili in cui una documentazione si disallinea dal software: la regola, per chi scrive documentazione tecnica, è descrivere cosa il codice *fa*, non cosa è *pronto a fare se qualcuno lo riattiva*. Finché «Phase 2» resta commentata, il report finale è una promessa, non una funzione.

In sintesi

Il pannello del festival è coerente con la filosofia del modulo, gli interruttori semplici e i numeri essenziali, ma porta le crepe del «fatto a interruttori»: una stima di unicità che si fida di un cookie cosmetico, una classifica che può derivare senza accorgersene, uno stato `finalist` mai usato e un report mai acceso. Sono i punti dove il concorso reale diverge dalla sua versione raccontata, e conoscerli è ciò che distingue chi sa governarlo da chi crede di farlo. La struttura che tiene insieme questo pannello (guardia, layout, backup) resta al Capitolo 14, di cui questa è la versione festival.

IMPORTANTE **Il Canone**

- È l'istanza-festival dell'area admin (Capitolo 14): struttura, guardia e backup vivono lì, qui restano gli interruttori e i KPI del concorso.
- I master switch vanno letti in modo coerente con come sono scritti (niente `'1'` contro `'true'`).
- Mostra KPI onesti (votanti unici per IP, consapevoli del drift di `vote_count`, Capitolo 18).
- Non spacciare per attive le feature costruite ma disabilitate, e toglì gli stati vestigiali (uno stato `finalist` mai impostato).

✧
✧ ✧

Prossimo Capitolo: Social Interactions & Reactions. L'ultima superficie del CMS, dove a scrivere nel database è il pubblico anonimo.

CAPITOLO 20: Social Interactions & Reactions

Quasi tutto, nel CMS, è scrittura riservata: pubblicare un articolo, caricare un file, comporre una newsletter, ognuna di queste azioni vive dietro `Auth::check()`. Ci sono due eccezioni, e questo capitolo parla di loro. Le **reazioni** agli articoli e i **messaggi** del form contatti sono le uniche due superfici in cui un visitatore non autenticato scrive davvero nel database. Sono il fronte più esposto del sito, e proprio per questo concentrano le difese più sofisticate che SimonePizziWebSite abbia costruito.

Una premessa necessaria: le reazioni esistono **solo** in SimonePizziWebSite. SitoRuntime e DISINTELLIGENZA non le hanno; la superficie dei messaggi, invece, ritorna altrove come form contatti. Quindi il capitolo è in larga parte mono-sito, ma la sua lente, *come il thin stack gestisce la scrittura pubblica anonima*, tocca fili che attraversano tutto il libro: l'identità hashata e l'anti-abuso (CAP 10), la sanitizzazione (CAP 8), l'email best-effort (CAP 13), il voto del festival (CAP 18).

E c'è un GOLD che vale la lettura: nello stesso codebase convivono **due filosofie opposte** di sanitizzazione. Il contenuto degli articoli, scritto da un admin di cui ci si fida, è salvato grezzo e ripulito al momento di mostrarlo; il testo dei messaggi, scritto da chiunque passi, è ripulito al momento di salvarlo. Non è una contraddizione: è la scelta giusta per ciascun contesto, e le due si guardano in faccia proprio qui.

**

1. Due superfici, un solo principio: difendere all'ingresso

Sotto le differenze, le due superfici condividono cinque tratti.

Entrambe sono endpoint-router su `REQUEST_METHOD` con un **gate selettivo**: in `messages.php` il `POST` è pubblico (chiunque invia un messaggio), mentre `GET`, `PUT` e `DELETE` (lista, marca-letto, elimina) passano da `Auth::check()`; le reazioni sono interamente pubbliche, e non hanno alcun ramo admin. L'identità di chi scrive è uno **pseudonimo derivato**, non un account né un cookie. L'integrità è affidata al **database**, non solo a una `if` applicativa. L'input pubblico è ripulito **al momento della scrittura**. E tutto degrada con grazia: le reazioni cadono a conteggio zero su qualsiasi errore (la pagina articolo non si rompe mai), e la mail di notifica è best-effort, perché la verità sta nel record salvato, non nell'email partita.

I prossimi paragrafi prendono questi cinque tratti uno per uno, partendo da quello che il modulo dichiara di sé con un po' troppa generosità: l'anonimato.

*
**

2. L'identità anonima, e perché un hash non è anonimato

Il votante non ha un account. Viene identificato da uno pseudonimo calcolato al volo dai due dati che il server vede a ogni richiesta: l'indirizzo IP e lo User-Agent.

```
// SPW reactions.php:25-29 - pseudonimo anonimo, nessun dato personale persi
↳ stito in chiaro
$voter_hash = hash('sha256',
    ($_SERVER['REMOTE_ADDR'] ?? 'unknown') .
    ($_SERVER['HTTP_USER_AGENT'] ?? 'unknown')
);
```

È una buona idea: nel database non finisce un indirizzo IP in chiaro, ma una stringa che non significa nulla per chi la legge. È meglio di quello che fa il voto del festival, che gli IP li salva in chiaro (CAP 18). Ma qui serve una precisazione onesta, perché è facile raccontarla meglio di com'è. Questo hash **non è salato**, e lo spazio degli indirizzi IPv4 è piccolo: poco più di quattro miliardi di valori, che un computer prova tutti in un tempo ridicolo. Combinato con uno User-Agent plausibile, un SHA256 (IP+UA) senza salt si **inverte per forza brutta**. Non protegge l'IP, lo offusca soltanto.

ATTENZIONE

Un hash non è anonimato: è pseudonimizzazione, e ha dei limiti «Lo hashiamo, quindi è anonimo e a norma» è una frase che si sente spesso, e quasi sempre è falsa. Un hash è una funzione deterministica: lo stesso input dà sempre lo stesso output, e se lo spazio degli input è piccolo (gli IP lo sono) chiunque può costruirsi la tabella inversa. Quello che si ottiene è pseudonimizzazione, utile per non avere l'IP in chiaro nel database, non anonimato irreversibile. Per renderlo davvero difficile da invertire serve un **salt segreto** custodito sul server e mai esposto: senza quello, l'hash è un lucchetto con la chiave appesa accanto. È comunque un passo avanti rispetto a salvare l'IP nudo; il problema è solo dichiararlo più di quello che è.

Un dettaglio minore nella stessa direzione: sia le reazioni sia i messaggi leggono il `REMOTE_ADDR` grezzo, non l'helper anti-spoofing `getClientIp()` usato altrove nel sito (CAP 10). Dietro un proxy o una CDN questo può far collassare molti visitatori su uno stesso indirizzo, e rendere il rate-limit del prossimo paragrafo più severo del dovuto.

*
**

3. Il rate-limit a due strati: perché uno solo non basta

Per impedire a uno script di gonfiare i conteggi, le reazioni hanno un limite di frequenza. Il primo strato conta le azioni recenti per `voter_hash`: oltre venti al minuto, la richiesta viene respinta. Sembra sufficiente, ma c'è una crepa, ed è il cuore di questo capitolo: lo `voter_hash` include lo User-Agent, che è un'intestazione **scelta dal client**. Basta cambiarla a ogni richiesta per ottenere un hash diverso ogni volta, e il primo strato non vede mai due azioni dello «stesso» votante.

La versione 1.19.0 ha tappato la crepa con un secondo argine, ancorato a una chiave che il client non controlla: il **solo IP**.

```
// SPW reactions.php:92-119 - due strati, perché il primo è aggirabile
// Strato 1: per voter_hash (IP + UA), max 20/min - ma lo UA lo sceglie il c
↳ client
$stmtRate = $pdo->prepare("SELECT COUNT(*) FROM article_reactions
    WHERE voter_hash = ? AND created_at >= DATE_SUB(NOW(), INTERVAL 1 MINUTE
↳ "); // >= 20 -> 429

// [v1.19.0] Strato 2: per SOLO IP, max 30/min - riusa login_attempts con na
↳ mespace 'rea:'
$rl_key = 'rea:' . substr(hash('sha256', $ip), 0, 40);
$stmtIpRate = $pdo->prepare("SELECT COUNT(*) FROM login_attempts
    WHERE ip_address = ? AND attempt_time >= DATE_SUB(NOW(), INTERVAL 1 MINU
↳ TE"); // >= 30 -> 429
```

Il secondo strato riusa la tabella `login_attempts`, la stessa che difende il login dalla forza bruta (CAP 10) e la newsletter dal mail-bombing (CAP 13), distinguendo gli usi con un prefisso di namespace (`rea:`). Una tabella, tre lavori.

CONSIGLIO

Se la chiave del rate-limit include input del client, serve un secondo strato È un errore facile e diffuso: si sceglie una chiave «forte» per il rate-limit (qui IP più User-Agent, più specifica del solo IP) senza accorgersi che una sua parte è sotto il controllo di chi vuoi limitare. Lo User-Agent lo decide il browser, e un attaccante lo cambia a ogni richiesta con una riga di codice: la chiave diventa nuova ogni volta e il limite non scatta mai. La difesa è affiancare un secondo strato su una chiave che il client non può falsificare a piacimento, come l'IP. I due strati lavorano insieme: il primo è preciso sui casi normali, il secondo regge quando il primo viene aggirato.

✱
✱ ✱

4. L'integrità vive nello schema, non nel codice

Una reazione funziona a interruttore: se non l'hai ancora data, un clic la aggiunge; se ce l'hai già, un altro clic la toglie. Questa logica sta nel codice, ma non è lì che vive la garanzia anti-doppione. Quella sta nello schema della tabella.

```
-- SPW migrate_reactions.php:19-27 - la UNIQUE KEY è la vera barriera anti-d
↳ uplicato
CREATE TABLE IF NOT EXISTS article_reactions (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    article_id  INT NOT NULL,
    reaction    VARCHAR(20) NOT NULL,
    voter_hash  VARCHAR(64) NOT NULL,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unique_vote (article_id, voter_hash, reaction) -- doppio vo
↳ to impossibile per costruzione
);
```

```
// SPW reactions.php:129-145 - il toggle nel codice; l'INSERT IGNORE si appo
↳ ggia alla UNIQUE KEY
if ($existing) { /* DELETE: rimuovi */ }
else { $pdo->prepare("INSERT IGNORE INTO article_reactions (article_id, reac
↳ tion, voter_hash)
    VALUES (?, ?, ?)"->execute([$article_id, $reaction, $voter_hash]); }
```

La differenza è sottile ma decisiva. Un controllo applicativo del tipo «leggi se esiste, poi inserisci» può essere scavalcato da una race condition: due richieste quasi simultanee leggono entrambe «non esiste» e inseriscono entrambe. La `UNIQUE KEY` invece rifiuta il duplicato a prescindere da cosa fa il codice, anche sotto doppio clic o richieste in corsa. È il principio del database come guardiano dell'integrità, ed è esattamente ciò che manca al contatore denormalizzato dei voti del festival (CAP 18), dove l'assenza di un vincolo equivalente lascia spazio al disallineamento.

NOTA

Lascia che sia il database a garantire l'integrità Quando una regola dice «questa combinazione può esistere una volta sola», il posto giusto per imporla è un vincolo nello schema, non un `if` nel codice applicativo. Il vincolo regge sotto concorrenza, sopravvive ai bug del codice che gli gira intorno, e vale per ogni via di scrittura presente e futura. Il codice può occuparsi dell'esperienza (il toggle, il messaggio all'utente); l'integrità, lasciala al motore che la sa difendere davvero.

*
**

5. La seconda superficie: i messaggi, e le due filosofie di sanitizzazione

Le reazioni sono metà del cluster. L'altra metà è `messages.php`, il form contatti. La tabella si crea da sola alla prima chiamata (lo schema viaggia col codice, con un `CREATE TABLE IF NOT EXISTS` e un `ALTER` difensivo), il `POST` è pubblico con il suo rate-limit (tre messaggi per IP ogni quindici minuti, contati sulla propria tabella), e la notifica all'amministratore è una `mail()` nativa fire-and-forget: se l'invio fallisce, il visitatore vede comunque «messaggio inviato», perché il record è già salvato e l'email è solo un canale secondario (CAP 13).

Ma il punto che conta è come viene trattato il testo che arriva da fuori. Viene ripulito **prima** di toccare il database.

```
// SPW messages.php:86-90 - input pubblico ripulito al WRITE-TIME: ciò che è  
entra nel DB è già innocuo  
$name = trim(strip_tags($data['name'] ?? ''));  
$email = trim(filter_var($data['email'] ?? '', FILTER_SANITIZE_EMAIL));  
$subject = trim(strip_tags($data['subject'] ?? ''));  
$message = trim(strip_tags($data['message'] ?? ''));
```

Qui si chiude il filo dell'input pubblico, e lo fa con una polarità rovesciata rispetto agli articoli. Il contenuto di un articolo (CAP 8) è scritto da un admin di cui ci si fida, viene salvato grezzo per non perderne la formatura ricca, e viene ripulito con `DOMPurify` solo **al momento di mostrarlo** (render-time). Il testo di un messaggio arriva da uno sconosciuto, e viene ripulito con `strip_tags` **al momento di salvarlo** (write-time): nel database finisce qualcosa di già privo di tag, e lo stored-XSS è neutralizzato all'origine. C'è pure una seconda rete: il pannello admin mostra il messaggio come text-node React, che riscrive da solo i caratteri speciali, e non usa mai `dangerouslySetInnerHTML`.

IMPORTANTE

Write-time o render-time: dove ripulire l'input dipende da chi te lo manda Nello stesso codebase convivono due strategie opposte, e nessuna delle due è sbagliata. L'input pubblico e non fidato va neutralizzato il prima possibile, alla scrittura: meno cose pericolose tieni nel database, meglio dormi. Il contenuto ricco e fidato (un articolo formattato) va invece preservato così com'è e ripulito dove serve la fedeltà, alla lettura, perché ripulirlo alla scrittura distruggerebbe la formattazione legittima. La domanda non è «write-time o render-time» in astratto, ma «di chi mi fido per questo dato»: la risposta decide il momento. Sbagliare verso non è elegante: salvare grezzo l'input pubblico apre lo stored-XSS, ripulire alla scrittura il contenuto editoriale lo mutila.

Un paio di note di contorno, per onestà. Il `POST` pubblico non ha protezione CSRF, ed è corretto così: non c'è un'azione privilegiata da forgiare, l'unico argine sensato è il rate-limit. E il doppio checkbox di consenso GDPR del form è verificato **solo lato client**: chi invia direttamente all'endpoint lo salta. È coerente con l'idea che il consenso sia esperienza utente e non barriera tecnica, ma va saputo.

*
**

6. Reazione contro voto: tarare l'anti-abuso sulla posta in gioco

Le reazioni di SimonePizziWebSite e il voto del festival di DISINTELLIGENZA (CAP 18) sono, tecnicamente, lo stesso gesto: scrittura pubblica anonima difesa da un'identità hashata e da una barriera anti-doppione. Ma servono a due cose diverse, e la rigidità è tarata di conseguenza. La reazione è libera e plurima: puoi darne più d'una allo stesso articolo, toglierle, rimetterle, e l'anti-abuso è leggero perché la posta in gioco è bassa. Il voto del festival è singolo e sorvegliato: una sola espressione per partecipante, una barriera per IP a finestra di ventiquattr'ore, un interruttore generale che lo chiude del tutto, perché lì si decide una classifica e l'incentivo a barare è reale.

NOTA

Stessa meccanica, due tarature: pesa l'anti-abuso sulla posta in gioco È inutile sorvegliare un «mi piace» come se fosse un'urna elettorale, e pericoloso trattare un voto che assegna un premio con la leggerezza di un like. La stessa cassetta degli attrezzi (identità derivata, vincolo di unicità, rate-limit) si tara su due livelli di rigidità a seconda di quanto costa l'abuso. Capire dove sta la posta in gioco, prima di scrivere il codice anti-frode, è ciò che evita sia la frizione inutile sia la difesa insufficiente.

*
**

7. La micro-interazione: ottimistica, ma con rete

Lato client la barra delle reazioni aggiorna il conteggio e lo stato del pulsante **prima** che il server risponda: il clic sembra istantaneo. Se la richiesta fallisce, l'aggiornamento viene annullato e la UI torna allo stato reale. È l'optimistic UI dei social, con il rollback che la rende onesta. Insieme alla degradazione a conteggio zero vista al §1 (se le reazioni non caricano, l'articolo si legge lo stesso), è il modo in cui un modulo «di contorno» non diventa mai un punto di rottura della pagina.

*
**

In sintesi

La scrittura pubblica anonima è il punto più esposto del CMS, e SimonePizziWebSite lo difende con cura: identità derivata invece dell'IP in chiaro, integrità imposta dallo schema invece che dal codice, rate-limit a due strati invece di uno aggirabile, e input ripulito all'ingresso invece che fidato. La lezione che lega tutto è quella delle due filosofie di sanitizzazione: non esiste un posto giusto in assoluto dove ripulire l'input, esiste la domanda «di chi mi fido», e la risposta cambia il verso della difesa. Resta da ricordare anche cosa il modulo *non* è: l'hash non rende anonimo nessuno, lo offusca; il consenso del form è cortesia, non barriera. Saperlo, e non raccontarsi che il lucchetto è più solido di com'è, è parte della stessa onestà tecnica che attraversa tutto il libro. Qui finisce il giro dentro il CMS: dalle fondamenta del backend fino all'ultimo clic di un visitatore che non sapremo mai chi è.

IMPORTANTE Il Canone

- Per la scrittura pubblica non autenticata, deriva uno pseudonimo con un hash **salato**: un hash di IP+UA senza salt non è anonimato, è reversibile.
- Se la chiave del rate-limit include input del client (lo User-Agent), serve un secondo strato ancorato all'IP; lascia l'integrità allo schema (UNIQUE + INSERT IGNORE).
- Sanitizza l'input pubblico **al write-time** (`strip_tags`) e il contenuto fidato **al render** (DOMPurify): dipende da chi te lo manda.
- Tara l'anti-abuso sulla posta in gioco: una reazione leggera non si difende come un voto sorvegliato.

*
**

Fine della Parte V. Le appendici raccolgono i materiali di servizio: la checklist per partire da zero e il caso del fork (FDCA), il progetto che eredita un intero CMS, debiti di sicurezza compresi.

Appendici

Strumenti pratici e casi di confine.

APPENDICE A - Boilerplate Checklist:

Avvio Nuovo Progetto miniCMS

Questa checklist riassume i passi pratici per inizializzare un nuovo progetto Web (Sito o Web App) basato sugli standard del «Modello Universale miniCMS». Per i dettagli implementativi, fare riferimento ai capitoli indicati. I rimandi (Cap. N) seguono la numerazione della Terza Edizione (20 capitoli + Appendici A e B).

**

Fase 1: Setup Ambiente e Sicurezza Iniziale

- ☐ Creare la struttura base delle cartelle (`public/api/, src/, scripts/`). (Cap. 2)
- ☐ Creare il file `public/.htaccess` per il routing della SPA (React Router). (Cap. 2)
- ☐ Eseguire lo scaffolding del database: cartella `public/api/.data/` con `.htaccess` → Deny from all. (Cap. 2, 3)
- ☐ Creare la cartella media (`public/api/uploads/`) e la cache (`public/api/.cache/`).
- ☐ Configurare `vite.config.ts` con il proxy corretto per evitare CORS in locale. (Cap. 6)
- ☐ Creare `.env.local` con le variabili di sviluppo (`VITE_API_URL=http://localhost/...`).
- ☐ Aggiungere `db_credentials.php` e `.env.local` al `.gitignore` (mai committare credenziali). (Cap. 15)

Fase 2: Configurazione Backend Core (PHP)

- ☐ Implementare `db.php` con connessione lazy a SQLite (`PRAGMA journal_mode=DELETE, busy_timeout=5000, PRAGMA foreign_keys=ON`). **Non** usare il WAL su hosting condiviso. (Cap. 3, 15)
- ☐ Includere l'auto-scaffolding in `db.php`: creazione della cartella `.data/` e dell'`.htaccess` di deny se non esistono. (Cap. 3)
- ☐ Creare `init_db.php` per generare le tabelle e l'utente admin di default con `password_hash()` (e cambiarne subito la password). (Cap. 3, 10)
- ☐ Creare `auth_helper.php` o `auth.php` con la classe `Auth::check()` che gestisce `session_start()` e gli header JSON. (Cap. 5)
- ☐ Configurare i cookie di sessione sicuri (`httponly, samesite=Strict, secure` se HTTPS). (Cap. 10)
- ☐ Applicare `date_default_timezone_set('Europe/Rome')` in **tutti** gli endpoint con logica temporale (l'incoerenza del fuso fa sparire i contenuti programmati). (Cap. 5, 12, 15)
- ☐ Nascondere gli errori in produzione (`display_errors = 0` o try-catch globale): un `die($e->getMessage())` sul fallimento connessione è un leak. (Cap. 10)

Fase 3: Configurazione Frontend Core (React)

- ☐ Implementare `src/api.ts` con la lettura della forma del payload (il pattern «Double Read» = leggere la busta di successo, non clonare la response) per intercettare gli errori del server. (Cap. 6)
- ☐ Configurare `AdminLayout.tsx` con la guardia di auth (route-guard loader o controllo al montaggio) e l'«Hard Logout» (`window.location.reload()`). (Cap. 14)
- ☐ Abilitare la «Role-Based UI» per mostrare solo le voci consentite al ruolo connesso (Admin vs Editor): nascondere la pagina è UX, **bloccare l'azione è sicurezza e va fatto sul server**. (Cap. 10, 14)
- ☐ Usare `key={item.id}` nel componente editor per forzare il reset del rich text editor al cambio articolo. (Cap. 8)

Fase 4: Integrazione Media e Contenuti

- ☐ Implementare `upload.php` con ridimensionamento automatico GD e sanitizzazione dei nomi (`uniqid()`, estensione ricostruita, non quella del client). (Cap. 7)
- ☐ Inserire il componente `MediaPicker` per il caricamento diretto di audio e immagini. (Cap. 8)
- ☐ Usare l'editor (Tiptap) con HTML come fonte di verità salvato grezzo; la protezione contro l'XSS è la sanitizzazione **al render** (DOMPurify), non la pulizia all'incolla, che è solo cosmetica. (Cap. 8)
- ☐ Implementare la logica slug con normalizzazione degli accenti italiani se il contenuto è in italiano. (Cap. 5)

Fase 5: SEO e Syndication

- ☐ Creare `public/index.php` come SEO Engine: query DB → iniezione dei meta tag nell'HTML di Vite (Dynamic Rendering con UA-sniff). (Cap. 11)
- ☐ Aggiungere il rinomina di `index.html` → `index_react.html` nel build script se `index.php` è in `public/`. (Cap. 2, 11)
- ☐ Creare `api/rss.php` con un feed RSS 2.0 valido (header corretto, date RFC 822, URL assoluti, content escapato). (Cap. 12)
- ☐ Aggiungere il tag `<link rel="alternate" type="application/rss+xml">` nell'`<head>` HTML. (Cap. 12)

Fase 6: Ottimizzazione e Deploy

- ☐ Configurare la «Programmazione Reale» tramite query SQL su `published_at` (confronto **nello stesso formato/fuso**, o delegato a `NOW()`). (Cap. 9)
- ☐ Configurare la cache JSON con TTL 300s per le query di listing pesanti. (Cap. 9)
- ☐ Impostare `clean-dist.js` nel processo di build per rimuovere i file `.sqlite` dalla cartella `dist/` (e ricreare a runtime le difese statiche che il build strippa). (Cap. 2, 14)
- ☐ Configurare l'header `Cache-Control: max-age=31536000` per i file in `uploads/` via `.htaccess`. (Cap. 7)

Fase 7: Sicurezza (le reti emerse dai casi reali)

Le voci di questa fase sono le difese che, mancando in almeno uno dei siti mappati, hanno prodotto le falle raccontate nel libro.

- ☐ **Upload con PHP-off:** nella cartella degli upload, un `.htaccess` che disabilita l'esecuzione PHP (prima barriera anti-RCE), più la validazione per magic-bytes (`info`), non per il MIME dichiarato dal client. (Cap. 7)
- ☐ **CSRF a 3 gradini:** token generato server-side, inviato al client e validato su ogni richiesta che muta stato (POST/PUT/DELETE/azioni admin). Un `confirm()` non è una difesa CSRF. (Cap. 10, 14)
- ☐ **Double opt-in newsletter:** due token distinti (conferma e disiscrizione); il link di disiscrizione ha bisogno di un segreto, non basta l'email in chiaro (non è GDPR-compliant). (Cap. 13)
- ☐ **Backup automatico fuori dalla document root,** con rotazione e nome non indovinabile; un cron protetto da segreto timing-safe e fail-closed. Avere lo script d'emergenza non sostituisce il backup. (Cap. 14, 15)
- ☐ **Sanitizzazione server-side condivisa:** i quattro emettitori del `content` (render, prerender SEO, feed RSS, newsletter) devono passare per la stessa pulizia; uno solo che dimentica riapre il buco XSS. (Cap. 8, 11, 12, 13)
- ☐ **Gate per ruolo, non solo per login:** gli endpoint admin verificano `isAdmin`, non solo `isLoggedIn`; le azioni potenti non sono GET senza token. (Cap. 10, 14)

Fase 8: Specifico per Tipologia di Sito

Per Siti con Newsletter (SitoRuntime pattern)

- ☐ Creare la tabella `subscribers` con lo schema **completo** del double opt-in (`email` UNIQUE, `is_active`, `confirmation_token`, `confirmed_at`, `subscribed_at`, `subscribed_ip`, `created_at`): una tabella, un solo CREATE. (Cap. 13, 15)
- ☐ Implementare `newsletter.php` con gate admin + azioni pubbliche (`subscribe`, `confirm`, `unsubscribe`). (Cap. 13)
- ☐ Usare il pattern `{EMAIL_PLACEHOLDER}` per il link di disiscrizione personalizzato con token. (Cap. 13)
- ☐ Distinguere il **throttle** dal **rate-limit**: `usleep(500000)` ogni 10 email regola la cadenza dell'invio (throttle), ma **non** protegge dal mail-bombing; per quello serve un vero rate-limit sull'azione di iscrizione. (Cap. 13)

Per Portfolio/Sito Personale (SimonePizziWebSite pattern)

- ☐ Aggiungere la tabella `projects` con `sort_order`, `is_visible`, `button_a`, `button_b`. (Cap. 16)
- ☐ Implementare `projects.php` con i 5 metodi HTTP, incluso PATCH per il toggle di visibilità e il riordinamento. (Cap. 16)
- ☐ Creare i componenti `PortfolioGrid.tsx`, `ProjectEditor.tsx`, `ProjectsList.tsx`. (Cap. 16)

Per Festival/Concorso (DISINTELLIGENZA / FDCA pattern)

- ☐ Aggiungere le tabelle `participants`, `votes`, `settings` con i master switch `registration_active` e `voting_active`. (Cap. 17, 18)
- ☐ Implementare `participants.php` con il workflow `pending` → `approved/rejected`. (Cap. 17)
- ☐ Implementare `votes.php` con l'anti-frode reale: la barriera è il vincolo **IP + finestra di 24h** (il cookie è solo cosmetico). (Cap. 18)
- ☐ Se il festival nasce come **fork**, mettere in sicurezza il backend da capo: il fork eredita ogni falla, e il fix non lo segue. (Appendice B)

Per Migrazione SQLite → MySQL (SitoRuntime pattern)

- ☐ Fare un backup del `.sqlite` **prima** di toccare il motore, e archivarlo fuori dal repo. (Cap. 15)
- ☐ Creare `db_credentials.php` separato (aggiungere al `.gitignore`). (Cap. 15)
- ☐ Aggiornare `db.php` con la connessione PDO MySQL (`utf8mb4`, `EMULATE_PREPARES=false`, `ATTR_TIMEOUT`). (Cap. 15)
- ☐ Eseguire `init_mysql.php` per creare lo schema MySQL sul server, poi rimuoverlo. (Cap. 15)
- ☐ Eseguire `migrate_to_mysql.php` per il trasloco dati (ONE-SHOT con verifica conteggi, eliminare dopo). (Cap. 15)
- ☐ A migrazione conclusa, **rimuovere i fossili del vecchio motore** dal repo (`init_db.php`, `SQLite`, `fix_*`, `optimize_db.php`, `emergency_revert_wal.php`). (Cap. 15)

**

Questa checklist accompagna la Terza Edizione del Modello Universale miniCMS. Per i dettagli implementativi, fare riferimento ai file `.md` dei capitoli e delle appendici corrispondenti.

APPENDICE B: Ciclo di vita di un fork

Il Modello descrive come si costruisce un sito thin stack. Non dice cosa succede quando lo si **forka**, eppure è un evento reale nella vita di questi progetti, e ne abbiamo un esempio mappato dall'interno: FDCA, il «Festival della Canzone Artificiale», nasce come fork di DISINTELLIGENZA, il «Festival della Disintelligenza Naturale». Stesso autore, stesso motore, tema nuovo: dalla comicità dell'errore umano alla musica generativa. Questa appendice racconta quella fase, perché ha rischi tutti suoi, e il primo è il più silenzioso di tutti.

Il Capitolo 2 ha già introdotto il pattern: due festival con la stessa base funzionale partono da una copia di ciò che già funziona. Il vantaggio è l'indipendenza; il rischio è il suo rovescio esatto. Qui lo guardiamo da vicino.

*

**

1. Il backend si congela, il frontend riparte

La prima cosa che si nota di FDCA è dove è andata l'energia del fork. La cartella `public/api/` è copiata da DISINTELLIGENZA **byte per byte**: stessi 28 file, stessa logica di autenticazione, voto, upload, newsletter, festival, zero differenze di contenuto. Tutto il lavoro nuovo è sul *guscio* pubblico: pagine vetrina riscritte (Home, Filosofia, Manifesto, Edizioni Passate, Contatti), un branding diverso, un cursore animato. È il pattern naturale di un re-skin: si cambia ciò che si vede, non ciò che funziona.

La conseguenza è che la logica di server è ereditata intatta. E «intatta» include ogni cosa, anche le falle.

*

**

2. Il fork eredita il debito: copiare un backend insicuro lo moltiplica

Questo è il punto centrale dell'appendice. Nessuna delle vulnerabilità di sicurezza di DISINTELLIGENZA è stata risolta nel fork, perché il backend è copiato riga per riga. Quello che era un problema in un sito diventa lo stesso identico problema in due.

Vulnerabilità ereditata	Dove è descritta	Stato in FDCA
Catena RCE da upload pubblico (upload no-auth, MIME dal client, nome conservato, niente PHP-off)	CAP 7	presente, immutata
Auth grado-zero (niente CSRF, niente rate-limit, niente protezione da fixation)	CAP 10	presente
Newsletter senza double opt-in né token + header injection	CAP 13	presente
Reset distruttivi senza token CSRF	CAP 10, CAP 14	presente
vote_count denormalizzato (drift della classifica)	CAP 18	presente
Render senza DOMPurify (stored-XSS)	CAP 8	presente

C'è un dettaglio che rende la storia ancora più istruttiva. Sul repository di DISINTELLIGENZA esiste un intervento aperto per chiudere la catena RCE dell'upload (CAP 7). Quel fix vive su DISINTELLIGENZA, e **non tocca FDCA**: il fork ha la sua copia del file vulnerabile, scollegata dall'originale. Il giorno in cui il backend di FDCA andrà online, la RCE sarà replicata, identica, su un secondo dominio, mentre tutti credono di averla risolta.

ATTENZIONE

Il fix non segue il fork Quando si forka copiando il backend, si ereditano le feature e si ereditano le falle, nello stesso gesto. Da quel momento i due rami sono indipendenti: ogni correzione di sicurezza fatta su uno **non** arriva all'altro per magia, va riapplicata a mano. È facile dimenticarlo, perché il fork sembra «un altro progetto», con un altro nome e un'altra versione. Ma sotto è lo stesso codice, e una falla chiusa in un ramo resta spalancata nell'altro finché qualcuno non la chiude di nuovo, di proposito. Prima di forkare un backend, conviene avere un elenco delle sue vulnerabilità note proprio per non perderlo di vista dopo la copia.

✱
✱ ✱

3. Il guscio scollegato

C'è una seconda sorpresa in FDCA: il frontend nuovo **non parla con il backend ereditato**. Non c'è un `src/api.ts`, non c'è una sola chiamata `fetch` verso `/api/`. Le pagine vetrina sono marketing puro, non connesso al CMS che gira dietro. FDCA, allo stato, è un sito vetrina con dietro un CMS pieno e funzionante ma irraggiungibile dalla sua stessa applicazione.

Non è un errore: è una fase. Prima la pelle, poi (forse) il ricablaggio. È lo scollamento tipico di un restyle in corso d'opera, e di per sé non fa danni, perché un backend che nessuno chiama non espone nulla. Il rischio è latente e differito: il giorno in cui il frontend verrà connesso, erediterà *anche operativamente* i comportamenti, e i bug, del backend di DISINTELLIGENZA. Lo scollamento di oggi è anche la ragione per cui le falle del §2 non sono ancora sfruttabili end-to-end, e per cui è facile dimenticarsene.

✱
✱ ✱

4. La versione che riparte da zero su codice che non riparte

FDCA si presenta come **v0.0.1**. Il `package.json` dice 0.0.1, il `metadata.json` parla di «Remix», il README è quello generato da Google AI Studio. Tutto comunica un prodotto nuovo, appena nato. Sotto, però, gira un backend a v0.5.x con tutto il suo debito accumulato.

ATTENZIONE

La versione racconta una nascita, il codice racconta un'eredità La discontinuità di versione *nasconde* la continuità del debito. È il rovescio del problema delle stringhe di versione divergenti che si vede in DISINTELLIGENZA (sidebar, init e package indicano tre numeri diversi): lì troppe versioni per un solo codice, qui una versione-zero per un codice tutt'altro che zero. In entrambi i casi il numero di versione mente sullo stato reale del software. Quando vedi un 0.0.1, non dare per scontato che sotto ci sia poco: guarda il codice, non l'etichetta.

**

5. Il fork che si scrive la roadmap da solo

FDCA include una cartella `ROADMAP-EVOLUZIONE-minicms` con nove capitoli, generati con assistenza AI, che pianificano di ricostruire il miniCMS in modo pulito. I titoli di quei capitoli **ricalcano i cluster** di questa stessa mappatura: Backend, API PHP, Frontend Bridge, Admin & Protected Routes, Festival Engine. È un artefatto prezioso, perché conferma dall'interno la struttura che questo manuale ricostruisce dall'esterno: la spina dorsale del thin stack è riconoscibile anche a chi ci lavora dentro, al punto da volerla riscrivere ordinata. Un progetto che documenta l'intenzione di evolvere il proprio motore è la prova che il pattern esiste ed è leggibile.

**

6. Un motore, due festival

C'è anche un lato luminoso del forking, e FDCA lo dimostra. Il modello festival (iscrizione, selezione, voto a turni, master switch di registrazione e voto, classifica derivata dal `vote_count`) è **lo stesso** nei due siti: cambia solo il tema, dall'errore umano alla musica generativa. È la prova positiva che il modulo concorso descritto ai Capitoli 17 e 18 è un componente davvero riusabile: lo stesso engine regge due festival diversi con un semplice re-skin.

Il forking, insomma, non è un male in sé. Riusare un dominio che funziona cambiando la pelle è efficiente e legittimo. Diventa un problema solo quando il re-skin si dimentica del motore: quando si copia il backend e si trascura che dentro quella copia c'erano anche le falle, e che da quel momento è responsabilità del fork tenerle chiuse.

**

In sintesi

Forkare un progetto thin stack è una fase reale e comune, e ha una sua fisiologia: il backend si congela, il frontend riparte, la versione si azzera, il tema cambia. Il rischio specifico è uno solo, ma è serio: l'eredità silenziosa del debito. Un fork che copia il backend verbatim si porta dietro ogni vulnerabilità, immutata, mentre la versione-zero e il nome nuovo raccontano un prodotto pulito. Il fix non segue il fork: va riapplicato a mano, su un ramo che ormai vive di vita propria. La regola, se si forka, è tenere insieme due liste: quella delle feature da riusare e quella delle falle da non ereditare. La prima è il motivo per cui si forka; la seconda è la ragione per cui un fork va trattato come un progetto da mettere in sicurezza da capo, non come una copia già a posto.

APPENDICE C: Testing e Deploy (cenni)

Due temi che il corpo del manuale tocca solo di sfuggita meritano almeno un punto di partenza: come si collauda un thin stack senza un framework che ti regali gli strumenti, e come lo si porta in produzione su un hosting economico. Sono cenni, non una guida completa: indicano la direzione e i tranelli principali, non ogni passo. Ma servono a smentire un equivoco, e cioè che «niente framework» voglia dire «niente test» e «deploy a sentimento».

**
**

1. Testing: niente framework non vuol dire niente test

Il modello rinuncia ai framework, non alla verifica. Anzi: la sua forma minimale rende alcuni test più facili, non più difficili.

Il backend PHP. La logica pura (slug, validazione, sanitizzazione, calcolo della visibilità) si collauda con PHPUnit come qualsiasi codice. Gli endpoint, invece, conviene provarli *funzionalmente*: una richiesta HTTP vera contro l'endpoint, e una verifica su status e forma del payload. Qui il database-a-file diventa un vantaggio inatteso. Invece di mockare PDO (laborioso e poco fedele), si apre un SQLite usa-e-getta, in memoria o su file temporaneo, lo si popola con dati noti e lo si butta a fine test.

```
// Un endpoint si collauda meglio con un DB reale effimero che con un mock d
↳ i PDO
$pdo = new PDO('sqlite::memory:'); // database di test, vive quanto i
↳ l test
$pdo->exec(file_get_contents('schema_test.sql')); // schema noto
// poi: chiama l'endpoint (via HTTP o includendo il file con $_GET/$_POST si
↳ mulati)
// e verifica status code + struttura JSON della risposta
```

Il frontend. Vitest e Testing Library coprono i componenti React. Il punto di forza è l'oggetto `api` del Capitolo 6: essendo un canale unico su `fetch`, si mocka in un punto solo, e i test dei componenti non toccano mai la rete vera.

I test che ripagano di più. In un sistema così, due famiglie di test valgono più di mille asserzioni di dettaglio:

- **Smoke test del contratto.** Un test che chiama ogni endpoint pubblico e verifica status più forma del payload coglie quasi tutte le regressioni del «contratto instabile» discusso al Capitolo 6. È la rete che manca proprio dove l'API non ha uno schema formale.
- **Test di non-regressione sulla sicurezza.** Un endpoint admin deve rispondere 401 o 403 senza sessione; un file `.php` caricato nella cartella upload non deve essere

eseguibile (Capitolo 7); una mutazione senza token CSRF deve fallire (Capitolo 10). Sono i controlli che impediscono a una falla già chiusa di riaprirsi in silenzio.

*
**

2. Deploy: dalla `dist/` all'hosting da cinque euro

Il modello nasce per girare su hosting condivisi economici, e il deploy è volutamente semplice: niente container, niente orchestratori. Ma «semplice» non vuol dire «improvvisato».

Cosa produce la build. `npm run build` genera la cartella `dist/` con gli asset statici di React. Due passi del processo di build, già visti nei Capitoli 2 e 11, sono difese, non dettagli: `clean-dist.js` rimuove i file `.sqlite` di sviluppo dalla distribuzione (un database di sviluppo spedito in produzione è un disastro), e se `index.php` vive in `public/` va rinominato `index.html` in `index_react.html`, perché l'entry-point PHP del SEO Engine deve avere la precedenza.

Cosa si carica, e cosa no. Sul server vanno la `dist/` compilata e la cartella `public/api/` con il PHP. Il file `db_credentials.php` (per i siti MySQL) si carica **a mano**, una volta, e non sta mai nel repo. Non si caricano mai la cartella `.data/` di sviluppo né i file di test.

Come si carica. Le tre vie, in ordine di robustezza: un `git pull` sul server se l'hosting lo consente (la più pulita); un deploy via SFTP scriptato; il caricamento FTP manuale (il più fragile, perché dimenticare un file è facile). Qualunque sia la via, conviene che il deploy sia *ripetibile*: uno script in `scripts/` che fa sempre gli stessi passi vale più di una procedura tenuta a memoria.

Una CI minima. Anche senza pipeline complesse, una sola GitHub Action che a ogni push esegue build, lint e i test della sezione precedente intercetta le regressioni prima che arrivino in produzione. I segreti (credenziali, chiavi) vivono nelle variabili d'ambiente del CI, mai nel repository. Un eventuale passo di deploy automatico via FTP/SFTP è un'aggiunta comoda, ma viene dopo: prima la rete dei test, poi l'automazione della consegna.

*
**

3. Il limite di questi cenni

Quanto sopra è una mappa, non il territorio. Un sito con requisiti di disponibilità seri vorrà ambienti separati (staging e produzione), migrazioni di schema versionate (il debito del Capitolo 15), monitoraggio degli errori e backup verificati, non solo eseguiti. Il thin stack non vieta nulla di tutto questo: semplicemente non te lo regala, e sta a te decidere quanto di quella disciplina il tuo progetto merita. La regola è la stessa del resto del libro: parti dal minimo che ti tiene al sicuro, e aggiungi solo quando un bisogno concreto lo chiede.

IMPORTANTE
Il Canone

- Collauda gli endpoint con un SQLite effimero (`:memory:` o file temporaneo), non mockando PDO.
- Mocka l'oggetto `api` in un punto solo per i test dei componenti React.
- Tieni due reti che ripagano: smoke test del contratto (status + forma del payload) e test di non-regressione sulla sicurezza (gate, upload, CSRF).
- Build con `clean-dist` (via i `.sqlite`); carica `dist/` + `public/api/`, mai `db_credentials.php` dal repo né la `.data/` di sviluppo.
- Una CI minima (build + lint + test a ogni push), con i segreti nelle variabili d'ambiente, non nel codice.

L'autore

Simone Pizzi è autore ed editore. Ha fondato Runtime Edizioni e Runtime Radio, per cui cura progetti editoriali, sonori e digitali. Ha pubblicato la raccolta di racconti «L'Albero dei Racconti» e la novella di fantascienza «Frequenza di Servizio».

Questo manuale nasce dal lavoro reale su quattro siti in produzione — SitoRuntime, DISINTELLIGENZA, FDCA e SimonePizziWebSite — e raccoglie il protocollo «thin stack» con cui li ha costruiti: React e TypeScript per la presentazione, PHP nativo e SQLite o MySQL per i dati, senza framework di backend né infrastruttura sproporzionata.

